LeetCode 热题 100

分类题解手册 • 完整 100 题

题目 · 思路 · 代码 · 复杂度 · 核心知识点

面向初学者 · 零基础也能读懂

100	17	20	80
题目总数	算法分类	简单题	中等 + 困难

整理日期：2026 年 09 月 14 日

使用指南

本手册按「算法分类」组织 LeetCode 热题 100 的全部 100 道题目。每个分类开头有一段该算法的核心思想概述，每道题都包含固定的五个部分，方便你按需查阅：

- 题目描述 —— 用通俗的语言复述题目要求，并给出输入输出示例；
- 解题思路 —— 从「最朴素的解法」讲到「最优解法」，一步一步说明为什么这样做是对的；
- 参考代码 —— 完整的 Python 实现（LeetCode 可直接提交），带详细中文注释；
- 复杂度分析 —— 时间与空间复杂度，并说明复杂度是怎么得来的；
- 核心知识点 —— 这道题真正要你掌握的能力点，以及可以横向对比的同类题目。

建议的学习路径：

- 第 1 步：先看「解题思路」，理解为什么需要这个算法，不要急着看代码；
- 第 2 步：自己尝试写出代码；写不出来再看「参考代码」，重点看注释里的关键步骤；
- 第 3 步：合上答案，凭记忆重写一遍，并想一想「如果题目条件变了，解法要怎么改」；
- 第 4 步：把「核心知识点」里的同类题一起做掉，形成一个知识模块。

阅读提示：文中代码均为 Python 3 实现。部分题目给出了多种解法（例如「方法一 / 方法二」），建议先掌握第一种最直观的，再理解后续的优化版本。标注「进阶」「拓展」的内容为加分项，第一遍学习时可以跳过。

分类总览

热题 100 共分为 17 个算法分类，各类别题量与题号如下：

算法分类	题量	包含题目编号
哈希	3 题	1、49、128
双指针	4 题	283、11、15、42
滑动窗口	2 题	3、438
子串	3 题	560、239、76
普通数组	5 题	53、56、189、238、41
矩阵	4 题	73、54、48、240
链表	14 题	160、206、234、141、142、21、2、19、24、25、138、148、23、146
二叉树	15 题	94、104、226、101、543、102、108、98、230、199、114、105、437、236、124
图	4 题	200、994、207、208
回溯	8 题	46、78、17、39、22、79、131、51
二分查找	6 题	35、74、34、33、153、4
栈	5 题	20、155、394、739、84
堆（优先队列）	3 题	215、347、295
贪心算法	4 题	121、55、45、763
动态规划	10 题	70、118、198、279、322、139、300、152、416、32
多维动态规划	5 题	62、64、5、1143、72
技巧	5 题	136、169、75、31、287
合计	**100 题**	简单 20 题 · 中等 68 题 · 困难 12 题

目 录

（在 Word 中选中本段按 F9，或右键选择“更新域”即可自动生成带页码的目录）

第 1 章　哈希　(3 题)

第 2 章　双指针　(4 题)

第 3 章　滑动窗口　(2 题)

第 4 章　子串　(3 题)

第 5 章　普通数组　(5 题)

第 6 章　矩阵　(4 题)

第 7 章　链表　(14 题)

第 8 章　二叉树　(15 题)

第 9 章　图　(4 题)

第 10 章　回溯　(8 题)

第 11 章　二分查找　(6 题)

第 12 章　栈　(5 题)

第 13 章　堆（优先队列）　(3 题)

第 14 章　贪心算法　(4 题)

第 15 章　动态规划　(10 题)

第 16 章　多维动态规划　(5 题)

第 17 章　技巧　(5 题)

题目总览（100 题索引）

序号	所属分类	题号	题目名称	难度
1	哈希	1	两数之和	简单
2	哈希	49	字母异位词分组	中等
3	哈希	128	最长连续序列	中等
4	双指针	283	移动零	简单
5	双指针	11	盛最多水的容器	中等
6	双指针	15	三数之和	中等
7	双指针	42	接雨水	困难
8	滑动窗口	3	无重复字符的最长子串	中等
9	滑动窗口	438	找到字符串中所有字母异位词	中等
10	子串	560	和为 K 的子数组	中等
11	子串	239	滑动窗口最大值	困难
12	子串	76	最小覆盖子串	困难
13	普通数组	53	最大子数组和	中等
14	普通数组	56	合并区间	中等
15	普通数组	189	轮转数组	中等
16	普通数组	238	除自身以外数组的乘积	中等
17	普通数组	41	缺失的第一个正数	困难
18	矩阵	73	矩阵置零	中等
19	矩阵	54	螺旋矩阵	中等
20	矩阵	48	旋转图像	中等
21	矩阵	240	搜索二维矩阵 II	中等
22	链表	160	相交链表	简单
23	链表	206	反转链表	简单
24	链表	234	回文链表	简单
25	链表	141	环形链表	简单
26	链表	142	环形链表 II	中等
27	链表	21	合并两个有序链表	简单

序号	所属分类	题号	题目名称	难度
28	链表	2	两数相加	中等
29	链表	19	删除链表的倒数第 N 个结点	中等
30	链表	24	两两交换链表中的节点	中等
31	链表	25	K 个一组翻转链表	困难
32	链表	138	随机链表的复制	中等
33	链表	148	排序链表	中等
34	链表	23	合并 K 个升序链表	困难
35	链表	146	LRU 缓存	中等
36	二叉树	94	二叉树的中序遍历	简单
37	二叉树	104	二叉树的最大深度	简单
38	二叉树	226	翻转二叉树	简单
39	二叉树	101	对称二叉树	简单
40	二叉树	543	二叉树的直径	简单
41	二叉树	102	二叉树的层序遍历	中等
42	二叉树	108	将有序数组转换为二叉搜索树	简单
43	二叉树	98	验证二叉搜索树	中等
44	二叉树	230	二叉搜索树中第 K 小的元素	中等
45	二叉树	199	二叉树的右视图	中等
46	二叉树	114	二叉树展开为链表	中等
47	二叉树	105	从前序与中序遍历序列构造二叉树	中等
48	二叉树	437	路径总和 III	中等
49	二叉树	236	二叉树的最近公共祖先	中等
50	二叉树	124	二叉树中的最大路径和	困难
51	图	200	岛屿数量	中等
52	图	994	腐烂的橘子	中等
53	图	207	课程表	中等
54	图	208	实现 Trie（前缀树）	中等
55	回溯	46	全排列	中等
56	回溯	78	子集	中等

序号	所属分类	题号	题目名称	难度
57	回溯	17	电话号码的字母组合	中等
58	回溯	39	组合总和	中等
59	回溯	22	括号生成	中等
60	回溯	79	单词搜索	中等
61	回溯	131	分割回文串	中等
62	回溯	51	N 皇后	困难
63	二分查找	35	搜索插入位置	简单
64	二分查找	74	搜索二维矩阵	中等
65	二分查找	34	在排序数组中查找元素的第一个和最后一个位置	中等
66	二分查找	33	搜索旋转排序数组	中等
67	二分查找	153	寻找旋转排序数组中的最小值	中等
68	二分查找	4	寻找两个正序数组的中位数	困难
69	栈	20	有效的括号	简单
70	栈	155	最小栈	中等
71	栈	394	字符串解码	中等
72	栈	739	每日温度	中等
73	栈	84	柱状图中最大的矩形	困难
74	堆（优先队列）	215	数组中的第 K 个最大元素	中等
75	堆（优先队列）	347	前 K 个高频元素	中等
76	堆（优先队列）	295	数据流的中位数	困难
77	贪心算法	121	买卖股票的最佳时机	简单
78	贪心算法	55	跳跃游戏	中等
79	贪心算法	45	跳跃游戏 II	中等
80	贪心算法	763	划分字母区间	中等
81	动态规划	70	爬楼梯	简单
82	动态规划	118	杨辉三角	简单
83	动态规划	198	打家劫舍	中等
84	动态规划	279	完全平方数	中等
85	动态规划	322	零钱兑换	中等

序号	所属分类	题号	题目名称	难度
86	动态规划	139	单词拆分	中等
87	动态规划	300	最长递增子序列	中等
88	动态规划	152	乘积最大子数组	中等
89	动态规划	416	分割等和子集	中等
90	动态规划	32	最长有效括号	困难
91	多维动态规划	62	不同路径	中等
92	多维动态规划	64	最小路径和	中等
93	多维动态规划	5	最长回文子串	中等
94	多维动态规划	1143	最长公共子序列	中等
95	多维动态规划	72	编辑距离	中等
96	技巧	136	只出现一次的数字	简单
97	技巧	169	多数元素	简单
98	技巧	75	颜色分类	中等
99	技巧	31	下一个排列	中等
100	技巧	287	寻找重复数	中等

第 1 章 哈希

本章核心思想

哈希表（Hash Table / 字典）的核心价值是「用空间换时间」：把数据存进去以后，查找某个元素只需要 O(1) 的时间。凡是题目里出现「查找某个值是否存在」「统计出现次数」「判断元素是否重复」这类需求，第一反应就应该是哈希表。Python 里用 dict（键值对）或 set（只存键）来实现。

1. 两数之和 【简单】

题目描述

给定一个整数数组 nums 和一个整数目标值 target，请你在该数组中找出和为目标值 target 的那两个整数，并返回它们的数组下标。你可以假设每种输入只会对应一个答案，且同一个元素在答案里不能重复出现。答案可以按任意顺序返回。

示例：

```
输入：nums = [2,7,11,15], target = 9
输出：[0,1]
解释：nums[0] + nums[1] = 2 + 7 = 9
```

解题思路

最直观的做法是两层循环枚举所有数对，但时间复杂度是 $O(n^2)$，太慢。

关键思路：对于当前遍历到的数字 x，我们其实需要的不是「另一个数字在哪里」，而是「target - x 在不在数组里、它的下标是多少」。这正是哈希表最擅长的：O(1) 判断某个值是否存在。

具体步骤：

1. 建一个空字典 seen，用来记录「已经遍历过的数字 → 它的下标」。

2. 从左到右遍历数组，设当前数字为 x。

3. 先算它需要的搭档 need = target - x，检查 need 是否已经在 seen 里。

4. 如果在，说明前面出现过这个搭档，直接返回 [seen[need], 当前下标]。

5. 如果不在，就把当前的 x 和它的下标存进 seen，继续往后走。

注意顺序：先查再存。这样能保证不会把自己和自己配对（因为存进去的时候还没查过自己）。

参考代码

```
def twoSum(nums, target):
    seen = {}                       # 记录 {数值: 下标}
    for i, x in enumerate(nums):
        need = target - x           # 我需要的搭档
        if need in seen:            # 搭档之前出现过吗?
            return [seen[need], i]
        seen[x] = i                 # 没找到就把自己存进去
    return []                       # 题目保证有解，走不到这里
```

复杂度分析

时间复杂度 O(n)：只需遍历一次数组，每次哈希查找都是 O(1)。空间复杂度 O(n)：最坏情况下要把 n 个数都存进字典。

核心知识点

- 哈希表：以 O(1) 的平均代价完成「查找元素是否存在」，这是本题提速的根本。
- 空间换时间：多用一个字典，把 $O(n^2)$ 的双层循环降到 O(n) 的单层循环。
- 先查后存：这个顺序保证了不会重复使用同一个元素。
- 返回值是「下标」而不是「数值」，所以字典要存 值→下标 的映射。

49. 字母异位词分组 【中等】

题目描述

给你一个字符串数组，请你将「字母异位词」组合在一起。字母异位词指由相同字母重新排列形成的字符串（例如 "ate" 和 "tea"）。可以按任意顺序返回结果列表。

示例：

输入：strs = ["eat","tea","tan","ate","nat","bat"]

输出：[["bat"],["nat","tan"],["ate","eat","tea"]]

解题思路

题目的难点在于：怎么判断两个单词是「同一组」？字母异位词的共同特征是——它们包含的字母种类和数量完全一样。

两种常用的「统一表示法」：

方法一（排序）：把单词的字母排序，异位词排序后结果一定相同。例如 eat/tea/ate 排序后都是 "aet"。

方法二（计数）：统计 26 个字母各出现多少次，拼成一个长度 26 的计数串作为键。

具体步骤（用排序法）：

1. 建立一个字典 groups，键是「归一化后的字符串」，值是「属于这一组的原单词列表」。

2. 遍历每个单词 word，把它排序得到 key = ''.join(sorted(word))。

3. 把 word 追加到 groups[key] 里（这一组不存在就自动新建）。

4. 最后返回 groups 的所有值。

本质是把「判断两个单词是否同组」这个难题，转化成「它们有没有同一个键」这个简单问题，而字典正好按这个键来分组。

参考代码

```
from collections import defaultdict

def groupAnagrams(strs):
    groups = defaultdict(list)          # 键: 归一化串, 值: 同组单词列表
    for word in strs:
        key = ''.join(sorted(word))     # ['a','e','t'] -> 'aet'
        groups[key].append(word)
    return list(groups.values())

# 方法二：计数作为键（当单词很长时更快，省掉排序）
def groupAnagrams2(strs):
    groups = defaultdict(list)
    for word in strs:
        cnt = [0] * 26
        for ch in word:
            cnt[ord(ch) - ord('a')] += 1
        key = tuple(cnt)                # 长度 26 的元组作为哈希键
        groups[key].append(word)
    return list(groups.values())
```

复杂度分析

时间复杂度 O(n·k log k)，n 是单词个数，k 是最长单词长度（排序开销）。若改用计数法则是 O(n·k)。

空间复杂度 O(n·k)，用于存储所有单词。

核心知识点

- 哈希分组：字典的「键」用来表达类别，同一个键就自动归到一组，这是极其常用的套路。
- 归一化思想：把形式不同但本质相同的东西，转换成同一个标准表示（排序后 / 计数向量），再拿去做比较或作为键。
- defaultdict(list)：Python 里做分组统计的标准写法，省去「键不存在要先建空列表」的判断。
- 元组可哈希：列表不能作为字典的键，但 tuple 可以，所以计数法里要转成 tuple。

128. 最长连续序列 【中等】

题目描述

给定一个未排序的整数数组 nums，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。请你设计并实现时间复杂度为 O(n) 的算法解决此问题。

> **示例：**
>
> ```
> 输入：nums = [100,4,200,1,3,2]
> 输出：4
> 解释：最长连续序列是 [1,2,3,4]，长度为 4。
> ```

解题思路

如果先排序再扫一遍，需要 O(n log n)，不满足要求。要求 O(n)，就要利用哈希集合的去重和 O(1) 查找。

核心观察：一个连续序列一定有个起点，而「起点」的判定标准是——它的前一个数 x-1 不在集合里。否则它就不是起点，可以从别的数往后数到它。

具体步骤：

1. 把所有数字放进集合 S（自动去重，同时支持 O(1) 判断某数是否存在）。

2. 遍历集合中的每个数 x，先判断 x-1 是否在 S 中。如果在，说明 x 不是序列起点，直接跳过。

3. 如果 x-1 不在，说明 x 是某段连续序列的起点，就从 x 开始不断尝试 x+1、x+2……直到断掉，统计长度。

4. 用全局变量记录最大长度。

为什么这样是 O(n)？因为每个数字最多只会被「作为序列内部元素」访问一次（只有起点会触发往后的扫描），总访问次数是 O(n)。

参考代码

```
def longestConsecutive(nums):
    s = set(nums)
    best = 0
    for x in s:
        if x - 1 in s:          # 不是起点，跳过，避免重复扫描
            continue
        y = x                   # 从起点开始往后数
        while y + 1 in s:
            y += 1
        best = max(best, y - x + 1)
    return best
```

复杂度分析

时间复杂度 O(n)：虽然有 while 循环，但每个元素只会被数到一次，总体仍是线性。空间复杂度 O(n)：集合存储所有元素。

核心知识点

- 哈希集合 set：兼有「去重」和「O(1) 查询」两个能力，是处理「元素是否出现过」类问题的利器。
- 找起点 + 顺延：只在起点处展开扫描，避免了对同一段序列的重复计算——这是把看似 $O(n^2)$ 降为 O(n) 的关键。
- 均摊复杂度分析：内层 while 的累计次数不超过 n，所以要按「整体」而不是「单次循环」来估算复杂度。
- 不要求原数组连续，只要求数值连续，因此可以用集合打散原顺序。

第 2 章 双指针

本章核心思想

双指针不是某种数据结构，而是一种「用两个位置变量同时移动来缩小搜索范围」的解题技巧。常见形态有三种：左右指针从两端向中间靠拢（适合有序数组、盛水问题）、快慢指针同向移动（适合原地删除、去重）、以及滑动窗口（下一章单独讲）。识别信号：数组/字符串、要求原地操作、或者有序性可以利用。

283. 移动零 【简单】

题目描述

给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。请注意，必须在不复制数组的情况下原地对数组进行操作。

示例：

```
输入：nums = [0,1,0,3,12]
输出：[1,3,12,0,0]
```

解题思路

「原地 + 保持相对顺序」是快慢指针的典型场景。

思路：把快指针 fast 当作「读指针」，慢指针 slow 当作「写指针」。fast 一路扫描所有元素，只要遇到非零数，就把它写到 slow 的位置，然后 slow 前进一格。这样所有非零数会被依次、按原顺序地搬到数组前半部分。

具体步骤：

1. slow = 0，表示下一个非零数应该放的位置。

2. fast 从 0 遍历到 n-1。

3. 若 nums[fast] != 0，就交换 nums[slow] 和 nums[fast]，然后 slow += 1。

4. 遍历结束后，[0, slow) 全是非零数（且相对顺序不变），[slow, n) 全是 0。

为什么用「交换」而不是「赋值」？交换可以一步到位把 0 换到后面，不需要额外的清尾步骤。

而且当 slow == fast 时交换是空操作，不影响正确性。

参考代码

```
def moveZeroes(nums):
    slow = 0                        # 写指针：下一个非零元素该放的位置
    for fast in range(len(nums)):  # 读指针：扫描全部元素
        if nums[fast] != 0:
            nums[slow], nums[fast] = nums[fast], nums[slow]
            slow += 1
    return nums
```

复杂度分析

时间复杂度 O(n)：只遍历一次数组。空间复杂度 O(1)：只用了两个下标变量，原地修改。

核心知识点

- 快慢指针 / 读写指针：用两个下标区分「扫描位置」和「写入位置」，是原地数组操作的通用范式。
- 原地算法（in-place）：只用常数级额外空间完成修改，常见于数组、链表题。
- 保持相对顺序：非零元素按扫描顺序被依次写入，天然有序。
- Python 的多元赋值可以一行完成交换，底层是先构造元组再解包。

11. 盛最多水的容器 【中等】

题目描述

给定一个长度为 n 的整数数组 height，第 i 条线的两个端点是 (i, 0) 和 (i, height[i])。找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。返回容器可以储存的最大水量。

> **示例：**
>
> 输入：height = [1,8,6,2,5,4,8,3,7]
>
> 输出：49
>
> 解释：选下标 1（高 8）和下标 8（高 7），宽度 7，高度取较小值 7，面积 = 7×7 = 49。

解题思路

容器的水量公式：面积 = 两线的间距（宽）× 两条线中较矮的那条的高度（高）。

暴力枚举所有线对是 $O(n^2)$。用左右指针从两端向中间收缩，可以做到 $O(n)$。

关键推理：一开始左右指针在两端，此时宽度最大。面积由「较矮的那条线」决定。如果移动较高的那条线，宽度变小了，而高度上限依然是那条矮线，面积不可能变大。所以只有移动「较矮的那条线」，才有可能让高度提升，从而让面积变大。这就是贪心式的收缩依据。

具体步骤：

1. left = 0，right = n-1，记录最佳答案 ans = 0。

2. 当 left < right 时：宽 = right - left，高 = min(height[left], height[right])，更新 ans。

3. 比较两边高度，把较矮的那一侧指针向内移动一格（相等时移动哪边都可以）。

4. 重复直到两指针相遇。

参考代码

```
def maxArea(height):
    left, right = 0, len(height) - 1
    ans = 0
    while left < right:
        h = min(height[left], height[right])   # 容器高度由矮的一侧决定
        ans = max(ans, h * (right - left))     # 更新最大面积
        if height[left] < height[right]:       # 只移动矮的那一侧
            left += 1
        else:
            right -= 1
    return ans
```

复杂度分析

时间复杂度 $O(n)$：左右指针一共移动 n-1 次。空间复杂度 $O(1)$：只用了常数个变量。

核心知识点

- 对撞双指针：从两端向中间收缩，每轮淘汰一个不可能成为答案的位置，把 $O(n^2)$ 降为 $O(n)$。
- 贪心思想的体现：每一步都做出局部最优选择（移动矮边），并能证明它不会错过全局最优。
- 面积由短板决定：这是「木桶效应」的直观应用。
- 正确性证明思路：宽度只会越来越小，所以只有提高「高」才可能更优，而只有移矮边才可能提高高。

15. 三数之和 【中等】

题目描述

给你一个整数数组 nums，判断是否存在三元组 [nums[i], nums[j], nums[k]] 满足 i != j、i != k 且 j != k，同时还满足 nums[i] + nums[j] + nums[k] == 0。请返回所有和为 0 且不重复的三元组。答案中不可以包含重复的三元组。

> **示例：**
>
> 输入：nums = [-1,0,1,2,-1,-4]
> 输出：[[-1,-1,2],[-1,0,1]]

解题思路

三个数的暴力枚举是 O(n³)。经典降维思路：先固定一个数，问题就变成「在剩下的数里找两个数之和等于目标值」，而这就是两数之和，可以用双指针在 O(n) 内解决。

具体步骤：

1. 先把数组排序。排序是双指针能够工作的前提（有序才能靠比较决定移动哪边）。

2. 枚举第一个数 nums[i]。为了去重，如果 i > 0 且 nums[i] == nums[i-1]，就跳过。

3. 设 left = i+1，right = n-1，目标值是 -nums[i]。

4. 计算三数之和 s = nums[i] + nums[left] + nums[right]：

   - 若 s == 0：记录答案；然后 left++、right--，并且跳过左右两侧所有重复值。

   - 若 s < 0：说明总和偏小，需要更大的数，left++。

   - 若 s > 0：说明总和偏大，需要更小的数，right--。

5. 当 left >= right 时，说明第一个数为 nums[i] 的情况找完了，i 进入下一轮。

去重是本题的易错点，务必在三处处理：i 的重复、left 的重复、right 的重复。

参考代码

```
def threeSum(nums):
    nums.sort()
    n = len(nums)
    res = []
    for i in range(n - 2):
        if nums[i] > 0:                  # 最小的数已经 > 0，后面不可能有解
            break
        if i > 0 and nums[i] == nums[i - 1]:   # 跳过重复的第一个数
            continue
        left, right = i + 1, n - 1
        target = -nums[i]
        while left < right:
            s = nums[left] + nums[right]
            if s == target:
                res.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1                  # 跳过重复的左值
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1                 # 跳过重复的右值
            elif s < target:
                left += 1                      # 总和太小，左指针右移
            else:
                right -= 1                     # 总和太大，右指针左移
    return res
```

复杂度分析

时间复杂度 $O(n^2)$：排序 $O(n \log n)$，外层枚举 n 次，每次双指针 $O(n)$。空间复杂度 $O(1)$（不计排序与结果数组）。

核心知识点

- 降维：把「三数之和」通过「固定一个数」转化为「两数之和」，这是处理 K 数之和的通用思路。
- 排序 + 双指针：排序带来单调性，单调性让双指针可以「根据比较结果决定移动哪一侧」。
- 去重三连：外层跳过重复首元素，内层跳过重复的 left/right，保证结果的唯一性。
- 剪枝：nums[i] > 0 时直接 break，因为最小的数都为正，三数之和不可能为 0。
- 答案是「数值三元组」而不是下标，所以可以先排序打乱原顺序。

42. 接雨水 【困难】

题目描述

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

> **示例：**
>
> 输入：height = [0,1,0,2,1,0,1,3,2,1,2,1]
> 输出：6
> 解释：中间凹下去的位置可以存水，总共能接 6 个单位的雨水。

解题思路

核心公式：对每个位置 i，它能接的雨水量 = min(左边最高的柱子, 右边最高的柱子) - 自己的高度。

直观理解：一个位置能装多少水，取决于它左右两侧「围栏」里较矮的那一边，水最多只能高到那儿，再减去自己占的高度。

方法一（预处理数组，最易懂）：

1. 先从前往后扫一遍，得到 leftMax[i] = 位置 i 左边（含自己）的最高柱子。

2. 再从后往前扫一遍，得到 rightMax[i] = 位置 i 右边（含自己）的最高柱子。

3. 对每个 i，累加 min(leftMax[i], rightMax[i]) - height[i]（负数则当作 0）。

方法二（双指针，空间 O(1)）：

其实我们并不需要同时知道左右两侧的最高值，只需要知道「较矮的那一侧」。用 left、right 两个指针从两端靠近，同时维护 leftMax 和 rightMax：

- 若 leftMax < rightMax：左侧是短板，位置 left 的水量可以被 leftMax 确定，处理 left 后 left++。

- 否则：右侧是短板，处理 right 后 right--。

这样只用两个变量就完成了计算。

参考代码

```
# 方法一：预处理前后缀最大值，思路最清晰
def trap(height):
    n = len(height)
    if n == 0:
        return 0
    leftMax = [0] * n
    rightMax = [0] * n
    leftMax[0] = height[0]
    for i in range(1, n):
        leftMax[i] = max(leftMax[i - 1], height[i])
    rightMax[n - 1] = height[n - 1]
    for i in range(n - 2, -1, -1):
        rightMax[i] = max(rightMax[i + 1], height[i])
    ans = 0
    for i in range(n):
        ans += min(leftMax[i], rightMax[i]) - height[i]
    return ans

# 方法二：双指针，空间 O(1)
def trap2(height):
    left, right = 0, len(height) - 1
    leftMax = rightMax = 0
    ans = 0
    while left < right:
        leftMax = max(leftMax, height[left])
        rightMax = max(rightMax, height[right])
        if leftMax < rightMax:      # 左侧是短板，left 位置的水位已确定
            ans += leftMax - height[left]
            left += 1
        else:
            ans += rightMax - height[right]
            right -= 1
    return ans
```

复杂度分析

方法一：时间 O(n)，空间 O(n)（两个辅助数组）。方法二：时间 O(n)，空间 O(1)。

核心知识点

- 逐位置贡献法：把「整体存水量」拆解成「每个位置存水量之和」，这是很多数组难题的破题方式。
- 前后缀最值预处理：先正扫一遍、再倒扫一遍，得到每个位置左右两侧的极值，是极其常见的手段。
- 短板效应：水位由左右两侧较矮的围栏决定。
- 双指针优化空间：当一侧的短板已经确定时，另一个方向的信息就不再需要，于是可以用双指针边算边丢。
- 单调栈也能解此题（按行计算横向水槽），可作为进阶拓展。

第 3 章 滑动窗口

本章核心思想

滑动窗口本质上是「双指针」的一种特殊形态：用一个左边界和一个右边界框住一段连续区间，右指针负责「扩张」窗口，左指针负责「收缩」窗口，整个过程窗口只前进不后退，因此能把 $O(n^2)$ 的区间枚举降到 $O(n)$。

识别信号：题目要求「连续的子数组 / 子串」，并且带有「最长、最短、满足某条件」这类字眼。

3. 无重复字符的最长子串 【中等】

题目描述

给定一个字符串 s，请你找出其中不含有重复字符的最长子串的长度。注意是「子串」，必须连续。

> **示例：**
>
> 输入：s = "abcabcbb"
> 输出：3
> 解释：最长无重复子串是 "abc"，长度为 3。

解题思路

暴力做法是枚举所有子串并检查是否重复，$O(n^3)$ 太慢。用滑动窗口可以做到 $O(n)$。

窗口的定义：始终保持窗口 [left, right] 内的字符「互不相同」。

右指针不断向右扩张，把新字符纳入窗口；如果这个字符和窗口内某个字符重复了，就把左指针移动到「重复字符的下一格」，让窗口重新变得无重复。

具体步骤：

1. 用一个字典 last 记录「每个字符最后一次出现的下标」。

2. right 从 0 遍历到末尾。若 s[right] 之前出现过，且它的下标 >= left，说明它在当前窗口内，把 left 更新为「该下标 + 1」。

3. 更新 last[s[right]] = right。

4. 此时窗口 [left, right] 一定无重复，用 right - left + 1 更新答案。

易错点：判断条件一定要带「>= left」，否则会用到已经滑出窗口的历史字符，导致 left 错误地回退。

参考代码

```
def lengthOfLongestSubstring(s):
    last = {}                # 字符 -> 最后一次出现的下标
    left = 0                 # 窗口左边界
    ans = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:   # 该字符在当前窗口内出现过
            left = last[ch] + 1               # 左边界跳到重复字符的下一位
        last[ch] = right
        ans = max(ans, right - left + 1)      # 更新最长长度
    return ans
```

复杂度分析

时间复杂度 O(n)：左右指针各自最多走 n 步。空间复杂度 O(min(n, 字符集大小))，字典最多存所有不同字符。

核心知识点

- 滑动窗口：用两个边界框住连续区间，右扩左缩，一次遍历解决区间问题。
- 哈希记录位置：字典存「字符→最后下标」，让判断重复和定位都能 O(1) 完成。
- 左指针不回退：left 永远只增不减，这是整体复杂度为 O(n) 的根本原因。
- 子串 vs 子序列：子串必须连续，子序列可以跳过元素，两者解法完全不同。

438. 找到字符串中所有字母异位词 【中等】

题目描述

给定两个字符串 s 和 p，找到 s 中所有 p 的字母异位词的子串，返回这些子串的起始索引。字母异位词指由相同字母重排列形成的字符串，且不考虑字符串中字母的顺序。答案按任意顺序返回。

示例：

输入：s = "cbaebabacd", p = "abc"

输出：[0,6]
解释：从 0 开始的 "cba" 和从 6 开始的 "bac" 都是 "abc" 的异位词。

解题思路

因为异位词长度一定等于 p 的长度，所以我们要找的是「长度为 len(p) 的窗口，其字符计数和 p 完全一致」。

这是长度固定的滑动窗口。

具体步骤：

1. 用两个长度为 26 的数组 need 和 win，分别记录 p 的字符计数、当前窗口的字符计数。

2. 先把 s 的前 len(p) 个字符放进窗口。

3. 检查 need 与 win 是否相等，相等就记录起始下标 0。

4. 之后窗口每次整体右移一格：加入新进来的字符 s[i]，同时移除滑出去的字符 s[i - len(p)]。

5. 每移动一次就比对一次两个计数数组，相等就记录答案。

关键点：窗口大小固定，所以不需要 while 收缩，每次「进一个、出一个」即可。

参考代码

```
def findAnagrams(s, p):
    n, m = len(s), len(p)
    if n < m:
        return []
    need = [0] * 26
    win = [0] * 26
    for ch in p:
        need[ord(ch) - ord('a')] += 1
    res = []
    for i in range(n):
        win[ord(s[i]) - ord('a')] += 1        # 新字符进窗口
        if i >= m:                            # 窗口长度超过 m，移除最左边
            win[ord(s[i - m]) - ord('a')] -= 1
        if win == need:                       # 计数完全一致 -> 是异位词
            res.append(i - m + 1)
    return res
```

复杂度分析

时间复杂度 O(n)：遍历一次 s，每次比对两个长度 26 的数组（常数）。空间复杂度 O(1)：两个固定大小的计数数组。

核心知识点

- 定长滑动窗口：窗口大小固定时，只需「进一出一」，不必用 while 收缩，代码更简洁。
- 字符计数数组：用长度 26 的数组代替字典统计字母，比较时可以直接用 == 判断，效率高。
- 把「是否异位词」转化为「计数数组是否相等」，是字符串比较类问题的常用转化。
- 下标换算：窗口起点 = 右边界 - 长度 + 1。

第 4 章 子串

本章核心思想

这一组题目仍然围绕「连续区间」展开，但技巧更进阶：前缀和 + 哈希、单调队列、以及难度最高的最小覆盖子串。它们共同考察「如何用合适的数据结构维护区间信息」。

560. 和为 K 的子数组 【中等】

题目描述

给你一个整数数组 nums 和一个整数 k，请你统计并返回该数组中和为 k 的子数组的个数。子数组是数组中元素的连续非空序列。

示例：

```
输入：nums = [1,1,1], k = 2
输出：2
解释：有两个子数组 [1,1] 的和为 2。
```

解题思路

子数组求和自然想到「前缀和」：pre[i] 表示前 i 个元素的和。那么子数组 nums[j..i] 的和 = pre[i+1] - pre[j]。

题目要求这个差等于 k，也就是 pre[i+1] - k = pre[j]，即「前面出现过多少个前缀和等于 当前前缀和 - k」。

于是可以用哈希表统计每个前缀和出现的次数。

具体步骤：

1. 建字典 cnt，初始 cnt[0] = 1（表示「空前缀」出现了一次，用于处理从下标 0 开始的子数组）。

2. 设 cur = 0，ans = 0。

3. 遍历每个数字 x：cur += x，然后 ans += cnt.get(cur - k, 0)（以当前位置结尾的合法子数组数）。

4. 再令 cnt[cur] += 1（把自己这个前缀和记进去，供后面使用）。

注意「先查后存」的顺序——如果先存后查，当 k == 0 时会把当前前缀和自己算进去，导致多计数。

同时也要注意字典里要统计「次数」而不是「是否存在」，因为同一个前缀和可能出现多次。

参考代码

```
def subarraySum(nums, k):
    cnt = {0: 1}          # 前缀和 -> 出现次数; 0 表示空前缀
    cur = 0
    ans = 0
    for x in nums:
        cur += x
        ans += cnt.get(cur - k, 0)   # 前面有多少个前缀和等于 cur-k
        cnt[cur] = cnt.get(cur, 0) + 1
    return ans
```

复杂度分析

时间复杂度 O(n)：一次遍历，哈希操作均为 O(1)。空间复杂度 O(n)：最坏情况下前缀和互不相同。

核心知识点

- 前缀和（Prefix Sum）：把「区间和」转化为「两个前缀和的差」，是处理子数组求和问题的第一武器。
- 哈希表统计前缀和出现次数：把「找多少个 j 使 pre[j] = cur - k」变成 O(1) 的字典查询。
- cnt[0] = 1 的哨兵技巧：用来处理「子数组从下标 0 开始」的情况，非常容易漏。
- 先查后存：保证不会把当前元素自己当成配对对象。

239. 滑动窗口最大值 【困难】

题目描述

给你一个整数数组 nums，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 k 个数字。滑动窗口每次只向右移动一位。返回滑动窗口中的最大值。

示例：

输入：nums = [1,3,-1,-3,5,3,6,7], k = 3

输出：[3,3,5,5,6,7]

解题思路

暴力做法每次窗口滑动都求一次 max，复杂度 O(nk)。用「单调队列」可以做到 O(n)。

核心思想：维护一个「下标队列」，队列中对应的数值是单调递减的。为什么可以这样？如果新来的数比队尾的数大，那队尾那个数在未来的任何窗口里都不可能成为最大值了（因为新数更大且更新），直接把它丢掉。

具体步骤：

1. 建一个双端队列 dq，存的是下标；队列内的数值保持从大到小。

2. 遍历每个下标 i：

- 先检查队首是否已经滑出窗口（下标 <= i - k），是就弹出。

- 然后从队尾开始，把所有「数值小于等于 nums[i]」的下标弹出。

- 把 i 入队。

- 当 i >= k - 1 时，队首下标对应的值就是当前窗口最大值，记录答案。

Python 里用 collections.deque 实现双端队列，两端插入删除都是 O(1)。

参考代码

```
from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()      # 存下标，对应数值单调递减
    res = []
    for i, x in enumerate(nums):
        # 1. 队首过期(已滑出窗口)则移除
        if dq and dq[0] <= i - k:
            dq.popleft()
        # 2. 队尾比当前值小的全部移除，保证单调性
        while dq and nums[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        # 3. 窗口成形后，队首就是最大值
        if i >= k - 1:
            res.append(nums[dq[0]])
    return res
```

复杂度分析

时间复杂度 O(n)：每个下标最多入队一次、出队一次。空间复杂度 O(k)：队列最多存 k 个下标。

核心知识点

- 单调队列：队列中元素保持单调（本题递减），队首即当前区间的极值，是解决「滑动窗口极值」的标准工具。
- 及时淘汰无用元素：被更大且更新的元素「压住」的旧元素永远不可能成为最大值，可以放心丢弃。
- 双端队列 deque：两端都能 O(1) 增删，是单调队列的载体。
- 存下标而非存值：这样既能比较数值，又能判断是否过期（下标是否出窗口）。
- 与「单调栈」区分：单调栈解决「下一个更大元素」，单调队列解决「窗口内极值」。

76. 最小覆盖子串 【困难】

题目描述

给你一个字符串 s、一个字符串 t。返回 s 中涵盖 t 所有字符的最小子串。如果 s 中不存在涵盖 t 所有字符的子串，则返回空字符串 ""。注意：如果 s 中存在这样的子串，我们保证它是唯一的答案。

> **示例：**
>
> 输入：s = "ADOBECODEBANC", t = "ABC"
> 输出："BANC"

解题思路

这是滑动窗口里最经典、也最难的模板题：求「满足条件的最短窗口」。

基本框架是「右指针扩张找可行解，左指针收缩找最优解」：

1. 用字典 need 记录 t 中各字符的需求数量，用字典 win 记录当前窗口中各字符的数量。

2. 用变量 valid 记录「已经满足需求（窗口内该字符数量 >= 需求数量）的字符种类数」，当 valid == len(need) 时说明窗口已经覆盖了 t 的所有字符。

3. right 向右扩张：把 s[right] 加入 win，若某字符数量达到需求，valid++。

4. 当窗口已覆盖（valid == len(need)）时，进入内层 while，尝试收缩左边界：

- 先用当前窗口长度更新最优答案。

- 把 s[left] 移出窗口，若因此导致某字符数量不足，valid--。

- left++。

5. 直到窗口不再覆盖，回到外层继续扩张。

有效性的判断是用「种类数 valid」而不是「逐字符比较」，这是让复杂度降到 O(n) 的关键。

参考代码

```
from collections import Counter

def minWindow(s, t):
    need = Counter(t)          # 目标字符需求
    win = {}                   # 当前窗口字符计数
    valid = 0                  # 已满足需求的字符种类数
    left = 0
    start, min_len = 0, float('inf')

    for right, ch in enumerate(s):
        # 右指针扩张
        if ch in need:
            win[ch] = win.get(ch, 0) + 1
            if win[ch] == need[ch]:
                valid += 1
        # 窗口已覆盖 t，尝试收缩左边界
        while valid == len(need):
            if right - left + 1 < min_len:      # 更新最优解
                start, min_len = left, right - left + 1
            lc = s[left]
            if lc in need:
                if win[lc] == need[lc]:
                    valid -= 1                  # 收缩后不再满足
                win[lc] -= 1
            left += 1
    return s[start:start + min_len] if min_len != float('inf') else ""
```

复杂度分析

时间复杂度 O(n + m)：n 为 s 长度，m 为 t 长度，每个字符最多进窗口一次、出窗口一次。空间复杂度 O(m)：字典最多存 t 中不同字符。

核心知识点

- 滑动窗口通用模板：外层扩张、内层收缩，可同时求「最长子串」（内层不收缩时）和「最短子串」（内层尽量收缩）。
- valid 计数法：把「窗口是否满足要求」压缩为一个整数比较，避免每轮都对整个字典做检查。
- Counter 计数器：Python 标准库自带的字符统计工具，比手写循环更简洁。
- float('inf') 作为「无解哨兵」，最后统一判断是否找到过合法窗口。
- 覆盖类问题的判定条件：需求的每个字符在窗口内的数量都 >= 需求量。

第 5 章 普通数组

本章核心思想

这一组题目考察数组上的经典算法思想：动态规划（最大子数组和）、区间贪心（合并区间）、原地轮转、前缀后缀积、以及利用「下标本身作为哈希」的原地算法。它们都很能体现「观察题目结构、设计巧妙算法」的能力。

53. 最大子数组和 【中等】

题目描述

给你一个整数数组 nums，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。子数组是数组中的一个连续部分。

示例：

```
输入：nums = [-2,1,-3,4,-1,2,1,-5,4]
输出：6
解释：连续子数组 [4,-1,2,1] 的和最大，为 6。
```

解题思路

这是动态规划的入门经典题（Kadane 算法）。

定义状态：f[i] 表示「以 nums[i] 结尾」的最大子数组和。注意是「必须以 i 结尾」，这个限定让状态转移变得简单。

转移方程：f[i] = max(nums[i], f[i-1] + nums[i])。

含义：要么自己单独成一个子数组，要么接在前面结尾的子数组后面。哪个和更大就选哪个。

直观理解：如果前面那段的和是负数，那它只会拖后腿，不如从自己重新开始。

具体步骤：

1. cur = nums[0]，ans = nums[0]。cur 表示「以当前位置结尾的最大和」。

2. 从下标 1 遍历：cur = max(x, cur + x)。

3. ans = max(ans, cur)。

4. 返回 ans。

由于 f[i] 只依赖 f[i-1]，可以用一个变量滚动更新，空间降到 O(1)。

参考代码

```
def maxSubArray(nums):
    cur = ans = nums[0]        # cur: 以当前位置结尾的最大和
    for x in nums[1:]:
        cur = max(x, cur + x)  # 前面是负数就重新开始
        ans = max(ans, cur)
    return ans

# 附：分治法(O(n log n))版本，作为进阶拓展
def maxSubArray2(nums):
    def solve(l, r):
        if l == r:
            return nums[l]
        mid = (l + r) // 2
        left_best = solve(l, mid)
        right_best = solve(mid + 1, r)
        # 跨越中点的最大和
        s = 0; lt = float('-inf')
        for i in range(mid, l - 1, -1):
            s += nums[i]; lt = max(lt, s)
        s = 0; rt = float('-inf')
        for i in range(mid + 1, r + 1):
            s += nums[i]; rt = max(rt, s)
        return max(left_best, right_best, lt + rt)
    return solve(0, len(nums) - 1)
```

复杂度分析

时间复杂度 O(n)（DP 版本）：一次遍历。空间复杂度 O(1)：只用两个变量滚动。分治版本时间 O(n log n)、空间 O(log n)。

核心知识点

- 动态规划的状态定义要「带约束」：f[i] 定义为「以 i 结尾」的最大和，才让转移方程变得可写。
- Kadane 算法：负数前缀果断丢弃，是贪心与 DP 结合的精妙之处。
- 滚动数组 / 状态压缩：只依赖前一个状态时，用变量代替整个 dp 数组。
- 分治法（拓展）：最大子数组和 = max(左半最大值, 右半最大值, 跨越中点的最大值)。

56. 合并区间 【中等】

题目描述

以数组 intervals 表示若干个区间的集合，其中单个区间为 intervals[i] = [starti, endi]。请你合并所有重叠的区间，并返回一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例：

```
输入：intervals = [[1,3],[2,6],[8,10],[15,18]]
输出：[[1,6],[8,10],[15,18]]
解释：[1,3] 和 [2,6] 重叠，合并为 [1,6]。
```

解题思路

区间类问题的第一反应是「先按左端点排序」。排序之后，能合并的区间必然相邻。

具体步骤：

1. 把所有区间按左端点从小到大排序。

2. 结果列表 res 先放入第一个区间。

3. 依次考察后面的区间 [l, r]：

- 若 l <= res 中最后一个区间的右端点，说明有重叠或相接，合并：把最后一个区间的右端点更新为 max(原右端点, r)。

- 否则说明不重叠，直接把 [l, r] 追加到 res。

4. 返回 res。

注意合并时右端点要取 max，因为有可能新区间完全被包含在旧区间里（例如 [1,10] 与 [2,3]）。

Python 的 list 可以直接比较，所以对 intervals 排序会默认按第一个元素、再按第二个元素排序。

参考代码

```
def merge(intervals):
    intervals.sort()                    # 按左端点排序
    res = [intervals[0]]
    for l, r in intervals[1:]:
        if l <= res[-1][1]:             # 与上一个区间重叠
```

```
            res[-1][1] = max(res[-1][1], r)
        else:                           # 不重叠，直接追加
            res.append([l, r])
    return res
```

复杂度分析

时间复杂度 O(n log n)：排序是瓶颈，合并只需一次线性扫描。空间复杂度 O(log n) 或 O(n)：取决于排序实现和结果存储。

核心知识点

- 区间问题先排序：按左端点排序后，重叠关系就只存在于相邻区间之间，问题大幅简化。
- 排序的「局部有序性」：利用排序带来的单调性，把全局比较变成相邻比较。
- 合并时取 max：处理「区间被完全包含」的情况，是常见的易错点。
- 类比题：插入区间（57）、用最少数量的箭引爆气球（452）、无重叠区间（435）都是同一套思路。

189. 轮转数组 【中等】

题目描述

给定一个整数数组 nums，将数组中的元素向右轮转 k 个位置，其中 k 是非负数。要求尽量想出更多的解决方案，至少有三种不同的方法可以解决这个问题，并使用空间复杂度为 O(1) 的原地算法。

示例：

```
输入：nums = [1,2,3,4,5,6,7], k = 3
输出：[5,6,7,1,2,3,4]
解释：向右轮转 1 步得 [7,1,2,3,4,5,6]，轮转 3 步得 [5,6,7,1,2,3,4]。
```

解题思路

先注意一个细节：如果 k >= n，轮转效果等同于轮转 k % n 步，所以先做 k %= n。

方法一（切片拼接，最易读，但需要 O(n) 空间）：

把数组分成后 k 个元素和前 n-k 个元素，重新拼接。

方法二（三次翻转，空间 O(1)，是本题的标准答案）：

这是一个很妙的技巧：先把整个数组翻转，再翻转前 k 个元素，最后翻转后 n-k 个元素。

验证：以 [1,2,3,4,5,6,7]、k=3 为例：

整体翻转 -> [7,6,5,4,3,2,1]

翻转前 3 个 -> [5,6,7,4,3,2,1]

翻转后 4 个 -> [5,6,7,1,2,3,4] ✓

直觉理解：整体翻转把「后面的元素」搬到了前面，但也把它们的顺序倒过来了；再分别翻转两段，就把两段内部的顺序各自还原。

具体步骤（三次翻转）：

1. k %= n。

2. 翻转区间 [0, n-1]。

3. 翻转区间 [0, k-1]。

4. 翻转区间 [k, n-1]。

参考代码

```
def rotate(nums, k):
    n = len(nums)
    k %= n                          # 处理 k > n 的情况

    def reverse(l, r):              # 原地翻转闭区间 [l, r]
        while l < r:
            nums[l], nums[r] = nums[r], nums[l]
            l += 1
            r -= 1

    reverse(0, n - 1)               # 1. 整体翻转
    reverse(0, k - 1)               # 2. 翻转前 k 个
    reverse(k, n - 1)               # 3. 翻转后 n-k 个

# 方法二(简单但费空间):
def rotate2(nums, k):
    n = len(nums)
    k %= n
    nums[:] = nums[n - k:] + nums[:n - k]
```

复杂度分析

三次翻转法：时间复杂度 O(n)，每个元素被翻转两次；空间复杂度 O(1)。切片法：时间 O(n)，空间 O(n)。

核心知识点

- 三次翻转技巧：用「反转 + 反转 + 反转」实现循环移位，是原地算法的经典构造。
- k %= n 取模处理：把「多轮无效轮转」归一化到有效范围内。
- nums[:] = ... 与 nums = ... 的区别：前者是原地修改（外部引用可见），后者只是重新绑定局部变量名。
- 环状替换法（拓展）：每个元素直接跳到目标位置，用计数控制避免死循环，也能达到 O(1) 空间。

238. 除自身以外数组的乘积 【中等】

题目描述

给你一个整数数组 nums，返回数组 answer，其中 answer[i] 等于 nums 中除 nums[i] 之外其余各元素的乘积。题目数据保证数组 nums 之中任意元素的全部前缀元素和后缀的乘积都在 32 位整数范围内。请不要使用除法，且在 O(n) 时间复杂度内完成此题。

示例：

```
输入：nums = [1,2,3,4]
输出：[24,12,8,6]
解释：answer[0] = 2×3×4 = 24，answer[1] = 1×3×4 = 12，依此类推。
```

解题思路

因为不能用除法，所以不能「先算总乘积再除以自己」（而且元素可能为 0，除法也不安全）。

关键观察：answer[i] = （i 左侧所有数的乘积）×（i 右侧所有数的乘积）。

于是我们可以分别计算「左侧前缀积」和「右侧后缀积」，再相乘。

具体步骤：

1. 建结果数组 res，长度与 nums 相同。

2. 第一趟从左往右：用变量 pre 记录「到 i 为止左边所有元素的乘积」，先令 res[i] = pre，再让 pre *= nums[i]。这一步做完，res[i] 存的就是左侧乘积。

3. 第二趟从右往左：用变量 suf 记录「到 i 为止右边所有元素的乘积」，令 res[i] *= suf，再让 suf *= nums[i]。

4. 返回 res。

这样只用结果数组本身作为存储，不需要额外的两个数组，空间复杂度降到 O(1)（不计输出数组）。

参考代码

```
def productExceptSelf(nums):
    n = len(nums)
    res = [1] * n
    pre = 1
    for i in range(n):            # 第一趟：res[i] = 左侧所有元素的乘积
        res[i] = pre
        pre *= nums[i]
    suf = 1
    for i in range(n - 1, -1, -1):  # 第二趟：乘上右侧所有元素的乘积
        res[i] *= suf
        suf *= nums[i]
    return res
```

复杂度分析

时间复杂度 O(n)：两趟线性扫描。空间复杂度 O(1)：除了输出数组外只用两个变量（输出数组通常不计入空间复杂度）。

核心知识点

- 前缀 / 后缀分解：把「除自己以外的聚合」拆成「左边 × 右边」，是避免除法、避免特殊值判断的通用技巧。
- 两趟扫描：一趟从左往右算前缀，一趟从右往左算后缀，中间用结果数组当缓冲。
- 空间优化：把中间结果直接写在输出数组上，省掉辅助数组。
- 不用除法：因为除法的前提是元素非零且无精度损失，而乘积法中 0 会破坏一切。

41. 缺失的第一个正数 【困难】

题目描述

给你一个未排序的整数数组 nums，请你找出其中没有出现的最小的正整数。请你实现时间复杂度为 O(n) 并且只使用常数级别额外空间的解决方案。

> **示例：**
>
> 输入：nums = [3,4,-1,1]
> 输出：2
> 解释：数组中包含 1、3、4，缺少 2，所以答案是 2。

解题思路

关键观察：长度为 n 的数组中，缺失的最小正整数一定落在 [1, n+1] 范围内。

因为如果 1 到 n 全都出现了，那答案就是 n+1；否则答案就是 1..n 中第一个没出现的数。

既然答案的范围是 1..n，我们就可以把「数字 x」放到「下标 x-1」的位置上，让数组自己变成一个「哈希表」——下标 i 上有值 i+1，说明 i+1 出现过。

具体步骤：

1. 遍历数组，对于每个位置 i，只要满足「nums[i] 在 [1, n] 范围内」且「它还没被换到正确位置（即 nums[i] != nums[nums[i]-1]）」，就把它交换到下标 nums[i]-1 处。用 while 反复交换，直到当前位置满足条件。

2. 交换完成后，再从头遍历：第一个满足 nums[i] != i+1 的位置，答案就是 i+1。

3. 如果全部都满足，说明 1..n 都在，答案是 n+1。

注意第 1 步的循环条件里用 nums[i] != nums[nums[i]-1] 而不是 nums[i] != i+1，因为要防止两个相同的数（如两个 1）之间无限互相交换。

参考代码

```
def firstMissingPositive(nums):
    n = len(nums)
    for i in range(n):
```

```
        # 把值为 x 的数放到下标 x-1 处
        while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
            j = nums[i] - 1
            nums[i], nums[j] = nums[j], nums[i]
    for i in range(n):
        if nums[i] != i + 1:
            return i + 1
    return n + 1
```

复杂度分析

时间复杂度 O(n)：每个数字最多被交换一次就归位，总交换次数不超过 n。空间复杂度 O(1)：原地交换。

核心知识点

- 「下标即哈希」的原地哈希思想：当数据范围是 [1, n] 时，用数组下标本身做映射，无需额外空间。
- 范围推断：先确定答案的取值范围，才能设计出对应的下标映射方案。
- 交换条件的边界处理：用 nums[nums[i]-1] != nums[i] 避免重复元素造成的死循环。
- 原地重排 + 二次扫描：第一趟归位，第二趟找出第一个「不在自己位置上」的下标。
- 同类思想：LeetCode 442、448 等题都用到了原地哈希。

第 6 章 矩阵

本章核心思想

矩阵题的核心是「二维坐标的遍历规律」。常见套路有三类：一是模拟（螺旋、旋转），关键是找到下标变换公式；二是利用矩阵自身的「行列有序」性质做二分或线性查找；三是原地标记（用矩阵自己的某一行/列当标记位），节省空间。

73. 矩阵置零 【中等】

题目描述

给定一个 m x n 的矩阵，如果一个元素为 0，则将其所在行和列的所有元素都设为 0。请使用原地算法。

示例：

```
输入：matrix = [[1,1,1],[1,0,1],[1,1,1]]
输出：[[1,0,1],[0,0,0],[1,0,1]]
```

解题思路

陷阱在于：如果一边扫描一边置零，新产生的 0 会被误认为是原本就有的 0，导致整片区域被错误清零。所以必须「先记录所有该置零的行和列，再统一处理」。

方法一（最简单，O(m+n) 空间）：用两个集合/数组记录哪些行、哪些列需要置零。

方法二（O(1) 空间，本题要求的「原地」最优解）：借用矩阵的「第一行」和「第一列」当作标记数组。

具体步骤（方法二）：

1. 先记录第一行、第一列本身是否需要置零（用两个布尔变量 flagRow0、flagCol0）。

2. 从 (1,1) 开始扫描整个矩阵：若 matrix[i][j] == 0，就把 matrix[i][0] 和 matrix[0][j] 标记为 0（用第一行第一列保存「第 i 行要清零」「第 j 列要清零」的信息）。

3. 再根据第一行、第一列的标记，把对应行和列置零（注意从 1 开始，不影响标记本身）。

4. 最后根据 flagRow0 / flagCol0 决定是否清空第一行、第一列。

参考代码

```
def setZeroes(matrix):
    m, n = len(matrix), len(matrix[0])
    flag_row0 = any(matrix[0][j] == 0 for j in range(n))
    flag_col0 = any(matrix[i][0] == 0 for i in range(m))

    # 用第一行/第一列保存标记
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][j] == 0:
                matrix[i][0] = 0
                matrix[0][j] = 0

    # 依据标记置零 (从 1 开始, 避免破坏标记)
    for i in range(1, m):
        for j in range(1, n):
            if matrix[i][0] == 0 or matrix[0][j] == 0:
                matrix[i][j] = 0

    # 处理第一行与第一列
    if flag_row0:
        for j in range(n):
            matrix[0][j] = 0
    if flag_col0:
        for i in range(m):
            matrix[i][0] = 0
    return matrix
```

复杂度分析

时间复杂度 O(m×n)：需要遍历矩阵常数次。空间复杂度 O(1)：只用两个布尔变量（方法二）。

核心知识点

- 不能边扫描边修改：因为新产生的 0 会污染后续判断，必须「先标记、后执行」。
- 原地标记技巧：把第一行/第一列「征用」为标记数组，是矩阵题常用的省空间手段。
- 边界单独处理：被征用的第一行和第一列需要额外变量记录原始状态。
- any() 生成器：Python 里判断「是否存在满足条件的元素」的简洁写法。

54. 螺旋矩阵 【中等】

题目描述

给你一个 m 行 n 列的矩阵 matrix，请按照顺时针螺旋顺序，返回矩阵中的所有元素。

> **示例：**
>
> 输入：matrix = [[1,2,3],[4,5,6],[7,8,9]]
>
> 输出：[1,2,3,6,9,8,7,4,5]

解题思路

这是一道纯模拟题，关键在于「用四个边界把已经走过的外圈剥掉」。

具体步骤：

1. 定义四个边界：top = 0、bottom = m-1、left = 0、right = n-1。

2. 循环执行四步，直到边界交叉：

   - 从左到右遍历 top 行，然后 top += 1。

   - 从上到下遍历 right 列，然后 right -= 1。

   - 判断 top <= bottom 后再从右到左遍历 bottom 行，然后 bottom -= 1。

   - 判断 left <= right 后再从下到上遍历 left 列，然后 left += 1。

3. 每走完一条边就收缩对应边界。

特别注意第 3、4 步的「边界判断」：当剩余只有一行或一列时，如果不加判断就会重复输出。

参考代码

```
def spiralOrder(matrix):
    if not matrix:
        return []
    top, bottom = 0, len(matrix) - 1
    left, right = 0, len(matrix[0]) - 1
    res = []
    while top <= bottom and left <= right:
        for j in range(left, right + 1):        # 上边：左->右
            res.append(matrix[top][j])
        top += 1
        for i in range(top, bottom + 1):        # 右边：上->下
```

```
            res.append(matrix[i][right])
        right -= 1
        if top <= bottom:                       # 下边：右->左
            for j in range(right, left - 1, -1):
                res.append(matrix[bottom][j])
            bottom -= 1
        if left <= right:                       # 左边：下->上
            for i in range(bottom, top - 1, -1):
                res.append(matrix[i][left])
            left += 1
    return res
```

复杂度分析

时间复杂度 O(m×n)：每个元素访问一次。空间复杂度 O(1)（不计输出数组）：只用四个边界变量。

核心知识点

- 边界模拟法：用 top/bottom/left/right 四个变量描述「待遍历区域」，是最不易出错的螺旋遍历写法。
- 收缩顺序：每走完一边就收一格，下一圈自然从内层开始。
- 特殊情况处理：单行 / 单列时，第 3、4 步必须加边界判断，否则会重复输出。
- 同类题：螺旋矩阵 II（59）是反过来的构造题；两者可以对照理解。

48. 旋转图像 【中等】

题目描述

给定一个 n × n 的二维矩阵 matrix 表示一个图像。请你将图像顺时针旋转 90 度。你必须在原地旋转图像，这意味着你需要直接修改输入的二维矩阵。请不要使用另一个矩阵来旋转图像。

> **示例：**
>
> 输入：matrix = [[1,2,3],[4,5,6],[7,8,9]]
>
> 输出：[[7,4,1],[8,5,2],[9,6,3]]

解题思路

直接计算旋转后的位置容易绕晕，常用技巧是把它拆成两步基本操作：

第一步：沿主对角线（左上—右下）翻转（转置）。

第二步：左右翻转（每一行做逆序）。

验证：

原矩阵 [[1,2,3],[4,5,6],[7,8,9]]

转置后 [[1,4,7],[2,5,8],[3,6,9]]

每行逆序后 [[7,4,1],[8,5,2],[9,6,3]] ✓ 正是顺时针旋转 90 度的结果。

具体步骤：

1. 双重循环，只处理 j > i 的元素，交换 matrix[i][j] 与 matrix[j][i]（转置）。

2. 对每一行调用 reverse，或直接 matrix[i].reverse()。

如果题目要求逆时针旋转 90 度，则改为「沿副对角线翻转 + 左右翻转」。

参考代码

```
def rotate(matrix):
    n = len(matrix)
    # 第一步：沿主对角线转置（只处理上三角，避免重复交换）
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
    # 第二步：每一行左右翻转
    for i in range(n):
        matrix[i].reverse()
    return matrix
```

复杂度分析

时间复杂度 $O(n^2)$：每个元素被处理常数次。空间复杂度 O(1)：原地交换，不需要额外矩阵。

核心知识点

- 分解复杂操作为基本操作：把「旋转 90 度」拆成「转置 + 行反转」，是化繁为简的典范。
- 坐标变换公式（拓展）：顺时针旋转时，(i, j) 的新位置是 (j, n-1-i)，也可以直接按四元环交换。
- 只处理上三角：转置时 j 从 i+1 开始，避免同一对元素被交换两次又还原。
- 举一反三：逆时针 = 副对角线翻转 + 行反转；180 度 = 上下翻转 + 左右翻转。

240. 搜索二维矩阵 II 【中等】

题目描述

编写一个高效的算法来搜索 m x n 矩阵 matrix 中的一个目标值 target。该矩阵具有以下特性：

每行的元素从左到右升序排列；每列的元素从上到下升序排列。

> **示例：**
>
> 输入：matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5
>
> 输出：true

解题思路

因为行列都有序，可以选取一个「特殊的起点」来逐步缩小范围。

关键观察：取「右上角」元素作为起点（也可以取左下角）。它有两个很好的性质：

- 它是所在行的最大值，是所在列的最小值。

- 于是：若 target < 当前值，说明这一整列都比 target 大，列可以排除（向左移）；

  若 target > 当前值，说明这一整行都比 target 小，行可以排除（向下移）。

具体步骤：

1. i = 0（行），j = n-1（列），从右上角开始。

2. 当 i < m 且 j >= 0 时：

   - 若 matrix[i][j] == target，返回 True。

   - 若 matrix[i][j] > target，j -= 1（排除当前列）。

   - 若 matrix[i][j] < target，i += 1（排除当前行）。

3. 走出边界说明不存在，返回 False。

每走一步就排除一整行或一整列，所以最多走 m+n 步。

参考代码

```
def searchMatrix(matrix, target):
    if not matrix or not matrix[0]:
        return False
    m, n = len(matrix), len(matrix[0])
    i, j = 0, n - 1            # 从右上角出发
    while i < m and j >= 0:
        cur = matrix[i][j]
        if cur == target:
            return True
        elif cur > target:
            j -= 1             # 当前列整体偏大，排除该列
        else:
            i += 1             # 当前行整体偏小，排除该行
    return False
```

复杂度分析

时间复杂度 O(m + n)：每步排除一行或一列，最多 m+n 步。空间复杂度 O(1)。

核心知识点

- 利用有序性选择「有利的起点」：右上角（或左下角）是同时能对行和列做排除的唯一位置。
- 线性查找法：把二维搜索问题转化为一条单调路径，比逐行二分（O(m log n)）更快。
- 行列有序矩阵的三种解法对比：暴力 O(mn)、逐行二分 O(m log n)、右上角法 O(m+n)（最优）。
- 为什么它等价于二叉搜索树：以右上角为根的 BST，左子树是左边的元素，下侧是下面的元素。

第 7 章 链表

本章核心思想

链表题的通用技巧是「虚拟头结点（dummy node）」和「快慢指针」。dummy 结点可以统一处理「头结点被删除或修改」的边界情况，让代码少写很多 if；快慢指针则常用于找中点、判断环、找倒数第 k 个等场景。链表题一定要在纸上画图，把指针的指向变化一步步画出来再写代码，能避免 90% 的空指针错误。

这一组是链表题的进阶部分，综合运用了快慢指针、原地反转、哈希映射、分治归并、优先队列，以及「哈希表 + 双向链表」这种经典组合数据结构。做完这一组，链表就算打通了。

160. 相交链表 【简单】

题目描述

给你两个单链表的头节点 headA 和 headB，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 null。题目数据保证整个链式结构中不存在环。函数返回结果后，链表必须保持其原始结构。

示例：

```
输入：intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5]
输出：Intersected at '8'
```

解题思路

两条链表相交后是「共同的尾巴」，长度差会导致两个指针无法同时到达交点。

巧妙的解法（双指针「走对方的路」）：

1. 指针 p 从 headA 出发，指针 q 从 headB 出发，每次都走一步。

2. 当 p 走到 null 时，让它从 headB 重新开始；当 q 走到 null 时，让它从 headA 重新开始。

3. 这样两个指针一共走的路程都是「lenA + lenB」，必然会在同一时刻到达相交点；如果两链表不相交，它们会同时到达 null（此时返回 None）。

直观理解：p 先走完 A 再走 B，q 先走完 B 再走 A，两者步数相同，于是就「对齐」了长度差。

若存在交点 x，则两指针走的路程都是 (a + b + c) 中的同一段，自然同时到达 x。

参考代码

```
class ListNode:
    def __init__(self, x):
        self.val = x
        self.next = None

def getIntersectionNode(headA, headB):
    p, q = headA, headB
    while p != q:                       # 相遇时退出
        p = p.next if p else headB      # 走完 A 就换到 B 的头部
        q = q.next if q else headA      # 走完 B 就换到 A 的头部
    return p                            # 交点 或 None
```

复杂度分析

时间复杂度 O(lenA + lenB)：两指针合计最多走两遍长度。空间复杂度 O(1)：只用两个指针。

核心知识点

- 双指针「换头」技巧：通过让指针走完一条链后切换到另一条链，消除长度差。
- 哈希表法（对照）：先遍历 A 把所有结点放进 set，再遍历 B 找第一个在 set 中的结点，思路更直观但空间 O(n)。
- 链表结点比较用 is / == 判断「是否同一对象」，而不是比较 val，因为可能有重复值。
- 相交是指「同一个结点对象」，而不是值相等。

206. 反转链表 【简单】

题目描述

给你单链表的头节点 head，请你反转链表，并返回反转后的链表。

示例：

输入：head = [1,2,3,4,5]

输出：[5,4,3,2,1]

解题思路

这是链表最基础的题目，也是必须背下来的模板。核心是「三个指针」：

- prev：已经反转好的那部分链表的头。

- cur：当前正在处理的结点。

- nxt：暂存 cur 的下一个结点（因为改完 cur.next 后就找不到后面的结点了）。

具体步骤：

1. prev = None，cur = head。

2. 当 cur 不为空时循环：

   - 先存好 nxt = cur.next（保护现场）。

   - 把 cur.next 指向 prev（完成反转方向）。

   - prev = cur，cur = nxt（两个指针整体前移一格）。

3. 循环结束时 cur 为 None，prev 就是新的头结点，返回 prev。

务必按「先存、再改、后移」的顺序写，否则会断链。也可以用递归实现，但递归有栈深度风险。

参考代码

```
# 迭代法（推荐）
def reverseList(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next     # 1. 存好下一个结点
        cur.next = prev    # 2. 反转指向
        prev = cur         # 3. prev 前移
        cur = nxt          # 4. cur 前移
    return prev            # prev 成为新的头结点

# 递归法（理解即可）
def reverseList2(head):
    if not head or not head.next:
        return head
    new_head = reverseList2(head.next)  # 先反转后面的部分
    head.next.next = head               # 让下一个结点指回自己
    head.next = None                    # 自己成为新的尾巴
    return new_head
```

复杂度分析

迭代法：时间 O(n)，空间 O(1)。递归法：时间 O(n)，空间 O(n)（递归调用栈）。

核心知识点

- 三指针法：prev / cur / nxt，是链表操作中最基础也最重要的模板。
- 「先存后改」原则：修改指针指向之前一定要先保存后续结点，否则会丢失后面的链表。
- dummy 头结点的思路延伸：返回 prev 而不是 head，因为头结点已经变成了尾结点。
- 递归的链表视角：假设「后面的部分已经反转好了」，只需处理当前结点与它的下一个结点的关系。
- 衍生题：反转链表 II（92，区间反转）、K 个一组翻转链表（25）都建立在本题之上。

234. 回文链表 【简单】

题目描述

给你一个单链表的头节点 head，请你判断该链表是否为回文链表。如果是，返回 true；否则，返回 false。

> **示例：**
>
> ```
> 输入：head = [1,2,2,1]
> 输出：true
> 输入：head = [1,2]
> 输出：false
> ```

解题思路

方法一（简单，O(n) 空间）：把链表的值全部读进一个数组，再用双指针从两端向中间比较。

方法二（进阶，O(1) 空间）：把链表的「后半段原地反转」，然后与前半段逐个比较。

具体步骤：

1. 用快慢指针找到中点：快指针每次走两步，慢指针每次走一步；快指针到终点时，慢指针正好在中点。

2. 从慢指针处开始，把后半段链表反转（复用「反转链表」的模板）。

3. 用两个指针分别从前半段开头和后半段开头出发，逐个比较值；全部相同则返回 true。

4. （可选）把后半段再反转回来，恢复链表原状。

奇偶长度处理：奇数长度时，中间那个结点两边都不影响比较，所以循环条件用「后半段指针不为空」即可。

参考代码

```
# 方法一：转数组 + 双指针（最易理解）
def isPalindrome(head):
    vals = []
    while head:
        vals.append(head.val)
        head = head.next
    return vals == vals[::-1]

# 方法二：找中点 + 反转后半段 + 比较（空间 O(1)）
def isPalindrome2(head):
    slow = fast = head
    while fast and fast.next:      # 快慢指针找中点
        slow = slow.next
        fast = fast.next.next
    # 反转从 slow 开始的后半段
    prev = None
    while slow:
        nxt = slow.next
        slow.next = prev
        prev = slow
        slow = nxt
    # prev 是后半段的头，与 head 逐个比较
    left, right = head, prev
    while right:
        if left.val != right.val:
            return False
        left = left.next
        right = right.next
    return True
```

复杂度分析

方法一：时间 O(n)，空间 O(n)。方法二：时间 O(n)，空间 O(1)。

核心知识点

- 快慢指针找中点：快指针走 2 步、慢指针走 1 步，是链表题中定位中点的标准做法。
- 「反转后半段」的思想：把「从两端向中间比较」转化为「两段正向比较」，从而避免递归栈或额外数组。
- 奇数长度自动兼容：多出来的中间结点不影响比较。
- vals[::-1] 是 Python 反转序列的切片写法，简洁高效。

141. 环形链表 【简单】

题目描述

给你一个链表的头节点 head，判断链表中是否有环。如果链表中有某个节点，可以通过连续跟踪 next 指针再次到达，则链表中存在环。如果链表中存在环，则返回 true；否则返回 false。

> **示例：**
>
> 输入：head = [3,2,0,-4], pos = 1
>
> 输出：true
>
> 解释：链表中有一个环，其尾部连接到下标为 1 的节点。

解题思路

方法一（哈希表）：遍历链表，把走过的结点放进集合；如果再次遇到同一个结点，说明有环。

方法二（快慢指针，又称 Floyd 判圈算法，空间 O(1)，是标准答案）：

让快指针每次走 2 步、慢指针每次走 1 步。如果链表有环，快指针一定会在环内追上慢指针（相遇）；如果无环，快指针会先到达终点（null）。

为什么一定会相遇？可以把过程想象成两个人在环形跑道上跑，速度快的人最终一定会「套圈」追上慢的人。

具体步骤：

1. slow = fast = head。

2. 当 fast 和 fast.next 都不为空时循环：slow 走一步，fast 走两步。

3. 若 slow == fast（是同一个结点），返回 true。

4. 循环正常结束（fast 走到 null），说明无环，返回 false。

参考代码

```
def hasCycle(head):
    slow = fast = head
```

```
    while fast and fast.next:      # fast 能走两步才继续
        slow = slow.next
        fast = fast.next.next
        if slow is fast:           # 相遇 -> 有环
            return True
    return False                   # 无环

# 方法一：哈希表（空间 O(n)，更直观）
def hasCycle2(head):
    seen = set()
    while head:
        if id(head) in seen:
            return True
        seen.add(id(head))
        head = head.next
    return False
```

复杂度分析

快慢指针：时间 O(n)（有环时慢指针最多走一圈），空间 O(1)。哈希表法：时间 O(n)，空间 O(n)。

核心知识点

- Floyd 判圈算法（龟兔赛跑）：快慢指针是链表环检测的标准解法，空间 O(1)。
- 循环条件 fast and fast.next：保证 fast.next.next 不会报空指针错误，这是必须写的守卫条件。
- 「相遇」判断用 is 判断是否为同一对象，而不是比较值。
- 为什么一定相遇（数学证明）：设环长为 L，两者距离每轮缩短 1，最终必然追上。

142. 环形链表 II 【中等】

题目描述

给定一个链表的头节点 head，返回链表开始入环的第一个节点。如果链表无环，则返回 null。不允许修改链表。

> **示例：**
>
> 输入：head = [3,2,0,-4], pos = 1
> 输出：返回索引为 1 的链表节点

解题思路

在第一题（判断有无环）的基础上，还要找出「入环点」。这里用到一个很漂亮的数学结论。

设：头结点到入环点的距离是 a，入环点到相遇点的距离是 b，相遇点再回到入环点的距离是 c，环长 L = b + c。

相遇时：慢指针走了 a + b，快指针走了 a + b + n×L（多绕了 n 圈）。又因为快指针速度是慢指针的 2 倍：

$$2(a + b) = a + b + n\times L \rightarrow a = n\times L - b = (n-1)\times L + c$$

结论：从「头结点」和从「相遇点」同时出发、速度相同，它们一定会在「入环点」相遇。

具体步骤：

1. 先用快慢指针找到相遇点（无环则返回 None）。

2. 让一个指针从头结点出发，另一个从相遇点出发，每次都走一步。

3. 它们相遇的位置就是入环点，返回该结点。

参考代码

```
def detectCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:                # 找到相遇点
            p = head                    # 一个从头出发
            q = slow                    # 一个从相遇点出发
            while p is not q:
                p = p.next
                q = q.next
            return p                    # 相遇处即入环点
    return None                         # 无环
```

复杂度分析

时间复杂度 O(n)：两次线性遍历。空间复杂度 O(1)。

核心知识点

- Floyd 算法的第二阶段：相遇点 + 头结点同时出发找入环点，是本题的精髓。
- 推导等式 2(a+b) = a+b+nL：把「速度关系」转化为「路程关系」，是解此题的关键一步。
- a = (n-1)L + c 的几何含义：从头走 a 步等价于从相遇点走 c 步（再加上若干整圈），所以两者在入环点重合。

- 哈希表法（对照）：记录首次重复访问的结点即为入环点，但空间 O(n)。

21. 合并两个有序链表 【简单】

题目描述

将两个升序链表合并为一个新的升序链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

> **示例：**
>
> 输入：`l1 = [1,2,4], l2 = [1,3,4]`
> 输出：`[1,1,2,3,4,4]`

解题思路

思路和「归并排序的合并步骤」完全一致，用两个指针分别遍历两个链表，每次挑较小的结点接到结果链表尾部。

具体步骤（用 dummy 虚拟头结点）：

1. 建一个虚拟头结点 dummy，让 tail = dummy 作为结果链表的尾指针。

2. 当两个链表都非空时：比较两指针的值，把较小的结点接到 tail.next，然后该指针前移。

3. tail 也前移一格。

4. 循环结束后，肯定有一条链表还没走完，直接接到 tail.next 上即可（剩下的一定是有序且都比已接的大）。

5. 返回 dummy.next。

dummy 结点的好处：不需要为「结果链表的第一个结点」写特殊判断，代码更统一。

参考代码

```
def mergeTwoLists(l1, l2):
    dummy = ListNode(0)          # 虚拟头结点
    tail = dummy
    while l1 and l2:
        if l1.val <= l2.val:     # 挑较小的
```

```
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 if l1 else l2  # 接上剩余部分
    return dummy.next
```

复杂度分析

时间复杂度 O(m + n)：每个结点被访问一次。空间复杂度 O(1)：只重新连接指针，不新建结点（dummy 除外）。

核心知识点

- 虚拟头结点 dummy：链表题的头号利器，统一处理「第一个结点」的边界情况。
- 双指针归并：与归并排序的 merge 步骤同源，是 O(n) 合并两个有序序列的标准做法。
- 「直接接上剩余的链表」：不需要循环逐个搬运，是链表相对于数组的优势（O(1) 拼接）。
- 衍生题：合并 K 个升序链表（23）——可以用分治两两合并，或用优先队列。

2. 两数相加 【中等】

题目描述

给你两个非空的链表，表示两个非负的整数。它们每位数字都是按照逆序的方式存储的，并且每个节点只能存储一位数字。请你将两个数相加，并以相同形式返回一个表示和的链表。你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

> **示例：**
>
> 输入：l1 = [2,4,3], l2 = [5,6,4]
> 输出：[7,0,8]
> 解释：342 + 465 = 807，逆序存储即为 [7,0,8]。

解题思路

本质就是「手算竖式加法」的链表版本。因为是逆序存储，正好可以从个位开始逐位相加，天然与进位方向一致。

具体步骤：

1. 建 dummy 头结点和 tail 指针，用 carry 保存进位（初始 0）。

2. 当 l1 或 l2 还有结点，或者 carry 不为 0 时循环：

   - 取出 l1、l2 当前位的值（若为空则记为 0）。

   - s = v1 + v2 + carry。

   - 新结点的值 = s % 10，新的 carry = s // 10。

   - 接到结果链表尾部，并前移各指针。

3. 返回 dummy.next。

注意循环条件必须包含 carry != 0，否则最高位的进位会丢失（例如 5 + 5 = 10）。

参考代码

```
def addTwoNumbers(l1, l2):
    dummy = ListNode(0)
    tail = dummy
    carry = 0                          # 进位
    while l1 or l2 or carry:
        v1 = l1.val if l1 else 0
        v2 = l2.val if l2 else 0
        s = v1 + v2 + carry
        carry = s // 10                # 新的进位
        tail.next = ListNode(s % 10)   # 当前位
        tail = tail.next
        if l1: l1 = l1.next
        if l2: l2 = l2.next
    return dummy.next
```

复杂度分析

时间复杂度 O(max(m, n))：遍历较长的链表长度。空间复杂度 O(max(m, n))：新建的结果链表长度。

核心知识点

- 模拟竖式加法：逐位相加 + 进位传递，是「大数运算」类题目（字符串相加 415、二进制求和 67）的统一思路。
- 循环条件的三个分支：l1、l2、carry 任意一个还存在就继续，这是不漏最高位进位的写法。
- 空结点补 0：v1 = l1.val if l1 else 0，避免写两套循环。

- 逆序存储的优势：正好从最低位开始算，与进位方向一致；若是正序存储（题 445）则需要先反转链表或借助栈。

19. 删除链表的倒数第 N 个结点 【中等】

题目描述

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。进阶：你能尝试使用一趟扫描实现吗？

> **示例：**
>
> 输入：head = [1,2,3,4,5], n = 2
> 输出：[1,2,3,5]

解题思路

因为要「一趟扫描」，不能先求长度再走 L-n 步。用「快慢指针 + 固定的间隔」来解决。

具体步骤：

1. 建虚拟头结点 dummy，让 dummy.next = head（这样删除头结点时也不需特殊处理）。

2. 让 fast 和 slow 都从 dummy 出发。

3. 先让 fast 向前走 n+1 步，这样 fast 和 slow 之间就隔了 n 个结点。

4. 然后 fast 和 slow 同时每次走一步，直到 fast 走到 null。

5. 此时 slow 正好停在「倒数第 n 个结点的前一个」，执行 slow.next = slow.next.next 完成删除。

6. 返回 dummy.next。

为什么走 n+1 步？因为我们要让 slow 停在待删结点的前一个位置，方便做删除操作。例如 [1,2,3,4,5] 删倒数第 2 个（即 4），slow 应停在 3 上。

参考代码

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0)
    dummy.next = head
    fast = slow = dummy
    for _ in range(n + 1):        # fast 先走 n+1 步
        fast = fast.next
    while fast:                   # 两指针同步前进
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next    # 删除 slow 后面的结点
    return dummy.next
```

复杂度分析

时间复杂度 O(L)：L 为链表长度，只遍历一趟。空间复杂度 O(1)。

核心知识点

- 相隔固定距离的双指针：快指针先走 n+1 步，然后同步前进，就能一次定位倒数第 n 个结点。
- dummy 虚拟头结点：处理「删除头结点」的边界情况，避免写 if head == 待删 的特殊分支。
- 为什么是 n+1：为了让慢指针停在待删结点的「前驱」，这样才好改 next 指针。
- 同款思想：找链表中点（快指针走两步）、找倒数第 k 个（本题），都是「固定间隔」的变体。

24. 两两交换链表中的节点 【中等】

题目描述

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在不修改节点内部的值的情况下完成本题（即，只能进行节点交换）。

示例：

输入：head = [1,2,3,4]
输出：[2,1,4,3]

解题思路

关键在于「画图理清指针顺序」。每次处理一对相邻结点 (a, b)，需要把 b 放到 a 前面，同时要让这一对的前驱 pre 指向 b、a 指向下一对的开头。

具体步骤（以一对 [a, b]、前驱为 pre 为例）：

1. 先记录下一对的开头 nxt = b.next。

2. 让 pre.next = b（前驱指向新的第一个结点）。

3. 让 b.next = a（b 指向 a）。

4. 让 a.next = nxt（a 指向下一对）。

5. 更新 pre = a，进入下一轮。

指针修改的顺序很重要：如果先改 a.next 再改 pre.next，就会丢失 b 的位置。通用法则是「先保存将要被覆盖的信息，再修改指针」。

参考代码

```
def swapPairs(head):
    dummy = ListNode(0)
    dummy.next = head
    pre = dummy                         # 当前这一对的前驱
    while pre.next and pre.next.next:
        a = pre.next
        b = a.next
        nxt = b.next                    # 1. 保存下一对的开头
        pre.next = b                    # 2. 前驱 -> b
        b.next = a                      # 3. b -> a
        a.next = nxt                    # 4. a -> 下一对
        pre = a                         # 5. 前驱移到本对末尾
    return dummy.next
```

复杂度分析

时间复杂度 O(n)：每个结点只访问一次。空间复杂度 O(1)：原地交换指针。

核心知识点

- 画图理指针：链表指针变换类题目，务必先画出「修改前」和「修改后」的图，按图写代码。
- 指针修改顺序：遵循「先保存、后覆盖」的原则，避免丢失结点引用。
- 循环条件 pre.next and pre.next.next：保证至少还有两个结点可以交换，这是防止空指针的关键。
- 递归写法（拓展）：把前两个结点交换后，递归处理剩下的链表，代码更简洁但空间 O(n)。

25. K 个一组翻转链表 【困难】

题目描述

给你链表的头节点 head，每 k 个节点一组进行翻转，请你返回修改后的链表。k 是一个正整数，它的值小于或等于链表的长度。如果节点总数不是 k 的整数倍，那么请将最后剩余的节点保持原有顺序。你不能只是单纯的改变节点内部的值，而是需要实际进行节点交换。

示例：

```
输入：head = [1,2,3,4,5], k = 2
输出：[2,1,4,3,5]
输入：head = [1,2,3,4,5], k = 3
输出：[3,2,1,4,5]
```

解题思路

这是「两两交换」的推广版：不再固定 2 个一组，而是每 k 个一组翻转。

具体步骤：

1. 用 dummy 头结点，pre 指向「上一组的尾结点」，初始化为 dummy。

2. 每轮先检查「从 pre.next 开始是否还有 k 个结点」：用 end 指针从 pre 出发走 k 步，若中途为 null，说明剩余不足 k 个，直接结束（保持原顺序）。

3. 记录 start = pre.next，nxt = end.next（下一组的起点，同时也是本组反转后的尾指向）。

4. 把 [start, end] 这一段断开，做一次「反转链表」（标准三指针模板），得到 newHead、newTail。

5. 把反转后的这一组接回去：pre.next = newHead，newTail.next = nxt。

6. 更新 pre = newTail（即原来的 start），继续下一组。

小技巧：可以把本组尾结点的 next 临时置为 None，这样可以直接调用反转函数，不用额外处理边界。

参考代码

```
def reverseKGroup(head, k):
    dummy = ListNode(0)
    dummy.next = head
    pre = dummy                          # 上一组的尾结点

    while True:
        # 1. 检查剩余是否够 k 个
        end = pre
        for _ in range(k):
            end = end.next
            if not end:
                return dummy.next        # 不足 k 个，直接返回
        nxt = end.next                   # 下一组的起点
        start = pre.next
        # 2. 反转 [start, end] 这一段
        prev, cur = None, start
        while cur is not nxt:
            tmp = cur.next
            cur.next = prev
            prev = cur
            cur = tmp
        # 3. 接回链表: pre -> 本组新头(prev)，本组新尾(start).next = nxt
        pre.next = prev
        start.next = nxt
        pre = start                      # 本组尾结点成为下一组的前驱
    return dummy.next
```

复杂度分析

时间复杂度 O(n)：每个结点被访问常数次。空间复杂度 O(1)：原地翻转。

核心知识点

- 分组处理框架：先「判定本组是否足 k 个」，再「翻转本组」，最后「接回链表」，逻辑清晰不易错。
- 复用反转模板：区间反转本质上就是一次「限定终点的反转链表」。
- 临时断链与重连：把本组尾结点指向 None（或限定 cur 不为 nxt）可以直接套用标准反转代码。
- pre / start / end / nxt 四指针分工：前驱、本组头、本组尾、下一组头，画图后一目了然。
- 难度定位：Hot 100 中链表部分最难的一题，建议先把 206 和 92（区间反转）做熟再来。

138. 随机链表的复制 【中等】

题目描述

给你一个长度为 n 的链表，每个节点包含一个额外增加的随机指针 random，该指针可以指向链表中的任何节点或空节点。构造这个链表的深拷贝。深拷贝应该正好由 n 个全新节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 next 指针和 random 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。复制链表中的指针都不应指向原链表中的节点。

示例：

```
输入：head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
输出：[[7,null],[13,0],[11,4],[10,2],[1,0]]
```

解题思路

难点在于：新结点的 random 可能指向「还没创建」的结点。

方法一（哈希表，最直观）：

1. 第一趟遍历：为每个原结点创建一个新结点，用字典 map 建立「原结点 → 新结点」的映射。

2. 第二趟遍历：设置新结点的 next 和 random，即 map[cur].next = map[cur.next]，map[cur].random = map[cur.random]（若 cur.random 为 None 则设为 None）。

3. 返回 map[head]。

方法二（原地拼接，空间 O(1)，进阶）：把每个新结点插到对应原结点后面（A→A'→B→B'→...），这样 A'.random = A.random.next 就能直接找到。最后再把两条链表拆开。

初学建议先掌握方法一，思路清晰、不容易写错。

参考代码

```
# 方法一：哈希表（推荐，思路清晰）
class Node:
    def __init__(self, x, next=None, random=None):
        self.val = x
        self.next = next
```

```
        self.random = random

def copyRandomList(head):
    if not head:
        return None
    mp = {}
    cur = head
    while cur:                          # 第一趟：建立 原结点 -> 新结点 的映射
        mp[cur] = Node(cur.val)
        cur = cur.next
    cur = head
    while cur:                          # 第二趟：串接 next 和 random
        mp[cur].next = mp.get(cur.next)
        mp[cur].random = mp.get(cur.random)
        cur = cur.next
    return mp[head]

# 方法二：原地拼接（空间 O(1)）
def copyRandomList2(head):
    if not head:
        return None
    cur = head
    while cur:                          # 1. 在每个原结点后插入克隆结点
        clone = Node(cur.val)
        clone.next = cur.next
        cur.next = clone
        cur = clone.next
    cur = head
    while cur:                          # 2. 设置 random
        if cur.random:
            cur.next.random = cur.random.next
        cur = cur.next.next
    dummy = Node(0)
    tail = dummy
    cur = head
    while cur:                          # 3. 拆分出克隆链表
        tail.next = cur.next
        tail = tail.next
        cur.next = cur.next.next    # 恢复原链表
        cur = cur.next
    return dummy.next
```

复杂度分析

方法一：时间 O(n)，空间 O(n)。方法二：时间 O(n)，空间 O(1)（不计新建结点）。

核心知识点

- 「原结点 → 新结点」映射表：处理「指针可能指向尚未创建的对象」这类问题的通用手段。
- 深拷贝与浅拷贝：深拷贝要求所有引用都指向新对象，不能与原链表共享任何结点。
- 两趟遍历策略：第一趟建结点、第二趟连指针，把复杂依赖拆成两步，大大降低难度。
- mp.get(x) 的妙用：当 x 为 None 时返回 None，正好符合 random 可以为空的要求。
- 原地拼接法（进阶）：利用「克隆结点紧跟原结点」的位置关系，用 cur.random.next 代替哈希查找。

148. 排序链表 【中等】

题目描述

给你链表的头结点 head，请将其按升序排列并返回排序后的链表。进阶：你可以在 O(n log n) 时间复杂度和常数级空间复杂度下，对链表进行排序吗？

示例：

输入：head = [4,2,1,3]
输出：[1,2,3,4]

解题思路

数组排序可以用快排，但链表不支持随机访问，快排的 partition 效率低。链表排序最适合「归并排序（自顶向下分治）」，因为链表的「拆分」和「合并」都是 O(1) 操作。

具体步骤：

1. 递归终止条件：链表为空或只有一个结点时，本身就是有序的，直接返回。

2. 用「快慢指针」找到链表中点，并在中点处断开，得到左右两条子链表。

3. 递归排序左半部分和右半部分。

4. 把两个有序链表合并（复用「合并两个有序链表」的代码）。

找中点的细节：快指针从 head.next 开始（而不是 head），这样当结点个数为 2 时，slow 会停在第一个结点，避免死循环（否则 split 出的左半可能等于原链表）。

如果想做到空间 O(1)，需要用「自底向上」的迭代式归并：从长度为 1 的段开始，逐轮把段长翻倍。

参考代码

```
def sortList(head):
    if not head or not head.next:
        return head
    # 1. 快慢指针找中点，fast 从 head.next 出发保证偶数长度时中点偏左
```

```
    slow, fast = head, head.next
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    mid = slow.next
    slow.next = None                # 断开成两段
    # 2. 递归排序
    left = sortList(head)
    right = sortList(mid)
    # 3. 合并两个有序链表
    dummy = ListNode(0)
    tail = dummy
    while left and right:
        if left.val <= right.val:
            tail.next = left
            left = left.next
        else:
            tail.next = right
            right = right.next
        tail = tail.next
    tail.next = left if left else right
    return dummy.next
```

复杂度分析

时间复杂度 O(n log n)：分治深度 log n，每层合并总代价 O(n)。空间复杂度 O(log n)：递归栈（自底向上迭代法可降到 O(1)）。

核心知识点

- 链表归并排序：链表排序的最优选择，拆分/合并都是 O(1) 指针操作，无需额外数组。
- 快慢指针找中点 + 断链：把链表拆成两半的标准手法。
- 分治思想（Divide & Conquer）：把大问题拆成两个同类子问题，解决后合并。
- fast 从 head.next 出发的技巧：保证偶数个结点时中点取左半边的末尾，避免无限递归。
- 对比：数组排序可用快排（随机访问 O(1)），链表更适合归并（顺序访问即可）。

23. 合并 K 个升序链表 【困难】

题目描述

给你一个链表数组，每个链表都已经按升序排列。请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例：

输入：`lists = [[1,4,5],[1,3,4],[2,6]]`

输出：`[1,1,2,3,4,4,5,6]`

解释：将它们合并到一个有序链表中得到 `1→1→2→3→4→4→5→6`。

解题思路

方法一（顺序两两合并，较慢）：把 lists[0] 和 lists[1] 合并，结果再和 lists[2] 合并……设最长链表长度为 n、链表个数为 k，则总复杂度 $O(k^2n)$，当 k 大时会超时。

方法二（分治两两合并，推荐）：模仿归并排序的思路，把 k 个链表两两配对合并，再对结果继续两两合并，直到只剩一个。合并轮数为 log k，总复杂度 O(kn log k)。

方法三（优先队列 / 最小堆，也很常用）：

1. 把所有链表的头结点放进一个小顶堆（按结点值比较）。

2. 每次弹出堆顶（当前最小结点），接到结果链表尾部。

3. 如果弹出的结点还有 next，就把 next 也放进堆。

4. 堆空时结束。复杂度 O(kn log k)，但常数比方法二略大。

Python 中用 heapq 实现堆；由于结点无法直接比较，需要存 (val, 唯一序号, 结点) 三元组。

参考代码

```
import heapq

# 方法二：分治两两合并 (推荐)
def mergeKLists(lists):
    if not lists:
        return None

    def merge2(a, b):              # 合并两个有序链表
        dummy = ListNode(0)
        tail = dummy
        while a and b:
            if a.val <= b.val:
                tail.next, a = a, a.next
            else:
                tail.next, b = b, b.next
            tail = tail.next
        tail.next = a if a else b
        return dummy.next
```

```
    while len(lists) > 1:          # 每轮两两合并
        merged = []
        for i in range(0, len(lists), 2):
            if i + 1 < len(lists):
                merged.append(merge2(lists[i], lists[i + 1]))
            else:
                merged.append(lists[i])
        lists = merged
    return lists[0]

# 方法三：最小堆
def mergeKLists2(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = ListNode(0)
    tail = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

复杂度分析

分治法：时间 O(kn log k)，空间 O(log k)（递归/循环层数）。堆方法：时间 O(kn log k)，空间 O(k)。

核心知识点

- 分治合并：把 k 路合并拆成 log k 轮两两合并，是降低复杂度的核心思路。
- 优先队列（最小堆）：多路归并的标准工具，「每次取全局最小」正是堆的强项。
- 元组比较问题：堆元素需要可比较，存入 (val, 唯一序号, 结点)，序号用于避免结点之间的比较。
- 复杂度对比：顺序合并 $O(k^2n)$ vs 分治/堆 O(kn log k)，k 较大时差距非常明显。
- 方法论：一道题多种解法时，要能说清「哪种更好、好在哪里」——这是面试的高频考点。

146. LRU 缓存 【中等】

题目描述

请你设计并实现一个满足 LRU（最近最少使用）缓存约束的数据结构。

实现 LRUCache 类：

LRUCache(int capacity) 以正整数作为容量 capacity 初始化 LRU 缓存；

int get(int key) 如果关键字存在，则返回其值，否则返回 -1；

void put(int key, int value) 如果关键字已存在，则变更其值；否则插入该组。如果插入超出容量，则逐出最久未使用的关键字。

函数 get 和 put 必须以 O(1) 的平均时间复杂度运行。

> **示例：**
>
> 输入：["LRUCache","put","put","get","put","get","put","get","get","get"]
> [[2],[1,1],[2,2],[1],[3,3],[2],[4,4],[1],[3],[4]]
> 输出：[null,null,null,1,null,-1,null,-1,3,4]

解题思路

要求 O(1) 的查询和 O(1) 的「更新最近使用顺序」，单一数据结构做不到，需要「哈希表 + 双向链表」组合：

- 哈希表：实现 key → 结点的 O(1) 定位。

- 双向链表：维护「使用顺序」。链表头部放最近使用的，尾部放最久未使用的。

  - 访问一个 key：把对应结点移动到头部。

  - 插入新 key：若超出容量，先删掉尾部结点（最久未使用），再把新结点插到头部。

为什么用双向链表？因为需要在 O(1) 时间内「删除任意一个结点」，只有双向链表有这个能力（单链表需要找前驱）。

实现技巧：用两个「哨兵结点」head 和 tail 组成空的双向链表，可以省掉大量空指针判断。

Python 中也可以用 OrderedDict（有序字典）直接实现，非常简洁，但面试通常要求手写双向链表。

参考代码

```
# 手写哈希表 + 双向链表 (标准答案)
class DNode:
    def __init__(self, key=0, val=0):
        self.key = key
        self.val = val
        self.prev = None
        self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.cache = {}                     # key -> DNode
        self.head = DNode()                 # 哨兵: 头部=最近使用
        self.tail = DNode()                 # 哨兵: 尾部=最久未使用
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node):                 # 从链表中摘下 node
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_to_head(self, node):            # 把 node 插到头部
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.cache:
            return -1
        node = self.cache[key]
        self._remove(node)                   # 移到头部表示刚使用过
        self._add_to_head(node)
        return node.val

    def put(self, key, value):
        if key in self.cache:
            node = self.cache[key]
            node.val = value
            self._remove(node)
            self._add_to_head(node)
        else:
            if len(self.cache) == self.cap:  # 容量满, 淘汰最久未使用
                lru = self.tail.prev
                self._remove(lru)
                del self.cache[lru.key]
            node = DNode(key, value)
            self.cache[key] = node
            self._add_to_head(node)

# 方法二: 直接用 OrderedDict (面试可用作对比说明)
from collections import OrderedDict

class LRUCache2:
    def __init__(self, capacity):
        self.cap = capacity
        self.od = OrderedDict()

    def get(self, key):
```

```
        if key not in self.od:
            return -1
        self.od.move_to_end(key)   # 移到末尾表示最近使用
        return self.od[key]

    def put(self, key, value):
        if key in self.od:
            self.od.move_to_end(key)
        self.od[key] = value
        if len(self.od) > self.cap:
            self.od.popitem(last=False)  # 弹出最早使用的
```

复杂度分析

get 和 put 均为 O(1)：哈希查找 O(1)，双向链表的插入/删除 O(1)。空间复杂度 O(capacity)。

核心知识点

- 哈希表 + 双向链表的组合设计：查询快 + 顺序可维护，这是工程中最经典的缓存实现。
- 为什么需要双向链表：O(1) 删除任意结点必须知道前驱，单链表做不到。
- 哨兵结点（sentinel）：用头尾两个虚拟结点消除边界判断，是双向链表实现的标准技巧。
- LRU 淘汰策略：最久未被访问的（链表尾部）优先淘汰，每次访问都把结点提到头部。
- 同类题：LFU 缓存（460）——按访问频率淘汰，需要更复杂的结构。

第 8 章 二叉树

本章核心思想

二叉树题的解法高度模板化，掌握以下三条主线就能覆盖绝大多数题目：

1）DFS（深度优先，递归）：前序 / 中序 / 后序，写「递归函数」时想清楚「这个函数要做什么、返回什么」。

2）BFS（广度优先，层序遍历）：用队列按层访问，适合求深度、右视图、层内聚合等问题。

3）二叉搜索树（BST）：中序遍历结果一定有序，这是 BST 题目的万能钥匙。

解题时的思考顺序：先想「用递归还是用队列」，再想「递归函数的返回值代表什么」，最后处理「左右子树的信息如何汇总」。

94. 二叉树的中序遍历 【简单】

题目描述

给定一个二叉树的根节点 root，返回它的中序遍历结果。中序遍历的顺序是：左子树 → 根节点 → 右子树。

示例：

```
输入：root = [1,null,2,3]
输出：[1,3,2]
```

解题思路

中序遍历的三种写法都要掌握，因为面试中经常被要求「不用递归」。

递归法：写一个函数，先递归左子树，再访问根，最后递归右子树。

迭代法（用显式栈模拟递归）：

1. 用一个栈和一个指针 cur（初始为 root）。

2. 当 cur 不为空或栈不为空时循环：

- 不断把 cur 压栈并令 cur = cur.left，一直走到最左边。

- 弹出栈顶结点，访问它（加入结果）。

- 令 cur = 弹出结点的 right，开始处理右子树。

还有一个「Morris 遍历」可以在 O(1) 空间完成，原理是利用叶子结点的空指针建立临时线索，属于进阶内容。

参考代码

```
# 递归法
def inorderTraversal(root):
    res = []
    def dfs(node):
        if not node:
            return
        dfs(node.left)          # 左
        res.append(node.val)    # 根
        dfs(node.right)         # 右
    dfs(root)
    return res

# 迭代法 (显式栈)
def inorderTraversal2(root):
    res, stack = [], []
    cur = root
    while cur or stack:
        while cur:              # 一路向左
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()       # 弹出最左的结点
        res.append(cur.val)
        cur = cur.right         # 转向右子树
    return res
```

复杂度分析

时间复杂度 O(n)：每个结点访问一次。空间复杂度 O(h)：h 为树高，递归栈或显式栈的深度。

核心知识点

- 递归三要素：确定递归函数的含义、终止条件、以及单层逻辑（本题是「左-根-右」）。
- 用栈模拟递归：所有递归算法都能改写成迭代 + 栈的形式，这是面试常考的点。
- 三种遍历的差别只在「访问根的位置」：前序（根左右）、中序（左根右）、后序（左右根）。
- BST 的中序遍历有序 —— 这是二叉搜索树题目的核心性质，务必牢记。

104. 二叉树的最大深度 【简单】

题目描述

给定一个二叉树 root，返回其最大深度。最大深度是指从根节点到最远叶子节点的最长路径上的节点数。

> **示例：**
>
> 输入：root = [3,9,20,null,null,15,7]
>
> 输出：3

解题思路

方法一（递归 / DFS，推荐）：一棵树的最大深度 = max(左子树深度, 右子树深度) + 1。

递归终止条件：空结点深度为 0。

方法二（BFS 层序遍历）：一层一层地处理，每处理完一层就把层数加一，最后一层的层数就是最大深度。

递归的思路非常简洁：「把问题交给左右子树，自己只负责把结果加一」，这就是二叉树递归的典型思维方式。

参考代码

```
# 递归 (DFS)
def maxDepth(root):
    if not root:
        return 0
    return max(maxDepth(root.left), maxDepth(root.right)) + 1

# 层序遍历 (BFS)
from collections import deque

def maxDepth2(root):
    if not root:
        return 0
    q = deque([root])
    depth = 0
    while q:
        for _ in range(len(q)):     # 逐层处理
            node = q.popleft()
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        depth += 1
    return depth
```

复杂度分析

递归法：时间 O(n)，空间 O(h)。BFS 法：时间 O(n)，空间 O(n)（最宽一层的结点数）。

核心知识点

- 「后序位置」的递归：先拿到左右子树的结果，再汇总成自己的结果（自底向上）。
- 树高与空间复杂度：递归栈深度等于树高 h，最坏（退化成链）为 O(n)，平衡树为 O(log n)。
- 层序遍历按层处理的写法：for _ in range(len(q)) 保证一次只处理当前层，是 BFS 的标准模板。
- 这是理解「递归返回值」最好的入门题：返回值 = 以该结点为根的子树深度。

226. 翻转二叉树 【简单】

题目描述

给你一棵二叉树的根节点 root，翻转这棵二叉树，并返回其根节点。翻转是指交换每个结点的左右子树。

> **示例：**
>
> 输入：root = [4,2,7,1,3,6,9]
> 输出：[4,7,2,9,6,3,1]

解题思路

对树中的每一个结点，交换它的左右孩子即可。遍历方式可以用前序、后序或层序，只要「每个结点都被处理一次」就行。

递归步骤：

1. 终止条件：结点为空则返回。

2. 交换当前结点的左右孩子。

3. 递归翻转左子树、右子树。

注意：交换操作放在「递归之前」还是「之后」都可以（前序/后序都行），但放在「中序」位置会出问题——因为中序会先处理左子树，交换后原来的左子树变成了右子树，会导致某些结点被处理两次。

参考代码

```
# 递归 (前序)
def invertTree(root):
    if not root:
        return None
    root.left, root.right = root.right, root.left   # 交换左右孩子
    invertTree(root.left)
    invertTree(root.right)
    return root

# 层序遍历 (BFS)
from collections import deque

def invertTree2(root):
    if not root:
        return None
    q = deque([root])
    while q:
        node = q.popleft()
        node.left, node.right = node.right, node.left
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    return root
```

复杂度分析

时间复杂度 O(n)：每个结点访问一次。空间复杂度 O(h)（递归）或 O(n)（BFS）。

核心知识点

- 递归遍历的三种位置：前序（进入结点时处理）、中序（处理完左子树后）、后序（处理完左右子树后）。
- 为什么中序不行：交换会改变「左子树」的含义，导致同一结点被访问两次。这是本题最容易被忽略的坑。
- 树形结构的修改往往可以「就地」完成，不需要新建结点。
- 经典趣闻：本题因某位工程师未能写出而走红，被戏称为「Homebrew 之树」，是面试热身必备题。

101. 对称二叉树 【简单】

题目描述

给你一个二叉树的根节点 root，检查它是否轴对称。

示例：

输入：root = [1,2,2,3,4,4,3]

输出：true

输入：root = [1,2,2,null,3,null,3]

输出：false

解题思路

判断「整棵树是否对称」，本质是判断「左子树和右子树是否互为镜像」。

所以需要写一个「比较两棵树是否镜像」的辅助函数，而不是比较「两棵树是否相同」。

镜像比较的判断条件（设两结点为 p、q）：

1. 都为空 → 对称，返回 true。

2. 只有一个为空，或值不相等 → 不对称，返回 false。

3. 递归比较「p 的左子树与 q 的右子树」、「p 的右子树与 q 的左子树」是否都镜像。

注意第 3 步的对应关系：是「左对右、右对左」，这正是「镜像」的含义。如果写成「左对左」，那就变成了判断两棵树是否相同。

也可以用队列做迭代版本：每次成对地取出两个结点比较，然后按「左-右、右-左」的顺序入队。

参考代码

```
# 递归
def isSymmetric(root):
    if not root:
        return True

    def check(p, q):                # 判断 p 与 q 是否互为镜像
        if not p and not q:
            return True
        if not p or not q or p.val != q.val:
            return False
        # 关键: 外侧对内侧 -> 左对右, 右对左
        return check(p.left, q.right) and check(p.right, q.left)
```

```
    return check(root.left, root.right)

# 迭代 (队列)
from collections import deque

def isSymmetric2(root):
    if not root:
        return True
    q = deque([root.left, root.right])
    while q:
        p, qq = q.popleft(), q.popleft()
        if not p and not qq:
            continue
        if not p or not qq or p.val != qq.val:
            return False
        q.append(p.left)      # 成对入队：外侧
        q.append(qq.right)
        q.append(p.right)     # 成对入队：内侧
        q.append(qq.left)
    return True
```

复杂度分析

时间复杂度 O(n)：每个结点访问一次。空间复杂度 O(h)：递归栈深度。

核心知识点

- 双树递归：递归函数同时操作两个结点，是处理「比较两棵树」类问题的标准写法。
- 镜像的对应关系：左对右、右对左。写错成左对左是本题最常见的错误。
- 「都为空」「只有一个为空」的边界判断顺序：先判断都为空才能安全访问 .val。
- 对称的本质：一棵树与自己的镜像相等。

543. 二叉树的直径 【简单】

题目描述

给你一棵二叉树的根节点，返回该树的直径。二叉树的直径是指树中任意两个节点之间最长路径的长度。这条路径可能经过也可能不经过根节点。两节点之间路径的长度由它们之间边数表示。

示例：

输入：root = [1,2,3,4,5]
输出：3
解释：路径 [4,2,1,3] 或 [5,2,1,3] 的长度为 3。

解题思路

关键观察：任意一条路径都可以看作「以某个结点为转折点，向左右两侧各延伸一段」。

所以对每个结点，经过它的最长路径长度 = 左子树深度 + 右子树深度。对所有结点取最大值，就是直径。

注意区分两个概念：

- 某结点的「深度」= 从该结点向下走到最远叶子的边数（用于向上传递）。

- 「直径」= 全局最大值（用于记录答案）。

具体步骤：写一个递归函数 depth(node) 返回该结点的深度：

1. 空结点返回 0。

2. 递归求 leftDepth 和 rightDepth。

3. 用 leftDepth + rightDepth 更新全局最优答案。

4. 返回 max(leftDepth, rightDepth) + 1 给上一层。

「一边递归、一边用全局变量记录答案」是树形 DP 的通用模式。

参考代码

```
def diameterOfBinaryTree(root):
    ans = 0

    def depth(node):                # 返回以 node 为根的子树深度
        nonlocal ans
        if not node:
            return 0
        l = depth(node.left)
        r = depth(node.right)
        ans = max(ans, l + r)       # 经过 node 的路径长度
        return max(l, r) + 1        # 向上返回深度

    depth(root)
    return ans
```

复杂度分析

时间复杂度 O(n)：每个结点访问一次。空间复杂度 O(h)：递归栈深度。

核心知识点

- 树形 DP：在递归返回的过程中同时维护一个「全局最优答案」，是树类问题的高频模式。
- 「返回什么」与「更新什么」分离：函数返回深度供父结点使用，全局变量记录直径作为最终答案。
- 路径的转折点思维：任意路径必有一个最高点（LCA），把它作为枚举对象。
- nonlocal 关键字：Python 中在嵌套函数里修改外层变量必须声明。
- 同类题：二叉树中的最大路径和（124）在此基础上加入了「负数剪枝」。

102. 二叉树的层序遍历 【中等】

题目描述

给你二叉树的根节点 root，返回其节点值的层序遍历结果（即逐层地，从左到右访问所有节点）。

示例：

```
输入：root = [3,9,20,null,null,15,7]
输出：[[3],[9,20],[15,7]]
```

解题思路

层序遍历（BFS）的标准模板：用队列逐层处理。

具体步骤：

1. 根结点入队。

2. 当队列非空时循环：

   - 先记录当前队列的长度 size = len(q)，这正好是「当前这一层」的结点数。

   - 循环 size 次：出队一个结点，把它的值加入当前层的列表，并把它的左右孩子入队。

   - 把当前层的列表加入结果。

3. 返回结果。

核心技巧就是「先取 len(q)」：这样内层循环只处理当前层，不会把下一层的结点也处理掉。这个模板可以直接套用到「右视图」「每层最大值」「锯齿形遍历」等大量题目上。

参考代码

```
from collections import deque

def levelOrder(root):
    if not root:
        return []
    res = []
    q = deque([root])
    while q:
        size = len(q)               # 当前层的结点数
        level = []
        for _ in range(size):
            node = q.popleft()
            level.append(node.val)
            if node.left:  q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)           # 加入这一层的结果
    return res
```

复杂度分析

时间复杂度 O(n)：每个结点入队出队各一次。空间复杂度 O(n)：队列最大长度为一层的结点数。

核心知识点

- BFS 层序遍历模板：队列 + 「先取 len(q)」是必须背下来的标准写法。
- deque 双端队列：popleft() 是 O(1)，而 list.pop(0) 是 O(n)，所以 BFS 一定要用 deque。
- 层内聚合：把「每层的中间结果」收集到一个列表，是处理层级信息的通用方式。
- 模板复用：右视图（199）取每层最后一个、层最大值（515）取每层 max、锯齿遍历（103）隔层反转，都是此模板的变形。

108. 将有序数组转换为二叉搜索树 【简单】

题目描述

给你一个整数数组 nums，其中元素已经按升序排列，请你将其转换为一棵平衡二叉搜索树（高度平衡：每个节点的左右两个子树的高度差的绝对值不超过 1）。

示例：

输入：nums = [-10,-3,0,5,9]

输出：[0,-3,9,-10,null,5]

解题思路

BST 的性质：中序遍历有序。反过来，如果用一个有序数组构造 BST，那么数组中间的数值应该是根（这样左右两边的数量最接近，树最平衡）。

具体步骤（分治 + 递归）：

1. 取当前区间 [left, right] 的中点 mid = (left + right) // 2，用 nums[mid] 作为根。

2. 用 [left, mid-1] 递归构造左子树。

3. 用 [mid+1, right] 递归构造右子树。

4. 返回根结点。

这样构造出来的树高度是 O(log n)，天然满足「高度平衡」的要求。如果题目不要求平衡，也可以取端点做根（退化成链表），但那样就没有意义了。

参考代码

```
def sortedArrayToBST(nums):
    def build(left, right):
        if left > right:
            return None
        mid = (left + right) // 2      # 取中点作为根，保证平衡
        node = TreeNode(nums[mid])
        node.left = build(left, mid - 1)
        node.right = build(mid + 1, right)
        return node

    return build(0, len(nums) - 1)
```

复杂度分析

时间复杂度 O(n)：每个元素恰好用来建立一个结点。空间复杂度 O(log n)：递归栈深度等于树高。

核心知识点

- 分治构造：把「构造整棵树」拆成「构造左子树 + 构造右子树」，是树构造类问题的通则。
- 取中点 → 平衡：中点作为根，左右子树结点数最多相差 1，因此高度平衡。

- BST 与有序数组的对应关系：BST 的中序遍历 = 有序数组。
- 升序数组构造出来的树，中序遍历一定还是原数组 —— 可以用来验证答案。
- 同类题：有序链表转换二叉搜索树（109），需先用快慢指针找链表中点。

98. 验证二叉搜索树 【中等】

题目描述

给你一个二叉树的根节点 root，判断它是否是一个有效的二叉搜索树。有效的 BST 定义：节点的左子树只包含小于当前节点的数，节点的右子树只包含大于当前节点的数，所有左子树和右子树自身也必须是二叉搜索树。

示例：

```
输入：root = [2,1,3]
输出：true
输入：root = [5,1,4,null,null,3,6]
输出：false
解释：根节点是 5，右子树有 3 < 5，不合法。
```

解题思路

常见错误做法：只比较「每个结点与它的左右孩子」的大小关系。这样能通过 [2,1,3]，但会漏掉「右子树里有个比根还小的结点」这种情况（例如 [5,1,4,null,null,3,6] 中的 3）。

正确做法一（上下界递归）：递归时携带一个允许的取值区间 (low, high)。

1. 访问结点时判断是否满足 low < node.val < high，不满足就返回 False。

2. 递归左子树时，上界收缩为 node.val（左子树所有值都要小于根）。

3. 递归右子树时，下界收缩为 node.val。

4. 初始区间是 (-∞, +∞)。

正确做法二（中序遍历）：BST 的中序遍历结果必须严格递增。只要记录前一个访问的值 pre，每次比较 node.val > pre 即可，不满足就直接返回 False。

做法二更简洁，也更体现 BST 的本质性质，推荐优先掌握。

参考代码

```
# 方法一：上下界递归
def isValidBST(root):
    def check(node, low, high):
        if not node:
            return True
        if not (low < node.val < high):
            return False
        return (check(node.left, low, node.val)     # 左子树上界 = 当前值
                and check(node.right, node.val, high))  # 右子树下界 = 当前值
    return check(root, float('-inf'), float('inf'))

# 方法二：中序遍历，检查是否严格递增
def isValidBST2(root):
    pre = float('-inf')
    ok = True
    def dfs(node):
        nonlocal pre, ok
        if not node or not ok:
            return
        dfs(node.left)                  # 中序：先左
        if node.val <= pre:             # 必须严格大于前驱
            ok = False
        pre = node.val                  # 更新前驱
        dfs(node.right)
    dfs(root)
    return ok
```

复杂度分析

时间复杂度 O(n)：每个结点访问一次。空间复杂度 O(h)：递归栈深度。

核心知识点

- BST 的本质：不只是「左 < 根 < 右」，而是「整个左子树的任意值都 < 根，整个右子树都 > 根」。
- 上下界传递法：通过参数把约束逐层收窄，是处理「全局约束」类树问题的通用技巧。
- 中序遍历有序性：BST 的充要条件之一，用它验证最简洁。
- 严格递增：必须用 <= 判断，因为 BST 不允许存在相等的结点。
- float('-inf') / float('inf') 作为哨兵上下界，避免对整数范围做硬编码假设。

230. 二叉搜索树中第 K 小的元素 【中等】

题目描述

给定一个二叉搜索树的根节点 root，和一个整数 k，请你设计一个算法查找其中第 k 小的元素（从 1 开始计数）。

示例：

```
输入：root = [3,1,4,null,2], k = 1
输出：1
```

解题思路

利用 BST 的核心性质：中序遍历得到的序列是严格递增的。所以「第 k 小的元素」就是中序遍历序列的第 k 个。

具体步骤（迭代中序，可以提前返回）：

1. 用栈模拟中序遍历，一路向左压栈。

2. 每次弹出一个结点，就把计数减一。

3. 当计数减到 0 时，当前结点就是第 k 小，直接返回。

为什么用迭代而不是递归？因为迭代可以在找到答案时立刻 return，不必遍历完整棵树，平均时间可以更优（虽然最坏仍是 O(n)）。

进阶提升：如果这棵树会被频繁修改查询，可以给每个结点维护「子树结点数」，这样查找可以做到 O(h)，属于「有序数组 / 分块」思想在树上的应用。

参考代码

```
def kthSmallest(root, k):
    stack = []
    cur = root
    while cur or stack:
        while cur:                  # 一路向左
            stack.append(cur)
            cur = cur.left
        cur = stack.pop()           # 中序位置：弹出即访问
        k -= 1
        if k == 0:
```

```
            return cur.val          # 找到第 k 小
        cur = cur.right
    return -1
```

复杂度分析

时间复杂度 O(h + k)：走到最左是 O(h)，之后每弹出一次即访问一个结点。空间复杂度 O(h)。

核心知识点

- BST 中序遍历 = 升序序列：这是所有「第 k 小 / 第 k 大」类问题的根本依据。
- 迭代中序遍历 + 提前终止：不必遍历完整棵树，找到即可返回。
- 「第 k 大」的做法：把中序改成「右-根-左」的逆序中序遍历即可。
- 进阶优化：给结点维护子树大小（size），可实现 O(log n) 的排名查询。

199. 二叉树的右视图 【中等】

题目描述

给定一个二叉树的根节点 root，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

> **示例：**
>
> 输入：root = [1,2,3,null,5,null,4]
> 输出：[1,3,4]

解题思路

「从右侧看」意味着：每一层只能看到最右边的那一个结点。所以只需在层序遍历时，取每层的最后一个结点。

方法一（BFS，最直观）：

1. 用队列做层序遍历。

2. 对每一层，记录该层的结点列表，取最后一个加入结果。

3. 也可以在内层循环中判断「如果是本层最后一个」，直接加入结果。

方法二（DFS，前序 + 深度判断）：

按「根-右-左」的顺序做 DFS，并记录当前深度。如果某个深度是「第一次访问」，那么这个结点一定是该层最右边的结点，把它加入结果。

因为优先访问右子树，所以每层第一个被访问到的就是最右结点。

参考代码

```
# 方法一：BFS 取每层最后一个
from collections import deque

def rightSideView(root):
    if not root:
        return []
    res = []
    q = deque([root])
    while q:
        size = len(q)
        for i in range(size):
            node = q.popleft()
            if i == size - 1:          # 本层最后一个
                res.append(node.val)
            if node.left:  q.append(node.left)
            if node.right: q.append(node.right)
    return res

# 方法二：DFS 根-右-左，每层第一个访问的即为右视图
def rightSideView2(root):
    res = []
    def dfs(node, depth):
        if not node:
            return
        if depth == len(res):          # 这一层第一次被访问
            res.append(node.val)
        dfs(node.right, depth + 1)     # 先右后左
        dfs(node.left, depth + 1)
    dfs(root, 0)
    return res
```

复杂度分析

时间复杂度 O(n)。空间复杂度：BFS 为 O(n)（最宽层），DFS 为 O(h)。

核心知识点

- 层序遍历的应用：每层的「最后一个」是右视图，「第一个」是左视图。
- depth == len(res) 的判断技巧：利用「结果数组长度」代表「已记录层数」，判断是否首次访问该层。
- DFS 的访问顺序决定结果：先右后左 → 右视图；先左后右 → 左视图。

- 同一问题的两种视角：BFS 从「层」的角度看，DFS 从「深度」的角度看，都能解题。

114. 二叉树展开为链表 【中等】

题目描述

给你二叉树的根结点 root，请你将它展开为一个单链表：展开后的单链表应该同样使用 TreeNode，其中 right 子指针指向链表中下一个结点，而左子指针始终为 null。展开后的单链表应该与二叉树先序遍历顺序相同。

> **示例：**
>
> 输入：root = [1,2,5,3,4,null,6]
> 输出：[1,null,2,null,3,null,4,null,5,null,6]

解题思路

展开后的顺序是「前序遍历」顺序，所以自然想到先做前序遍历得到序列，再串成链表。但那样需要额外空间。下面给出「原地 O(1) 空间」的经典解法。

关键观察（找「右子树的前驱」）：对当前结点 cur：

1. 如果 cur 没有左孩子，直接令 cur = cur.right，处理下一个。

2. 如果有左孩子，就找到左子树中「最右边」的结点（也就是前序遍历中 cur 的直接后继、右子树的接入点），记为 pre。

3. 把 cur 的右子树整段挂到 pre.right 上（即 pre.right = cur.right）。

4. 把 cur 的左子树移到右边（cur.right = cur.left），并把 cur.left 置为 None。

5. cur = cur.right，继续处理。

这个解法本质上是「Morris 遍历」的思路，用已存在的结点关系代替栈，做到 O(1) 空间。

如果面试时觉得难以现场推导，用「前序遍历存列表 + 重新串接」的 O(n) 空间解法也是可以接受的。

参考代码

```
# 原地 O(1) 空间解法
def flatten(root):
    cur = root
    while cur:
        if cur.left:
            # 1. 找到左子树的最右结点 (右子树的接入点)
            pre = cur.left
            while pre.right:
                pre = pre.right
            # 2. 把右子树挂到 pre 右边
            pre.right = cur.right
            # 3. 左子树移到右边, 左指针置空
            cur.right = cur.left
            cur.left = None
        cur = cur.right           # 4. 处理下一个

# 方法二: 前序遍历 + 重新串接 (空间 O(n), 更易理解)
def flatten2(root):
    nodes = []
    def preorder(node):
        if not node:
            return
        nodes.append(node)
        preorder(node.left)
        preorder(node.right)
    preorder(root)
    for i in range(len(nodes) - 1):
        nodes[i].left = None
        nodes[i].right = nodes[i + 1]
    if nodes:
        nodes[-1].left = None
        nodes[-1].right = None
```

复杂度分析

原地法：时间 O(n)（每个结点最多被访问两次），空间 O(1)。列表法：时间 O(n)，空间 O(n)。

核心知识点

- Morris 思想：利用已有的空指针/结点关系代替递归栈，实现 O(1) 空间遍历。
- 「找左子树最右结点」的意义：它是前序遍历中当前结点的直接后继，右子树应接在它后面。
- 原地改造二叉树：修改指针时要注意先把需要的信息保存好，再改变指向。
- 工程取舍：面试中若原地解法推导不出来，先写出 O(n) 空间的正确解，再尝试优化，也是加分项。

105. 从前序与中序遍历序列构造二叉树 【中等】

题目描述

给定两个整数数组 preorder 和 inorder，其中 preorder 是二叉树的先序遍历，inorder 是同一棵树的中序遍历，请构造二叉树并返回其根节点。假设树中没有重复的元素。

示例：

```
输入：preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
输出：[3,9,20,null,null,15,7]
```

解题思路

需要用到前序和中序的「结构性质」：

- 前序遍历的第一个元素一定是「根」。

- 在中序遍历中找到这个根，左边就是「左子树的中序」，右边是「右子树的中序」。

- 左子树的结点个数确定了，就能在前序数组里切出「左子树的前序」和「右子树的前序」。

具体步骤：

1. 用哈希表记录「中序数组中每个值对应的下标」，这样定位根的位置是 O(1)。

2. 递归函数 build(preL, preR, inL, inR)：

   - 若区间为空，返回 None。

   - 根的值 = preorder[preL]，在中序中的位置为 idx。

   - 左子树结点个数 leftSize = idx - inL。

   - 递归构造左子树：前序区间 [preL+1, preL+leftSize]，中序区间 [inL, idx-1]。

   - 递归构造右子树：前序区间 [preL+leftSize+1, preR]，中序区间 [idx+1, inR]。

3. 返回根结点。

区间下标的计算是本题最容易出错的地方，建议在纸上画出数组分区示意图再写代码。

参考代码

```
def buildTree(preorder, inorder):
    idx_map = {v: i for i, v in enumerate(inorder)}  # 值 -> 中序下标

    def build(preL, preR, inL, inR):
        if preL > preR:
            return None
        root_val = preorder[preL]          # 前序第一个是根
        root = TreeNode(root_val)
        idx = idx_map[root_val]            # 根在中序中的位置
        left_size = idx - inL              # 左子树结点数
        root.left = build(preL + 1, preL + left_size, inL, idx - 1)
        root.right = build(preL + left_size + 1, preR, idx + 1, inR)
        return root

    return build(0, len(preorder) - 1, 0, len(inorder) - 1)
```

复杂度分析

时间复杂度 O(n)：每个结点构造一次，哈希查找 O(1)。空间复杂度 O(n)：哈希表 + 递归栈。

核心知识点

- 遍历序列的结构性质：前序首元素是根；中序中根的位置划分左右子树。
- 哈希表消除查找开销：把「在中序中找根」从 O(n) 降到 O(1)，整体从 O(n²) 优化到 O(n)。
- 区间下标推导：leftSize = idx - inL 是关键公式，建议画图理解。
- 同类题：从中序与后序遍历序列构造二叉树（106）——后序的最后一个元素是根。
- 「前序 + 后序」无法唯一确定一棵二叉树，因为无法区分只有一个孩子的结点属于左还是右。

437. 路径总和 III 【中等】

题目描述

给定一个二叉树的根节点 root，和一个整数 targetSum，求该二叉树里节点值之和等于 targetSum 的路径的数目。路径不需要从根节点开始，也不需要在叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

示例：

输入：root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8

输出：3

解题思路

方法一（暴力 DFS，易懂）：以每个结点为起点，向下搜索所有可能路径，统计和为 targetSum 的数量。时间复杂度 $O(n^2)$（每个结点作为起点都要遍历它的子树）。

方法二（前缀和 + 回溯，最优 O(n)）：这是本题的精髓。把树看成从根向下的路径，则任意一段「从上到下」的路径和 = 两个前缀和的差。

于是问题转化为：在「从根到当前结点」这条路径上，有多少个前缀和等于「当前前缀和 - targetSum」。

具体步骤：

1. 用字典 cnt 记录「从根到当前结点路径上，各个前缀和出现的次数」，初始 cnt[0] = 1（空路径）。

2. 进入一个结点时：cur += node.val，然后 ans += cnt.get(cur - targetSum, 0)。

3. 把 cur 记录进 cnt。

4. 递归处理左右子树。

5. 离开结点时（回溯），把 cnt[cur] 减一（恢复现场），保证 cnt 只反映「当前路径」的前缀和。

第 5 步的回溯至关重要，否则会统计到「不共线的路径」。

参考代码

```
# 方法一：暴力 DFS (O(n^2)，直观)
def pathSum(root, targetSum):
    def count_from(node, remain):       # 从 node 出发向下，剩余和为 remain
        if not node:
            return 0
        c = 1 if node.val == remain else 0
        c += count_from(node.left, remain - node.val)
        c += count_from(node.right, remain - node.val)
        return c

    if not root:
        return 0
    return (count_from(root, targetSum)
            + pathSum(root.left, targetSum)
            + pathSum(root.right, targetSum))

# 方法二：前缀和 + 回溯 (O(n))
```

```
from collections import defaultdict

def pathSum2(root, targetSum):
    cnt = defaultdict(int)
    cnt[0] = 1                          # 空前缀和出现 1 次
    ans = 0

    def dfs(node, cur):
        nonlocal ans
        if not node:
            return
        cur += node.val
        ans += cnt[cur - targetSum]     # 有多少前缀和等于 cur - target
        cnt[cur] += 1                   # 把自己记入路径
        dfs(node.left, cur)
        dfs(node.right, cur)
        cnt[cur] -= 1                   # 回溯：离开路径时撤销

    dfs(root, 0)
    return ans
```

复杂度分析

方法一：时间 $O(n^2)$，空间 $O(h)$。方法二：时间 $O(n)$，空间 $O(n)$（哈希表 + 递归栈）。

核心知识点

- 前缀和思想在树上的应用：把「路径和」转化为「两个前缀和之差」，与数组题（560）完全同源。
- 回溯（恢复现场）：在树的 DFS 中，进入时修改状态、离开时恢复状态，保证状态只反映当前路径。
- cnt[0] = 1 的哨兵：处理「路径恰好从根开始」的情况。
- 复杂度对比：暴力 $O(n^2)$ vs 前缀和 $O(n)$，是「用哈希表换时间」的又一例证。
- 必须回溯的原因：不同分支的路径不能混在一起统计。

236. 二叉树的最近公共祖先 【中等】

题目描述

给定一个二叉树，找到该树中两个指定节点的最近公共祖先。最近公共祖先：对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。

示例：

```
输入：root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
输出：3
```

解题思路

后序遍历思路：对于每个结点，判断它的左右子树中是否找到了 p 和 q。

递归函数的定义：dfs(node) 返回「以 node 为根的子树中，p 和 q 的最近公共祖先」；如果只找到其中一个（或都没找到），就返回找到的那个结点（或 None）。

分情况讨论：

1. 若 node 为空、或 node 就是 p、或 node 就是 q —— 返回 node。（找到目标或到底了）

2. 递归左子树得到 left，递归右子树得到 right。

3. 若 left 和 right 都不为空：说明 p、q 分别在两侧，当前 node 就是最近公共祖先，返回 node。

4. 若只有一侧不为空：说明 p、q 都在那一侧（或只找到了一个），返回那一侧的结果。

5. 两侧都为空：返回 None。

直观理解：如果一个结点的左子树里能找到 p（或 q）、右子树里能找到另一个，那这个结点就是分岔点，也就是最近公共祖先。

参考代码

```
def lowestCommonAncestor(root, p, q):
    if not root or root is p or root is q:
        return root                     # 到底了或找到了目标
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right:
        return root                     # p、q 分居两侧 -> 当前结点即答案
    return left if left else right      # 都在某一侧
```

复杂度分析

时间复杂度 O(n)：最坏情况遍历所有结点。空间复杂度 O(h)：递归栈深度。

核心知识点

- 后序遍历的天然优势：先拿到左右子树的结果，再决定自己返回什么。
- 「返回值语义」的设计：返回找到的目标结点或 LCA，让父结点可以据此判断。
- 分岔点即 LCA：p、q 分别位于左右子树时，当前结点就是最近的公共祖先。
- 一个结点可以是自己的祖先：所以 root is p 时直接返回 p。
- BST 的简化版（题 235）：利用「值的大小关系」可以直接判断往左还是往右，无需遍历整棵树。

124. 二叉树中的最大路径和 【困难】

题目描述

二叉树中的路径被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径中至多出现一次。该路径至少包含一个节点，且不一定经过根节点。路径和是路径中各节点值的总和。给你一个二叉树的根节点 root，返回其最大路径和。

> **示例：**
>
> ```
> 输入：root = [-10,9,20,null,null,15,7]
> 输出：42
> 解释：路径 15→20→7，和为 42。
> ```

解题思路

与「二叉树的直径」（543）同源，但这里路径和可能为负，所以需要「负数剪枝」。

对每个结点 node，考虑「以 node 为最高点」的路径：它由「向左下延伸的最佳一段」和「向右下延伸的最佳一段」以及 node 自己组成。

定义 dfs(node)：返回「以 node 为起点、向下延伸的单边最大路径和」——注意是单边，因为向上返回时不能分叉。

具体步骤：

1. 空结点返回 0。

2. left = max(0, dfs(node.left))，right = max(0, dfs(node.right))。**这里的 max(0, ·) 就是剪枝**：如果子树的最大贡献是负数，宁可不选它（路径到当前结点为止）。

3. 用 left + node.val + right 更新全局答案（这是「穿过 node 的完整路径」）。

4. 返回 node.val + max(left, right)（只能选一边向上延伸）。

理解重点：**「更新答案」时可以用两条边，「向上返回」时只能用一条边**，因为路径是一条链，不能出现分叉。

参考代码

```
def maxPathSum(root):
    ans = float('-inf')

    def dfs(node):                # 返回：以 node 为起点向下延伸的单边最大和
        nonlocal ans
        if not node:
            return 0
        left = max(0, dfs(node.left))     # 负数不如不选
        right = max(0, dfs(node.right))
        ans = max(ans, left + node.val + right)   # 穿过 node 的完整路径
        return node.val + max(left, right)        # 向上只能走一边

    dfs(root)
    return ans
```

复杂度分析

时间复杂度 O(n)：每个结点访问一次。空间复杂度 O(h)：递归栈深度。

核心知识点

- 单边返回 vs 双边更新：向上返回只能是单边（路径不能分叉），更新答案时可以用双边。这是本题最核心的区分点。
- 负数剪枝：max(0, 子树贡献)，避免把「拖后腿」的负数纳入路径。
- 树形 DP 的通用模式：dfs 返回值是「对父结点的贡献」，全局变量记录「完整路径的答案」。
- 初始值用 float('-inf')：因为答案可能全是负数，不能初始化为 0。
- 与「二叉树的直径」对比：直径求边数，本题求和；两者框架完全一致。

第 9 章 图

本章核心思想

图相关的题目在 Hot 100 中主要集中在「网格搜索」和「拓扑排序」两类。

1）网格 DFS/BFS：把二维网格看成图，每个格子是结点，上下左右相邻是边。「岛屿数量」「腐烂的橘子」都是此类，关键是掌握「访问标记」与「方向数组」。

2）拓扑排序：处理「有依赖关系的任务排序」，用「入度 + 队列」（Kahn 算法）或 DFS 检测环。

3）前缀树（Trie）：字符串集合的高效存储结构，也是「多叉树 + 哈希」的典型设计。

200. 岛屿数量 【中等】

题目描述

给你一个由 '1'（陆地）和 '0'（水）组成的二维网格 grid，请你计算网格中岛屿的数量。岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。此外，你可以假设该网格的四条边均被水包围。

示例：

```
输入：grid = [["1","1","0"],["1","0","0"],["0","0","1"]]
输出：2
```

解题思路

核心思路：遍历整个网格，每遇到一块还没访问过的陆地 '1'，就说明发现了一座新岛屿，然后从它出发用 DFS（或 BFS）把「与它相连的所有陆地」全部标记为已访问，避免重复计数。

具体步骤：

1. ans = 0，遍历每个格子 (i, j)。

2. 若 grid[i][j] == '1'：ans += 1，然后调用「沉岛」函数把这块陆地及其连通区域全部改成 '0'。

3. 沉岛函数（DFS）：若越界或当前格子不是 '1'，直接返回；否则把当前格子置为 '0'，再递归处理上下左右四个方向。

4. 遍历结束返回 ans。

「把访问过的陆地改成 0」是一种「原地标记」，能省掉额外的 visited 数组。

如果不想修改输入，也可以用单独的 visited 布尔数组。

参考代码

```
def numIslands(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    ans = 0

    def sink(i, j):                 # 把与 (i,j) 相连的陆地全部"沉没"
        if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] != '1':
            return
        grid[i][j] = '0'            # 原地标记为已访问
        sink(i + 1, j)
        sink(i - 1, j)
        sink(i, j + 1)
        sink(i, j - 1)

    for i in range(m):
        for j in range(n):
            if grid[i][j] == '1':
                ans += 1
                sink(i, j)
    return ans

# BFS 版本 (避免递归栈过深)
from collections import deque

def numIslands2(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    ans = 0
    dirs = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    for i in range(m):
        for j in range(n):
            if grid[i][j] != '1':
                continue
            ans += 1
            q = deque([(i, j)])
            grid[i][j] = '0'
            while q:
                x, y = q.popleft()
                for dx, dy in dirs:
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == '1':
                        grid[nx][ny] = '0'
                        q.append((nx, ny))
    return ans
```

复杂度分析

时间复杂度 O(m×n)：每个格子最多被访问一次。空间复杂度 O(m×n)：最坏情况下（全是陆地）递归栈或队列的深度。

核心知识点

- 网格上的图遍历：把二维网格视作图，「上下左右」即四条边，这是所有网格题的思维基础。
- 连通分量计数：每发现一个未访问的连通块就计数加一，是标准的 Flood Fill（漫水填充）思路。
- 原地标记：用修改输入数组的方式代替 visited 数组，节省空间。
- 方向数组 dirs：把四个方向写成列表，代码更简洁、不易漏方向。
- 递归深度风险：网格很大时 DFS 可能栈溢出，改写 BFS 更稳妥。

994. 腐烂的橘子 【中等】

题目描述

在给定的 m x n 网格 grid 中，每个单元格可以有以下三个值之一：值 0 代表空单元格；值 1 代表新鲜橘子；值 2 代表腐烂的橘子。每分钟，腐烂的橘子会使其上下左右四个方向上相邻的新鲜橘子腐烂。返回直到单元格中没有新鲜橘子为止所必须经过的最小分钟数。如果不可能，返回 -1。

> **示例：**
>
> 输入：grid = [[2,1,1],[1,1,0],[0,1,1]]
> 输出：4

解题思路

这是「多源 BFS（广度优先搜索）」的经典题：所有腐烂的橘子同时开始向外扩散，求最远的新鲜橘子被波及的时间。

为什么用 BFS 而不是 DFS？因为腐烂是「按层同时扩散」的过程，每一分钟向外扩一层，BFS 的「层」正好对应「时间」。而且是多个源点，所以要「多源入队」。

具体步骤：

1. 第一遍扫描网格：统计新鲜橘子的数量 fresh，同时把所有腐烂橘子的坐标入队。

2. 若 fresh == 0，直接返回 0（没有新鲜橘子，不需要时间）。

3. BFS：记录当前队列的大小作为「这一分钟要处理的橘子数」，逐个出队并向四周扩散；被感染的新鲜橘子入队，fresh 减一。处理完这一层后，分钟数加一（前提是队列里还有下一层）。

4. BFS 结束后，若 fresh == 0 则返回分钟数，否则返回 -1（有无法被感染的新鲜橘子）。

参考代码

```
from collections import deque

def orangesRotting(grid):
    m, n = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for i in range(m):
        for j in range(n):
            if grid[i][j] == 2:
                q.append((i, j))      # 所有腐烂橘子作为多源起点
            elif grid[i][j] == 1:
                fresh += 1            # 统计新鲜橘子数
    if fresh == 0:
        return 0
    minutes = 0
    dirs = [(1, 0), (-1, 0), (0, 1), (0, -1)]
    while q and fresh > 0:
        minutes += 1                  # 这一层代表一分钟
        for _ in range(len(q)):       # 只处理当前层的橘子
            x, y = q.popleft()
            for dx, dy in dirs:
                nx, ny = x + dx, y + dy
                if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == 1:
                    grid[nx][ny] = 2  # 感染
                    fresh -= 1
                    q.append((nx, ny))
    return minutes if fresh == 0 else -1
```

复杂度分析

时间复杂度 O(m×n)：每个格子入队出队最多一次。空间复杂度 O(m×n)：队列最坏情况。

核心知识点

- 多源 BFS：把所有起点一次性入队，再统一向外扩散，是处理「多个源同时扩散」问题的标准套路。
- BFS 层与时间对应：每处理完一层代表「一分钟」，所以 while 里要用 for _ in range(len(q)) 做层次划分。
- 把 rounds 计数放在循环开头：需要在每层开始时 +1，同时保证不会多算一轮。
- 先判 fresh == 0 的边界：没有新鲜橘子时答案应为 0 而不是 1。

- 区分 BFS 与 DFS 的适用场景：求「最短步数 / 最少时间」用 BFS，求「是否存在路径」两者皆可。

207. 课程表 【中等】

题目描述

你这个学期必须选修 numCourses 门课程，记为 0 到 numCourses - 1。在选修某些课程之前需要一些先修课程。先修课程按数组 prerequisites 给出，其中 prerequisites[i] = [ai, bi] 表示如果要学习课程 ai 则必须先学习课程 bi。请你判断是否可能完成所有课程的学习？如果可以，返回 true；否则，返回 false。

> **示例：**
>
> 输入：numCourses = 2, prerequisites = [[1,0]]
> 输出：true
> 解释：总共有 2 门课程。学习课程 1 之前需要完成课程 0，可以完成。

解题思路

这道题的本质是「判断有向图中是否存在环」：如果课程之间形成了循环依赖，就无法完成。这属于「拓扑排序」问题。

方法一（Kahn 算法 / BFS + 入度）：

1. 建立邻接表 graph：bi → [所有以 bi 为前置的课程]。

2. 计算每门课的入度 indeg：入度表示「还有几门先修课没上」。

3. 把所有入度为 0 的课程（没有先修要求的）放入队列。

4. 不断出队：把该课程「修掉」，记数加一；把它指向的课程入度减一，若某课程入度变为 0，说明它的先修课都修完了，入队。

5. 最后若「已修课程数 == numCourses」，说明无环，返回 True；否则返回 False。

方法二（DFS + 三色标记）：用 0/1/2 表示「未访问 / 正在访问 / 已完成」。DFS 过程中若遇到「正在访问」的结点，说明找到了环。

参考代码

```
from collections import deque, defaultdict

# 方法一：Kahn 算法 (BFS + 入度)
def canFinish(numCourses, prerequisites):
    graph = defaultdict(list)
    indeg = [0] * numCourses
    for a, b in prerequisites:          # b -> a
        graph[b].append(a)
        indeg[a] += 1
    q = deque([i for i in range(numCourses) if indeg[i] == 0])
    done = 0
    while q:
        cur = q.popleft()
        done += 1
        for nxt in graph[cur]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:         # 先修课已全部完成
                q.append(nxt)
    return done == numCourses

# 方法二：DFS + 三色标记
def canFinish2(numCourses, prerequisites):
    graph = defaultdict(list)
    for a, b in prerequisites:
        graph[b].append(a)
    state = [0] * numCourses            # 0 未访问，1 访问中，2 已完成
    def dfs(u):
        if state[u] == 1:
            return False                # 回到访问中的结点 -> 有环
        if state[u] == 2:
            return True
        state[u] = 1
        for v in graph[u]:
            if not dfs(v):
                return False
        state[u] = 2
        return True
    for i in range(numCourses):
        if not dfs(i):
            return False
    return True
```

复杂度分析

Kahn 算法：时间 O(V + E)，空间 O(V + E)。DFS 法：时间 O(V + E)，空间 O(V + E)。V 为课程数，E 为依赖关系数。

核心知识点

- 拓扑排序：把一个有向无环图（DAG）排成线性序列，使得所有边都从前指向后。
- Kahn 算法：基于入度的 BFS 拓扑排序，「入度为 0 即可处理」是核心判据。
- 判环的两种方式：Kahn 算法看「是否能处理完所有结点」；DFS 看「是否回到访问中的结点」。

- 三色标记法：0/1/2 分别表示未访问/访问中/已完成，用「访问中」状态检测环。
- 实际应用：编译依赖、任务调度、课程安排、包管理器依赖解析。
- 衍生题：课程表 II（210）——不是判断能否完成，而是输出一个合法顺序，直接把 Kahn 的出队顺序作为答案。

208. 实现 Trie（前缀树） 【中等】

题目描述

Trie（发音类似 "try"）或者说前缀树是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补完和拼写检查。

请你实现 Trie 类：Trie() 初始化前缀树对象；

void insert(String word) 向前缀树中插入字符串 word；

boolean search(String word) 如果字符串 word 在前缀树中，返回 true，否则返回 false；

boolean startsWith(String prefix) 如果之前已经插入的字符串 word 的前缀之一为 prefix，返回 true，否则返回 false。

示例：

```
输入：["Trie","insert","search","search","startsWith","insert","search"]
[[],["apple"],["apple"],["app"],["app"],["app"],["app"]]
输出：[null,null,true,false,true,null,true]
```

解题思路

Trie 的结构：每个结点代表一个字符，从根到某结点的路径拼成一个前缀。结点里用字典存「子字符 → 子结点」，并用一个布尔标记 isEnd 表示「是否有单词在这里结束」。

插入 insert：从根出发，逐字符往下走；不存在就新建结点；最后把末尾结点的 isEnd 置为 True。

查找 search：从根出发逐字符往下走，中途走不通就返回 False；走完后还要检查 isEnd（确保是完整单词）。

前缀查找 startsWith：与 search 几乎相同，但不需要检查 isEnd（只要有这条路径就是合法前缀）。

复杂度：每次操作只与字符串长度有关，与已存单词数量无关。这就是 Trie 相对哈希表的优势——哈希表无法高效回答「是否存在以某前缀开头的单词」。

参考代码

```
class Trie:
    def __init__(self):
        self.children = {}     # 子字符 -> 子结点
        self.is_end = False    # 是否有单词在此结束

    def insert(self, word):
        node = self
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Trie()
            node = node.children[ch]
        node.is_end = True

    def _find(self, s):        # 从根出发走完 s，返回终点结点或 None
        node = self
        for ch in s:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def search(self, word):
        node = self._find(word)
        return node is not None and node.is_end

    def startsWith(self, prefix):
        return self._find(prefix) is not None
```

复杂度分析

插入、查找、前缀查找的时间复杂度均为 O(L)，L 为字符串长度。空间复杂度 O(N×L)，N 为单词数。

核心知识点

- Trie（前缀树 / 字典树）：以「路径表示前缀」的树形结构，共享公共前缀以节省空间。
- 字典建树：用 dict 存子结点，比固定 26 长度数组更省空间，但失去 O(1) 数组索引；两者可按需选择。
- isEnd 标记的必要性：区分「完整单词」与「只是一个前缀」，这是 search 与 startsWith 的唯一差别。
- Trie 的典型应用：自动补全、拼写检查、IP 路由（最长前缀匹配）、词频统计。

- 衍生题：添加与搜索单词（211）——在 Trie 的查找中加入 '.' 通配符，需要 DFS 回溯。

第 10 章 回溯

本章核心思想

回溯算法（Backtracking）本质是「带撤销的暴力搜索」：沿着一条路往前走，走不通就退回上一步，换另一条路继续试。

通用模板（务必背下来）：

```
def backtrack(路径, 选择列表):
    if 满足结束条件:
        记录结果; return
    for 选择 in 选择列表:
        做选择          # 修改状态
        backtrack(路径, 新的选择列表)
        撤销选择        # 恢复状态（这一步就是「回溯」）
```

所有回溯题的区别只在于三个问题：

1）什么时候把「路径」加入结果集？（子集问题在进入函数时加、排列问题在叶子处加）

2）选择列表是什么？（由题目决定）

3）如何避免重复？（用 start 索引、用 used 数组、或排序后跳过同层重复元素）

46. 全排列 【中等】

题目描述

给定一个不含重复数字的数组 nums，返回其所有可能的全排列。你可以按任意顺序返回答案。

示例：

```
输入：nums = [1,2,3]
输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

解题思路

全排列的特点是「顺序重要」，所以不能用 start 索引来避免重复，而要用 used 数组标记「哪个元素已经被用过」。

具体步骤：

1. 建 used 数组（长度 n，初始全 False）和 path 列表（当前排列）。

2. 回溯函数：

- 若 path 长度 == n，说明找到一个完整排列，把 path 的副本加入结果，返回。

- 遍历所有下标 i：若 used[i] 为 True 就跳过；否则 used[i] = True、path.append(nums[i])，递归，然后撤销（path.pop()、used[i] = False）。

特别注意：加入结果时一定要用 path[:]（拷贝），否则后续回溯会把已经存好的列表改掉，最后结果全是空列表。这是初学者最容易踩的坑。

参考代码

```
def permute(nums):
    n = len(nums)
    res = []
    path = []
    used = [False] * n

    def backtrack():
        if len(path) == n:            # 找到一个完整排列
            res.append(path[:])       # 必须拷贝!
            return
        for i in range(n):
            if used[i]:
                continue              # 已使用过，跳过
            used[i] = True            # 做选择
            path.append(nums[i])
            backtrack()               # 递归
            path.pop()                # 撤销选择
            used[i] = False

    backtrack()
    return res
```

复杂度分析

时间复杂度 O(n × n!)：共 n! 个排列，每个排列拷贝需要 O(n)。空间复杂度 O(n)：递归栈与 path。

核心知识点

- 回溯三要素：路径（已做的选择）、选择列表（还能做什么）、结束条件。
- used 数组：排列类问题用它标记元素是否已使用，保证每个元素在每个位置只出现一次。
- path[:] 深拷贝：把「当前路径的快照」存入结果，否则会被后续回溯清除。
- 撤销选择（回溯）的对称性：做了什么选择，就要做对应的逆操作。
- 剪枝：本题无重复元素所以无需额外剪枝；若有重复元素（题 47），需要「排序 + 同层去重」。

78. 子集 【中等】

题目描述

给你一个整数数组 nums，数组中的元素互不相同。返回该数组所有可能的子集（幂集）。解集不能包含重复的子集。你可以按任意顺序返回解集。

示例：

```
输入：nums = [1,2,3]
输出：[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```

解题思路

子集问题的关键特点：**每个结点都是一个答案**（不像全排列只在叶子处收集），因为任何一个「部分选择」本身就构成一个合法子集。

为了避免重复子集，用 start 索引控制「只能选当前位置及之后的元素」，这样 [1,2] 和 [2,1] 就只会出现一次（顺序固定）。

具体步骤：

1. 回溯函数 backtrack(start)：

   - 一进入函数，就把当前 path 的副本加入结果（因为 path 的任何状态都是合法子集）。

   - 从 start 开始遍历 i：path.append(nums[i])，递归 backtrack(i + 1)，然后 path.pop()。

2. 从 backtrack(0) 开始。

另一种思路（选或不选）：对每个元素做二元决策——「选它」或「不选它」，形成一个二叉决策树，叶子就是所有子集。两种写法都值得掌握。

参考代码

```
# 写法一：start 索引法
def subsets(nums):
    res = []
    path = []

    def backtrack(start):
        res.append(path[:])          # 每个结点都是一个子集
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)         # 只能选后面的元素
            path.pop()

    backtrack(0)
    return res

# 写法二：每个元素「选 / 不选」的二叉决策
def subsets2(nums):
    res = []
    path = []

    def dfs(i):
        if i == len(nums):           # 决策完所有元素
            res.append(path[:])
            return
        dfs(i + 1)                   # 不选 nums[i]
        path.append(nums[i])
        dfs(i + 1)                   # 选 nums[i]
        path.pop()

    dfs(0)
    return res
```

复杂度分析

时间复杂度 $O(n \times 2^n)$：共 2^n 个子集，每个需要 $O(n)$ 拷贝。空间复杂度 $O(n)$：递归栈深度。

核心知识点

- 子集问题的收集时机：进入回溯函数时即可收集，因为任何中间状态都是合法子集。
- start 索引去重：强制「只往后选」，避免同一组元素产生不同顺序的重复答案。
- 「选/不选」的二叉决策树：换一个视角看子集问题，是理解回溯本质的好方法。
- 组合、子集、排列的对比：组合/子集用 start 索引（顺序无关），排列用 used 数组（顺序有关）。
- 衍生题：子集 II（90）有重复元素，需要排序 + 同层去重；组合总和（39/40）同理。

17. 电话号码的字母组合 【中等】

题目描述

给定一个仅包含数字 2-9 的字符串，返回所有它能表示的字母组合。答案可以按任意顺序返回。

数字到字母的映射如下（与电话按键相同）：2-abc, 3-def, 4-ghi, 5-jkl, 6-mno, 7-pqrs, 8-tuv, 9-wxyz。

> **示例：**
>
> 输入：digits = "23"
>
> 输出：["ad","ae","af","bd","be","bf","cd","ce","cf"]

解题思路

这是一个「多叉树」的回溯问题：第 i 层的选择就是「第 i 个数字对应的那几个字母」。

具体步骤：

1. 先建立数字到字母的映射表（用字典或列表）。

2. 回溯函数 backtrack(index)：index 表示当前处理到哪个数字。

   - 若 index == len(digits)，说明所有数字都处理完了，把 path 加入结果，返回。

   - 取出 digits[index] 对应的字母串 letters，遍历其中每个字母：

     path.append(ch)，递归 backtrack(index + 1)，path.pop()。

3. 从 backtrack(0) 开始。

边界情况：如果输入的 digits 是空字符串，应该返回空列表 []（注意不是 ['']）。

参考代码

```
def letterCombinations(digits):
    if not digits:
        return []
    phone = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz',
    }
    res = []
```

```
    path = []

    def backtrack(index):
        if index == len(digits):          # 所有数字处理完毕
            res.append(''.join(path))
            return
        for ch in phone[digits[index]]:   # 当前数字的每个候选字母
            path.append(ch)
            backtrack(index + 1)
            path.pop()

    backtrack(0)
    return res
```

复杂度分析

时间复杂度 $O(4^n \times n)$：n 为数字个数，最多 4^n 种组合，每个组合耗 O(n) 拼接。空间复杂度 O(n)：递归栈深度。

核心知识点

- 多叉树回溯：每层的分支数等于「当前数字对应字母的个数」，与全排列的「n 叉」不同。
- index 作为层标识：用「处理到第几个位置」代替 start 或 used，适合「每个位置有若干候选」的问题。
- 字符串拼接：path 用列表存字符，最后 ''.join(path) 拼接，比字符串相加更高效。
- 空输入要返回 []：这是本题最容易丢的边界。
- 映射表设计：字典比多个 if-else 更清晰，也可以用列表（下标即数字）。

39. 组合总和 【中等】

题目描述

给你一个无重复元素的整数数组 candidates 和一个目标整数 target，找出 candidates 中可以使数字和为目标数 target 的所有不同组合，并以列表形式返回。candidates 中的同一个数字可以无限制重复被选取。如果至少一个数字的被选数量不同，则两种组合是不同的。对于给定的输入，保证和为 target 的不同组合数少于 150 个。

示例：

输入：candidates = [2,3,6,7], target = 7

输出：[[2,2,3],[7]]

解题思路

与「子集」框架类似，但要处理两个新问题：

1）元素可以重复使用 → 递归时传入的 start 就是 i 本身（而不是 i+1），表示「还可以继续选自己」。

2）需要判断「当前和是否达到目标」→ 用参数 remain 记录还差多少。

具体步骤：

1. 回溯函数 backtrack(start, remain)：

- 若 remain == 0：把 path 加入结果，返回。

- 若 remain < 0：已经超过目标，直接返回（剪枝）。

- 遍历 i 从 start 到末尾：path.append(candidates[i])，递归 backtrack(i, remain - candidates[i])，path.pop()。

2. 从 backtrack(0, target) 开始。

进阶剪枝：先把 candidates 升序排序，那么当 candidates[i] > remain 时，后面的元素只会更大，也一定超过目标，可以直接 break（而不是只 continue）。这个剪枝能显著提速。

参考代码

```
def combinationSum(candidates, target):
    candidates.sort()              # 排序后可以提前 break 剪枝
    res = []
    path = []

    def backtrack(start, remain):
        if remain == 0:            # 正好凑齐
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if candidates[i] > remain:   # 后面的更大，直接剪枝
                break
            path.append(candidates[i])
            backtrack(i, remain - candidates[i])   # 传 i 表示可重复选自己
            path.pop()

    backtrack(0, target)
    return res
```

复杂度分析

时间复杂度 O(S)：S 为所有可行解的长度之和（与搜索树规模相关）。空间复杂度 O(target)：递归栈深度。

核心知识点

- 可重复选取的处理：递归时传 i 而不是 i + 1，表示「当前元素还能再选」。
- 用 remain 代替 sum(path)：避免每次重新求和，效率更高。
- 排序 + break 剪枝：利用有序性在「已经超标」时终止整个循环。
- 与「子集」的关系：元素不可重复选（题 40）时传 i+1，且需要额外跳过同层重复元素。
- 组合 vs 排列：组合不关心顺序（用 start），排列关心顺序（用 used）。

22. 括号生成 【中等】

题目描述

数字 n 代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且有效的括号组合。

> **示例：**
>
> 输入：n = 3
> 输出：["((()))","(()())","(())()","()(())","()()()"]

解题思路

把生成过程想象成「从左到右逐位填括号」，每填一个位置有「填左括号」或「填右括号」两种选择，但要满足两个约束条件：

1. 左括号数量不超过 n（不能多用左括号）。

2. 右括号数量不能超过左括号数量（否则会出现非法前缀，如 ")"、"())"）。

具体步骤：

1. 回溯函数 backtrack(left, right, path)：left/right 分别表示已用左/右括号数。

2. 若 path 长度 == 2n，说明生成完毕，加入结果。

3. 若 left < n：可以填左括号，递归 backtrack(left+1, right, path + '(')。

4. 若 right < left：可以填右括号，递归 backtrack(left, right+1, path + ')')。

这两个约束天然「剪掉了所有非法分支」，不需要生成后再验证，效率很高。（也可以推广到更一般的问题：任何「前缀约束」类问题都可以用这种「边生成边剪枝」的思路。）

参考代码

```
def generateParenthesis(n):
    res = []

    def backtrack(left, right, path):
        if len(path) == 2 * n:        # 生成完成
            res.append(path)
            return
        if left < n:                  # 还能放左括号
            backtrack(left + 1, right, path + '(')
        if right < left:              # 右括号数量不能超过左括号
            backtrack(left, right + 1, path + ')')

    backtrack(0, 0, '')
    return res
```

复杂度分析

时间复杂度 $O(4^n / \sqrt{n})$：第 n 个卡特兰数（合法括号组合数）。空间复杂度 O(n)：递归栈深度。

核心知识点

- 边生成边剪枝：用约束条件提前排除非法分支，避免「先穷举再筛选」的低效做法。
- 前缀约束：合法性条件 right < left 是「任意前缀中左括号不少于右括号」的直接表达。
- 卡特兰数：合法括号组合的数量是卡特兰数 C(2n, n)/(n+1)，这类「配对 / 树形结构」问题常与之相关。
- path + '(' 的写法：利用字符串不可变性创建新对象，相当于自动完成「撤销」操作。
- 同类约束题：括号生成、有效的括号字符串、解码方法等，都可用类似的「状态约束」思路。

79. 单词搜索 【中等】

题目描述

给定一个 m x n 二维字符网格 board 和一个字符串单词 word。如果 word 存在于网格中，返回 true；否则，返回 false。单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中「相邻」单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

示例：

```
输入：board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"
输出：true
```

解题思路

这是「网格 DFS + 回溯」的经典组合：从每个格子出发尝试匹配 word，走不通就退回换方向。

具体步骤：

1. 遍历网格的每个格子作为起点，调用 dfs(i, j, k)，其中 k 表示「正在匹配 word 的第 k 个字符」。

2. dfs 的逻辑：

   - 若越界，返回 False。

   - 若 board[i][j] != word[k]，返回 False。

   - 若 k == len(word) - 1，说明全部匹配成功，返回 True。

   - 把 board[i][j] 临时改成特殊字符（比如 '#'）表示「本路径已使用」，防止回头重复访问。

   - 向四个方向递归 dfs，任一方向返回 True 就返回 True。

   - 恢复 board[i][j] 的原值（回溯），返回 False。

「临时修改 + 恢复现场」是网格回溯的核心技巧，用原地修改代替 visited 数组。

参考代码

```
def exist(board, word):
    m, n = len(board), len(board[0])

    def dfs(i, j, k):
        if i < 0 or i >= m or j < 0 or j >= n:
            return False
        if board[i][j] != word[k]:
            return False
        if k == len(word) - 1:            # 最后一个字符也匹配成功
            return True
        tmp = board[i][j]
        board[i][j] = '#'                 # 标记已使用，防止走回头路
        found = (dfs(i + 1, j, k + 1) or dfs(i - 1, j, k + 1)
                 or dfs(i, j + 1, k + 1) or dfs(i, j - 1, k + 1))
        board[i][j] = tmp                 # 回溯：恢复现场
        return found

    for i in range(m):
        for j in range(n):
            if dfs(i, j, 0):
                return True
    return False
```

复杂度分析

时间复杂度 O(m × n × 3^L)：L 为单词长度，每个格子作为起点有约 3^L 种搜索路径（不含回头）。

空间复杂度 O(L)：递归栈深度。

核心知识点

- 网格回溯：DFS + 状态标记 + 恢复现场，是「在网格中搜索路径」的通用范式。
- 原地标记法：用特殊字符替换当前字符表示「已访问」，比 visited 数组更省空间。
- 剪枝：先判断字符是否匹配再递归，可以大幅减少无效搜索。
- 为什么分支数是 3 而不是 4：因为不能往回走（上一个格子已被标记），所以每次最多 3 个方向。
- 同类题：岛屿数量（200）是「求连通块」，本题是「求特定路径」，都用网格 DFS。

131. 分割回文串 【中等】

题目描述

给你一个字符串 s，请你将 s 分割成一些子串，使每个子串都是回文串。返回 s 所有可能的分割方案。

示例：

```
输入：s = "aab"
输出：[["a","a","b"],["aa","b"]]
```

解题思路

把「分割」看成「在字符串上依次切出若干段」，这天然是一个回溯问题：每层的选择是「切多长」。

具体步骤：

1. 回溯函数 backtrack(start)：start 是「当前还没被分割部分的起始下标」。

2. 若 start == len(s)：说明整个字符串都切完了，把 path 加入结果，返回。

3. 遍历 end 从 start 到 len(s)-1：截取子串 s[start:end+1]。

   - 若它是回文串，就 path.append(该子串)，递归 backtrack(end + 1)，然后 path.pop()。

   - 若不是回文，跳过（continue）。

4. 从 backtrack(0) 开始。

优化点：如果每次都现场判断回文，会有很多重复计算。可以先用「动态规划」预处理出一个二维数组 isPal[i][j]，表示「s[i..j] 是否为回文」，这样判断就变成 O(1)。这道题也是「回溯 + DP 预处理」结合的典型。

参考代码

```
def partition(s):
    n = len(s)
    # 1. DP 预处理: isPal[i][j] 表示 s[i..j] 是否为回文
    isPal = [[False] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            if s[i] == s[j] and (j - i <= 2 or isPal[i + 1][j - 1]):
                isPal[i][j] = True
    res = []
    path = []

    def backtrack(start):
        if start == n:                  # 分割完成
            res.append(path[:])
            return
        for end in range(start, n):
            if not isPal[start][end]:   # 不是回文就跳过
                continue
```

```
            path.append(s[start:end + 1])
            backtrack(end + 1)          # 从下一段开始
            path.pop()

    backtrack(0)
    return res
```

复杂度分析

时间复杂度 $O(n \times 2^n)$：最坏情况有 2^{n-1} 种分割方式，DP 预处理 $O(n^2)$。空间复杂度 $O(n^2)$：isPal 数组。

核心知识点

- 「切割问题」的回溯建模：用 start 表示剩余部分的起点，用 end 枚举切点，是字符串分割类问题的通用形式。
- 回文判断的 DP 预处理：isPal[i][j] = (s[i]==s[j]) and (j-i<=2 or isPal[i+1][j-1])，是回文子串判定的标准递推式。
- DP 预处理 + 回溯：用空间换时间，避免在搜索过程中重复判断回文。
- 与「子集」的相似性：都是「枚举切分点」的框架，只是本题多了一个「回文」合法性约束。
- 衍生题：分割回文串 II（132）求最少分割次数，是纯 DP 问题。

51. N 皇后 【困难】

题目描述

按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。n 皇后问题研究的是如何将 n 个皇后放置在 n×n 的棋盘上，并且使皇后彼此之间不能相互攻击。给你一个整数 n，返回所有不同的 n 皇后问题的解决方案。每一种解法包含一个不同的 n 皇后问题的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例：

输入：n = 4

输出：[[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]

解题思路

这是回溯法最经典的难题。关键在于「如何高效判断某个位置能否放皇后」。

核心洞察：**按行放置**。因为每行只能放一个皇后，所以第 r 行的皇后放在哪一列，就是一个「逐行的选择问题」，天然形成回溯的层结构（行号 = 递归深度）。

冲突判断只需要三个集合：

1. 列冲突：cols 集合记录已占用的列。

2. 主对角线（左上-右下，'\' 方向）：同一条对角线上满足 (row - col) 相同。

3. 副对角线（右上-左下，'/' 方向）：同一条对角线上满足 (row + col) 相同。

具体步骤：

1. backtrack(row)：表示「正在决定第 row 行皇后放哪一列」。

2. 若 row == n，说明 n 行都放好了，把棋盘转成字符串列表加入结果。

3. 遍历 col 从 0 到 n-1：若列 col、对角 row-col、反对角 row+col 都不冲突：

   - 在棋盘的 (row, col) 放 'Q'，三个集合都加入对应标记。

   - 递归 backtrack(row + 1)。

   - 撤销：棋盘恢复 '.'，三个集合移除标记。

用集合做 O(1) 的冲突检测，比每次扫描棋盘（O(n)）快得多，这是本题的关键优化。

参考代码

```
def solveNQueens(n):
    res = []
    board = [['.'] * n for _ in range(n)]
    cols = set()        # 已占用的列
    diag1 = set()       # 主对角线 (row - col)
    diag2 = set()       # 副对角线 (row + col)

    def backtrack(row):
        if row == n:                    # 所有行都放好了
            res.append([''.join(r) for r in board])
            return
        for col in range(n):
```

```
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue                # 冲突，换下一列
            # 放置皇后
            board[row][col] = 'Q'
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)

            backtrack(row + 1)

            # 撤销
            board[row][col] = '.'
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0)
    return res
```

复杂度分析

时间复杂度 $O(n!)$：每行尝试 n 列，实际因剪枝远小于 $n!$。空间复杂度 $O(n^2)$：棋盘 + 三个集合。

核心知识点

- 「按行决策」的建模：把 $n \times n$ 的二维选择问题降成一维的逐行选择，是本题最重要的简化。
- 对角线的数学表示：主对角线 row-col 相同，副对角线 row+col 相同，这是棋盘题的必备技巧。
- 用集合做 $O(1)$ 冲突检测：把 $O(n)$ 的扫描降为 $O(1)$，是性能优化的关键。
- 回溯的对称性：放置时添加标记，撤销时移除标记，一一对应。
- 剪枝的重要性：最优解在搜索树中可以大量剪枝，使实际复杂度远低于理论上界。
- 衍生题：N 皇后 II（52）只需返回方案数量，可以把棋盘换成计数变量，空间降到 $O(n)$。

第 11 章 二分查找

本章核心思想

二分查找的前提是「有序」或「具有二段性质」（即可以找到一个分界点，使得一侧全满足条件、另一侧全不满足）。

最标准、最不容易写错的模板（左闭右闭区间）：

```
left, right = 0, n - 1
while left <= right:
    mid = (left + right) // 2
    if 满足条件: return mid
    elif 目标在右侧: left = mid + 1
    else: right = mid - 1
```

写二分的三个要点：

1. 明确「区间含义」（左闭右闭还是左闭右开），并保持一致。
2. 循环条件是 left <= right 还是 left < right，取决于区间定义。
3. mid 用 (left + right) // 2 可能溢出（在 C++/Java 中），Python 无此问题，但可用 left + (right-left)//2 更规范。

35. 搜索插入位置 【简单】

题目描述

给定一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置。请必须使用时间复杂度为 O(log n) 的算法。

示例：

```
输入：nums = [1,3,5,6], target = 5
输出：2
```

输入：nums = [1,3,5,6], target = 2

输出：1

解题思路

标准的二分查找，只是「找不到时返回插入位置」。

关键观察：循环结束时，left 正好等于「第一个大于 target 的位置」，也就是该插入的下标。

具体步骤（左闭右闭区间）：

1. left = 0，right = len(nums) - 1。

2. while left <= right：mid = (left + right) // 2。

   - 若 nums[mid] == target，直接返回 mid。

   - 若 nums[mid] < target，left = mid + 1。

   - 否则 right = mid - 1。

3. 循环结束返回 left（此时 left 是插入位置）。

为什么返回 left？因为跳出循环时 left = right + 1，而 left 左侧的所有元素都 < target，left 及其右侧的元素都 > target，所以 left 就是正确的插入位置。

参考代码

```
def searchInsert(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1      # 目标在右半部分
        else:
            right = mid - 1     # 目标在左半部分
    return left                 # 找不到时 left 就是插入位置
```

复杂度分析

时间复杂度 O(log n)。空间复杂度 O(1)。

核心知识点

- 二分查找标准模板：左闭右闭区间 + while left <= right。
- 返回 left 的含义：循环结束时 left 是「第一个大于 target 的下标」，即插入位置。
- 边界理解：target 比所有元素小 → left = 0；比所有元素大 → left = n。两种情况都正确。
- Python 不存在整数溢出问题，但写 left + (right - left) // 2 是好习惯，也是面试加分点。

74. 搜索二维矩阵 【中等】

题目描述

给你一个满足下述两条属性的 m x n 整数矩阵：每行中的整数从左到右按非严格递增顺序排列；每行的第一个整数大于前一行的最后一个整数。给你一个整数 target，如果 target 在矩阵中，返回 true；否则，返回 false。

> **示例：**
>
> 输入：matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
> 输出：true

解题思路

注意题目的第二个条件：「每行的第一个整数大于前一行的最后一个整数」。这意味着整个矩阵「按行展开后就是一个全局有序的一维数组」！

于是可以直接对整个矩阵做二分查找，只需把一维下标映射到二维坐标即可。

具体步骤：

1. 总元素个数 total = m × n，在 [0, total-1] 范围内二分。

2. mid 对应的二维坐标是 (mid // n, mid % n)。

3. 比较 matrix[mid // n][mid % n] 与 target，按标准二分缩小范围。

另一种写法（两次二分）：先对「第一列」二分，找出 target 可能所在的那一行；再在该行内二分查找。同样是 O(log(mn))。

注意与「搜索二维矩阵 II」（题 240）区分：那道题的行列有序但不满足「跨行不等式」，不能用整体二分，只能用右上角线性查找。

参考代码

```
def searchMatrix(matrix, target):
    if not matrix or not matrix[0]:
        return False
    m, n = len(matrix), len(matrix[0])
    left, right = 0, m * n - 1
    while left <= right:
        mid = (left + right) // 2
        val = matrix[mid // n][mid % n]   # 一维下标 -> 二维坐标
        if val == target:
            return True
        elif val < target:
            left = mid + 1
        else:
            right = mid - 1
    return False
```

复杂度分析

时间复杂度 O(log(m×n))。空间复杂度 O(1)。

核心知识点

- 虚拟展平：当矩阵满足「跨行有序」时，可以把它当作一维有序数组来二分。
- 下标映射公式：一维下标 k ↔ 二维坐标 (k // n, k % n)，这是矩阵与一维数组互转的标准方法。
- 读题的关键性：「每行第一个数大于上一行最后一个数」这个条件决定了能否整体二分。
- 与题 240 的对比：题目条件不同，解法就完全不同，要仔细分辨矩阵的性质。

34. 在排序数组中查找元素的第一个和最后一个位置 【中等】

题目描述

给你一个按照非递减顺序排列的整数数组 nums，和一个目标值 target。请你找出给定目标值在数组中的开始位置和结束位置。如果数组中不存在目标值 target，返回 [-1, -1]。你必须设计并实现时间复杂度为 O(log n) 的算法解决此问题。

示例：

输入：nums = [5,7,7,8,8,10], target = 8

输出：[3,4]

解题思路

要 O(log n)，就要用两次二分：一次找「左边界」，一次找「右边界」。

找左边界的二分（关键在「等于时不停，继续向左收缩」）：

1. left = 0，right = n-1，记录 ans = -1。

2. 每次算出 mid：若 nums[mid] == target，先记录 ans = mid，然后 right = mid - 1（继续往左找）。

3. 若 nums[mid] < target，left = mid + 1；否则 right = mid - 1。

4. 循环结束后 ans 就是左边界。

找右边界完全对称：等于时记录 ans 并令 left = mid + 1（继续往右找）。

另一种更简洁的写法：用「查找第一个 >= target 的位置」作为通用函数，则左边界 = lower_bound(target)，右边界 = lower_bound(target + 1) - 1。

参考代码

```
def searchRange(nums, target):
    def find_left():
        lo, hi, ans = 0, len(nums) - 1, -1
        while lo <= hi:
            mid = (lo + hi) // 2
            if nums[mid] == target:
                ans = mid          # 记录后继续向左收缩
                hi = mid - 1
            elif nums[mid] < target:
                lo = mid + 1
            else:
                hi = mid - 1
        return ans

    def find_right():
        lo, hi, ans = 0, len(nums) - 1, -1
        while lo <= hi:
            mid = (lo + hi) // 2
            if nums[mid] == target:
                ans = mid          # 记录后继续向右收缩
                lo = mid + 1
            elif nums[mid] < target:
```

```
                lo = mid + 1
            else:
                hi = mid - 1
        return ans

    return [find_left(), find_right()]
```

复杂度分析

时间复杂度 O(log n)：两次二分。空间复杂度 O(1)。

核心知识点

- 二分查找的变体：查找「最左 / 最右位置」时，命中后不要立即返回，而是继续收缩边界。
- 左右边界的对称性：找左边界时命中后收右界，找右边界时命中后收左界。
- lower_bound 概念：第一个 >= target 的下标，是二分查找族里最通用的原语。
- 必须 O(log n)：题目明确要求时不能用线性扫描，否则虽然答案对但不符合要求。
- 应用场景：在有序数据中定位某个值的范围、统计出现次数。

33. 搜索旋转排序数组 【中等】

题目描述

整数数组 nums 按升序排列，数组中的值互不相同。数组在预先未知的某个下标 k 上进行了旋转，例如 [0,1,2,4,5,6,7] 在下标 3 处旋转后变成 [4,5,6,7,0,1,2]。给你旋转后的数组 nums 和一个整数 target，如果 nums 中存在这个目标值 target，则返回它的下标，否则返回 -1。你必须设计一个时间复杂度为 O(log n) 的算法解决此问题。

> **示例：**
>
> 输入：nums = [4,5,6,7,0,1,2], target = 0
> 输出：4

解题思路

旋转后的数组虽然整体无序，但有个重要性质：**取 mid 后，左半段和右半段中必有一段是有序的**。

具体步骤：

1. 标准二分，算出 mid。

2. 判断哪一半有序：

- 若 nums[left] <= nums[mid]，说明「左半段 [left, mid] 有序」。

此时若 target 落在 [nums[left], nums[mid]) 区间内，就在左半段找（right = mid - 1）；否则去右半段（left = mid + 1）。

- 否则「右半段 [mid, right] 有序」。

此时若 target 落在 (nums[mid], nums[right]] 区间内，就在右半段找（left = mid + 1）；否则去左半段（right = mid - 1）。

3. 找到 target 就返回下标，否则循环结束返回 -1。

核心思想：**用「有序的那一半」来判断 target 在哪一侧**。这是处理「局部有序」数组的通用方法。

参考代码

```
def search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:          # 左半段有序
            if nums[left] <= target < nums[mid]:
                right = mid - 1              # target 在左半段
            else:
                left = mid + 1               # target 在右半段
        else:                                # 右半段有序
            if nums[mid] < target <= nums[right]:
                left = mid + 1               # target 在右半段
            else:
                right = mid - 1              # target 在左半段
    return -1
```

复杂度分析

时间复杂度 O(log n)。空间复杂度 O(1)。

核心知识点

- 「半边有序」性质：旋转数组中，以 mid 分割后必有一半是完全有序的，这是解题的基石。
- 先判断哪半有序，再用区间包含关系决定搜索方向：顺序不能反。
- 区间开闭的细节：nums[left] <= target < nums[mid] 中的 < 是严格的，因为 nums[mid] 已经在前面判过了。
- 边界判断用 nums[left] <= nums[mid]：等号是为了处理「只剩两个元素」的情况。
- 衍生题：搜索旋转排序数组 II（81）允许重复元素，此时 nums[left] == nums[mid] 无法判断，需要先 left++ 缩小区间。

153. 寻找旋转排序数组中的最小值 【中等】

题目描述

已知一个长度为 n 的数组，预先按照升序排列，经由 1 到 n 次旋转后，得到输入数组。例如原数组 nums = [0,1,2,4,5,6,7] 在变化后可能得到 [4,5,6,7,0,1,2]。给你一个元素值互不相同的数组 nums，它原来是一个升序排列的数组，并按上述情形进行了多次旋转。请你找出并返回数组中的最小元素。你必须设计一个时间复杂度为 O(log n) 的算法解决此问题。

示例：

```
输入：nums = [3,4,5,1,2]
输出：1
```

解题思路

二分查找的又一变体。关键是比较 nums[mid] 与 nums[right]（注意是右端点，不是左端点）。

推理：

- 若 nums[mid] < nums[right]：说明从 mid 到 right 是递增的，最小值一定在 [left, mid] 这一侧（含 mid），所以 right = mid。

- 若 nums[mid] > nums[right]：说明最小值一定在 mid 的右侧（因为右半段发生了「跌落」），所以 left = mid + 1。

具体步骤：

1. left = 0，right = n - 1。

2. while left < right（注意不是 <=，因为我们要收敛到一个位置）：

   - mid = (left + right) // 2。

   - 若 nums[mid] < nums[right]，right = mid。

   - 否则 left = mid + 1。

3. 循环结束时 left == right，返回 nums[left]。

参考代码

```
def findMin(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[right]:
            right = mid           # 最小值在左半段(含 mid)
        else:
            left = mid + 1        # 最小值在 mid 右侧
    return nums[left]
```

复杂度分析

时间复杂度 O(log n)。空间复杂度 O(1)。

核心知识点

- 比较对象的选择：应该比较 nums[mid] 与 nums[right]，而不是与 nums[left]（后者无法判断）。
- right = mid 而不是 mid - 1：因为 mid 本身可能是最小值，不能排除。
- 循环条件 left < right：目标是「收敛到唯一位置」，所以用 < 而不是 <=。
- 二分的三种终止形式：找到即返回、收敛到 left、收敛到 left==right，要根据题意选择。
- 衍生题：寻找旋转排序数组中的最小值 II（154）允许重复元素，需要额外处理 nums[mid] == nums[right] 时 right-- 的情况。

4. 寻找两个正序数组的中位数 【困难】

题目描述

给定两个大小分别为 m 和 n 的正序（从小到大）数组 nums1 和 nums2，请你找出并返回这两个正序数组的中位数。算法的时间复杂度应该为 O(log (m+n))。

> **示例：**
>
> ```
> 输入：nums1 = [1,3], nums2 = [2]
> 输出：2.0
> 解释：合并数组 = [1,2,3]，中位数是 2。
> ```

解题思路

方法一（归并，O(m+n)，简单但不符合要求）：合并两个有序数组，取中间元素。适合先写出正确解再优化。

方法二（二分划分，O(log min(m,n))，面试目标解法）：

核心思路：把两个数组各切一刀，分成「左半部分」和「右半部分」，使得：

左半部分的总元素个数 = 右半部分的总元素个数（或差 1），且左半部分的最大值 <= 右半部分的最小值。

只要满足这个条件，中位数就能从左边的最大值和右边的最小值直接算出来。

具体步骤：

1. 为了保证二分的范围更小，始终对「较短的数组」做二分（必要时交换两数组）。

2. 设短数组长度为 m，在 [0, m] 上二分划分点 i，则 j = (m + n + 1) // 2 - i（由「左半总数」推出）。

3. 检查四个边界值：maxLeft1、minRight1、maxLeft2、minRight2。

- 若 maxLeft1 <= minRight2 且 maxLeft2 <= minRight1，说明找到正确的划分。

- 若 maxLeft1 > minRight2，说明 i 太大，向左收缩（right = i - 1）。

- 否则 i 太小，向右扩大（left = i + 1）。

4. 计算中位数：若总数为奇数，就是 max(左半最大值)；偶数则为两者的平均。

边界用 ±∞ 填充，避免繁琐的空数组判断。

参考代码

```
def findMedianSortedArrays(nums1, nums2):
    # 保证 nums1 是较短的那个，二分范围更小
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1
    m, n = len(nums1), len(nums2)
    total_left = (m + n + 1) // 2      # 左半部分应有的元素个数
    left, right = 0, m
    while left <= right:
        i = (left + right) // 2        # nums1 的划分点
        j = total_left - i             # nums2 的划分点
        # 四个边界值，用正负无穷处理空侧
        L1 = nums1[i - 1] if i > 0 else float('-inf')
        R1 = nums1[i] if i < m else float('inf')
        L2 = nums2[j - 1] if j > 0 else float('-inf')
        R2 = nums2[j] if j < n else float('inf')
        if L1 <= R2 and L2 <= R1:      # 找到正确划分
            if (m + n) % 2 == 1:
                return float(max(L1, L2))
            return (max(L1, L2) + min(R1, R2)) / 2
        elif L1 > R2:                  # i 太大，左移
            right = i - 1
        else:                          # i 太小，右移
            left = i + 1
    return 0.0

# 简单版本：归并 (O(m+n)，面试可先写这个)
def findMedianSortedArrays_simple(nums1, nums2):
    merged = sorted(nums1 + nums2)
    n = len(merged)
    if n % 2 == 1:
        return float(merged[n // 2])
    return (merged[n // 2 - 1] + merged[n // 2]) / 2
```

复杂度分析

二分解法：时间复杂度 O(log min(m, n))，空间复杂度 O(1)。归并解法：时间 O((m+n) log(m+n))，空间 O(m+n)。

核心知识点

- 「划分」的思路：把「找中位数」转化为「把两个数组切成等长的两半」，是最难也最优雅的思路。
- 为什么二分短的数组：j 由 i 推出，若 i 的范围超过短数组长度，j 会变成负数，所以必须让 m <= n。
- 哨兵值 ±∞：用于处理划分点落在数组端点的情况，避免大量边界判断。

- 奇偶统一公式：(m+n+1)//2 使左半部分在奇数时多一个元素，从而可以统一处理奇偶。
- 二分不只是「找值」，也可以是「找划分位置」——这是二分思想的重要拓展。

第 12 章 栈

本章核心思想

栈（Stack）是「后进先出」（LIFO）的线性结构，Python 里直接用 list 实现（append / pop 均为 O(1)）。

栈的三大典型应用场景：

1）括号匹配、表达式求值：遇到「左」压栈，遇到「右」弹栈配对。

2）单调栈：维护一个单调递增/递减的栈，用于求「下一个更大/更小元素」。

3）辅助栈：用一个额外的栈保存「当前状态下的最值 / 额外信息」，如最小栈。

判断某题是否该用栈：如果问题的结构是「嵌套的」「需要与最近的未匹配元素配对」，就该考虑栈。

20. 有效的括号 【简单】

题目描述

给定一个只包括 '('，')'，'{'，'}'，'['，']' 的字符串 s，判断字符串是否有效。有效字符串需满足：左括号必须用相同类型的右括号闭合；左括号必须以正确的顺序闭合；每个右括号都有一个对应的相同类型的左括号。

示例：

```
输入：s = "()[]{}"
输出：true
输入：s = "([)]"
输出：false
```

解题思路

括号匹配是栈最经典的应用。核心思想：「最近遇到的左括号，应该最先被闭合」，这正是后进先出。

具体步骤：

1. 建一个空栈，以及一个「右括号 → 左括号」的映射表。

2. 遍历字符串的每个字符 ch：

- 若 ch 是左括号，压栈。

- 若 ch 是右括号：若栈为空，说明没有对应的左括号，返回 False；否则弹出栈顶，检查是否与 ch 匹配，不匹配则返回 False。

3. 遍历结束后，若栈为空说明全部正确配对，返回 True；否则返回 False。

三个易错点：① 遇到右括号时栈可能已经空了；② 弹出的左括号类型必须与右括号匹配；③ 最后必须检查栈是否为空（例如 "((" 这种只有左括号的情况）。

参考代码

```
def isValid(s):
    pairs = {')': '(', ']': '[', '}': '{'}   # 右括号 -> 左括号
    stack = []
    for ch in s:
        if ch in '([{':
            stack.append(ch)                 # 左括号入栈
        else:
            if not stack:                    # 右括号但栈空 -> 不合法
                return False
            if stack.pop() != pairs[ch]:     # 类型不匹配 -> 不合法
                return False
    return not stack                         # 栈空才说明全部配对
```

复杂度分析

时间复杂度 O(n)：每个字符进栈出栈各一次。空间复杂度 O(n)：栈最坏情况（全是左括号）。

核心知识点

- 栈的「最近匹配」特性：后进先出正好对应「最近遇到的左括号最先闭合」。
- 用字典做映射表：把「配对判断」变成一次字典查询，代码更简洁。
- 两个必须检查的地方：弹栈前判空、遍历后判栈空。
- 同类题：删除字符串中的所有相邻重复项（1047）、简化路径（71）都是栈的典型应用。

155. 最小栈 【中等】

题目描述

设计一个支持 push，pop，top 操作，并能在常数时间内检索到最小元素的栈。实现 MinStack 类：MinStack() 初始化堆栈对象；void push(int val) 将元素 val 推入堆栈；void pop() 删除堆栈顶部的元素；int top() 获取堆栈顶部的元素；int getMin() 获取堆栈中的最小元素。

示例：

```
输入：["MinStack","push","push","push","getMin","pop","top","getMin"] [[],[-2],[0],[-3],[],[],[],[]]
输出：[null,null,null,null,-3,null,0,-2]
```

解题思路

难点在于：普通的栈只能看到栈顶，无法 O(1) 得到最小值。而且 pop 之后，最小值可能需要「恢复到之前的状态」。

解法一（辅助栈，推荐）：用一个额外的栈 min_stack 同步记录「每一步的最小值」。

- push(x)：主栈压入 x；min_stack 压入 min(x, min_stack 栈顶)。这样 min_stack 的栈顶永远是「当前整个栈的最小值」。

- pop()：两个栈同时弹出。

- getMin()：返回 min_stack 栈顶。

解法二（存差值，空间更省）：每个元素存「与当时最小值的差」，用一个变量记录最小值。原理巧妙但代码更绕，属于进阶技巧。

推荐掌握解法一，因为「辅助栈同步记录状态」是通用的解题模式。

参考代码

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []       # 辅助栈：记录每一步的最小值

    def push(self, val):
        self.stack.append(val)
        if self.min_stack:
```

```
            self.min_stack.append(min(val, self.min_stack[-1]))
        else:
            self.min_stack.append(val)

    def pop(self):
        self.stack.pop()
        self.min_stack.pop()      # 两栈同步弹出

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1] # 辅助栈栈顶即当前最小值
```

复杂度分析

所有操作均为 O(1)。空间复杂度 O(n)：需要额外的辅助栈。

核心知识点

- 辅助栈思想：用一个额外的栈保存「每个状态下的附加信息」，是设计类题目的常用模式。
- 同步压弹：两个栈必须同步操作，才能保证 min_stack 栈顶始终对应当前最小值。
- min(val, min_stack[-1]) 的含义：最小值要么是新的 val，要么是之前的最小值。
- 进阶解法（差值法）：空间 O(1)（不加辅助栈），但需要处理整数溢出，理解即可。
- Python 中栈直接用 list，self.stack[-1] 取栈顶，append/pop 都是 O(1)。

394. 字符串解码 【中等】

题目描述

给定一个经过编码的字符串，返回它解码后的字符串。编码规则为: k[encoded_string]，表示其中方括号内部的 encoded_string 正好重复 k 次。注意 k 保证为正整数。你可以认为输入字符串总是有效的；输入字符串中没有额外的空格，且输入的方括号总是符合格式要求的。此外，你可以认为原始数据不包含数字，所有的数字只表示重复的次数 k，例如不会出现像 3a 或 2[4] 的输入。

示例：

```
输入：s = "3[a]2[bc]"
输出："aaabcbc"
```

输入：s = "3[a2[c]]"
输出："accaccacc"

解题思路

这道题的嵌套结构（如 3[a2[c]]）天然适合用栈来处理。

核心思路：遇到「数字、左括号」就保存现场（压栈），遇到「右括号」就弹出栈顶、把括号内的字符串重复拼接。

具体步骤：

1. 维护两个栈：num_stack（保存待用的重复次数）和 str_stack（保存「外层已经积累的字符串」）。

2. 维护两个变量：cur_num（当前正在读的数字）和 cur_str（当前正在积累的字符串）。

3. 遍历每个字符 ch：

- 若是数字：cur_num = cur_num * 10 + int(ch)（处理多位数字）。

- 若是 '['：把 cur_num 压入 num_stack、cur_str 压入 str_stack，然后重置 cur_num = 0、cur_str = ''。

- 若是 ']'：弹出重复次数 k 和之前的字符串 prev，令 cur_str = prev + cur_str * k。

- 若是字母：cur_str += ch。

4. 返回 cur_str。

关键点：「遇到左括号时把外层的字符串暂存起来」，因为内层的解码要先完成。

参考代码

```
def decodeString(s):
    num_stack = []
    str_stack = []
    cur_num = 0
    cur_str = ''
    for ch in s:
        if ch.isdigit():
            cur_num = cur_num * 10 + int(ch)   # 支持多位数字
        elif ch == '[':
            num_stack.append(cur_num)          # 保存现场
            str_stack.append(cur_str)
```

```
            cur_num = 0
            cur_str = ''
        elif ch == ']':
            k = num_stack.pop()
            prev = str_stack.pop()
            cur_str = prev + cur_str * k       # 解码并拼接外层
        else:
            cur_str += ch
    return cur_str
```

复杂度分析

时间复杂度 O(n + 输出长度)：每个字符处理一次，字符串重复拼接的开销与输出长度成正比。空间复杂度 O(深度)：栈的深度取决于嵌套层数。

核心知识点

- 栈处理嵌套结构：遇到「开始」压栈保存现场，遇到「结束」弹栈恢复现场。
- 双栈设计：一个存数字、一个存字符串，分别对应「重复次数」和「外层上下文」。
- 多位数字的读取：cur_num = cur_num * 10 + int(ch)，这是逐字符解析数字的标准写法。
- 同类型题：基本计算器（224）、逆波兰表达式求值（150）、括号匹配（20），都是栈解析表达式的家族。
- 递归解法（拓展）：遇到 '[' 递归求值、遇到 ']' 返回，本质与栈解完全相同。

739. 每日温度 【中等】

题目描述

给定一个整数数组 temperatures，表示每天的温度，返回一个数组 answer，其中 answer[i] 是指对于第 i 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 0 来代替。

示例：

```
输入：temperatures = [73,74,75,71,69,72,76,73]
输出：[1,1,4,2,1,1,0,0]
```

解题思路

这是「单调栈」的模板题：求「下一个更大元素」的距离。

核心思想：维护一个「栈内存放的下标，其对应温度单调递减」的栈。新来的温度如果比栈顶温度高，就说明栈顶那天终于等到了更高的温度，可以出栈并结算答案。

具体步骤：

1. 建栈，存的是下标；结果数组 ans 全为 0。

2. 遍历每个下标 i：

- 当栈非空且 temperatures[i] > temperatures[栈顶] 时：弹出栈顶下标 j，令 ans[j] = i - j。

- 循环执行上一步，直到栈空或栈顶温度 >= 当前温度。

- 把 i 压栈。

3. 返回 ans。

为什么是 O(n)？每个下标最多入栈一次、出栈一次，总操作次数 ≤ 2n。

「单调栈」的适用信号：题目出现「下一个更大 / 更小的元素」「等待多少天」「左边/右边第一个比它高的」。

参考代码

```
def dailyTemperatures(temperatures):
    n = len(temperatures)
    ans = [0] * n
    stack = []          # 存下标，对应温度单调递减
    for i, t in enumerate(temperatures):
        # 当前温度比栈顶高 -> 栈顶那天找到了答案
        while stack and t > temperatures[stack[-1]]:
            j = stack.pop()
            ans[j] = i - j
        stack.append(i)
    return ans
```

复杂度分析

时间复杂度 O(n)：每个下标最多入栈出栈一次。空间复杂度 O(n)：栈最坏情况（温度递减）。

核心知识点

- 单调栈：栈内元素保持单调性，「新元素打破单调」时就批量结算答案。
- 存下标而非存值：因为答案需要「距离」，下标相减即可，这是单调栈的常见做法。

- 为什么单调递减：栈里保留的是「还没找到更高温度」的日子，越靠后的温度越低（否则早被弹出了）。
- 均摊 O(n)：虽然有嵌套 while，但总操作次数受限，这是均摊分析的应用。
- 衍生题：下一个更大元素 I/II（496/503）、接雨水（42）、柱状图中最大的矩形（84）。

84. 柱状图中最大的矩形 【困难】

题目描述

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。求在该柱状图中，能够勾勒出来的矩形的最大面积。

> **示例：**
>
> 输入：`heights = [2,1,5,6,2,3]`
>
> 输出：`10`

解题思路

关键思路：对每根柱子 i，以它的高度 heights[i] 作为矩形的高，矩形的宽就是「左右两侧第一个比它矮的柱子之间的距离 - 1」。所以问题转化为：对每根柱子，求「左边第一个更矮的位置」和「右边第一个更矮的位置」。

这正是单调栈的用武之地（与「每日温度」同源，只是要找「更小」）。

具体步骤（单调递增栈）：

1. 在 heights 两端各加一个高度为 0 的哨兵柱子（简化边界处理，保证所有柱子最终都会被弹出）。

2. 遍历每个下标 i：当栈非空且 heights[栈顶] > heights[i] 时：

   - 弹出栈顶 j，它就是要计算的柱子。

   - 弹出后新的栈顶就是「左边第一个比它矮的位置」，i 是「右边第一个比它矮的位置」。

   - 宽度 w = i - 新栈顶 - 1，面积 = heights[j] × w，更新最大值。

3. 把 i 压栈。

哨兵的作用：左哨兵（0）保证每个柱子都有左边界；右哨兵（0）保证循环结束后栈能清空。

不加哨兵就要写很多额外的收尾代码，容易出错。

参考代码

```
def largestRectangleArea(heights):
    # 两端加哨兵 0，简化边界
    hs = [0] + heights + [0]
    stack = [0]                 # 存下标，对应高度单调递增
    ans = 0
    for i in range(1, len(hs)):
        # 当前柱子比栈顶矮 -> 栈顶柱子可以结算了
        while hs[stack[-1]] > hs[i]:
            h = hs[stack.pop()]         # 被结算柱子的高度
            w = i - stack[-1] - 1       # 左右更矮柱子之间的距离
            ans = max(ans, h * w)
        stack.append(i)
    return ans
```

复杂度分析

时间复杂度 O(n)：每个下标最多入栈出栈一次。空间复杂度 O(n)：栈。

核心知识点

- 「以每根柱子为高，向两侧扩展」的枚举视角：把求最大面积的难题转化为对每根柱子求边界。
- 单调递增栈：找「左右两侧第一个更小元素」的标准工具。
- 宽度公式 w = i - stack[-1] - 1：左边界是「弹出后的新栈顶」，右边界是当前下标 i。
- 哨兵技巧：两端补 0，免去「栈未清空要额外收尾」和「左边界判断」的麻烦。
- 与「每日温度」的对比：一个找「更大」，一个找「更小」，栈的单调方向相反，框架一致。
- 同类题：接雨水（42）也可以用单调栈做（横向计算水量），可对照理解。

第 13 章 堆（优先队列）

本章核心思想

堆（Heap）是一种「能快速取到最值」的完全二叉树结构。Python 的 heapq 提供的是小顶堆：

```
heapq.heappush(h, x)   # 入堆, O(log n)
heapq.heappop(h)       # 弹出最小元素, O(log n)
h[0]                   # 查看最小元素, O(1)
```

要当大顶堆用，可以存负数（取出来再取反）。

堆的典型场景：Top K 问题（第 K 大、前 K 个高频）、多路归并、数据流中位数、任务调度。

Top K 的核心技巧：维护一个大小为 K 的堆，把复杂度从「全排序 O(n log n)」降到 O(n log k)。

215. 数组中的第 K 个最大元素 【中等】

题目描述

给定整数数组 nums 和整数 k，请返回数组中第 k 个最大的元素。请注意，你需要找的是数组排序后的第 k 个最大的元素，而不是第 k 个不同的元素。你必须设计并实现时间复杂度为 O(n) 的算法解决此问题。

示例：

```
输入：nums = [3,2,1,5,6,4], k = 2
输出：5
```

解题思路

方法一（小顶堆，O(n log k)，工程上最常用）：

维护一个大小为 k 的小顶堆。遍历数组，把元素压入堆；当堆的大小超过 k 时弹出堆顶（当前最小的）。遍历结束后，堆顶就是第 k 大的元素。

直观理解：堆里始终保留「目前见到的最大的 k 个元素」，其中最小的那个自然就是第 k 大。

方法二（快速选择 Quick Select，平均 O(n)，题目要求的解法）：

基于快速排序的分区思想：随机选一个基准元素 pivot，把数组分成「比 pivot 大」和「比 pivot 小」两部分。若「大于 pivot 的元素个数」正好是 k-1，那 pivot 就是答案；否则根据大小关系只在其中一侧继续分区，不必处理另一侧。

因为每次只递归一侧，平均复杂度是 O(n)（最坏 $O(n^2)$，随机化基准可以避免）。

初学建议先掌握堆解法（思路简单且稳定），再理解快速选择。

参考代码

```
import heapq, random

# 方法一: 小顶堆 (O(n log k))
def findKthLargest(nums, k):
    heap = []
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)     # 弹掉最小的，保留最大的 k 个
    return heap[0]                  # 堆顶即第 k 大

# 方法二: 快速选择 (平均 O(n))
def findKthLargest2(nums, k):
    def quick_select(l, r, k):      # 在 nums[l..r] 中找第 k 大
        pivot = nums[random.randint(l, r)]
        # 三路划分: 大于/等于/小于 pivot
        big = [x for x in nums[l:r+1] if x > pivot]
        eq  = [x for x in nums[l:r+1] if x == pivot]
        small = [x for x in nums[l:r+1] if x < pivot]
        if k <= len(big):
            return quick_select(l, l + len(big) - 1, k)
        elif k <= len(big) + len(eq):
            return pivot
        else:
            return quick_select(r - len(small) + 1, r, k - len(big) - len(eq))
    return quick_select(0, len(nums) - 1, k)

# 最简写法 (面试可用，但复杂度 O(n log n))
def findKthLargest3(nums, k):
    return sorted(nums, reverse=True)[k - 1]
```

复杂度分析

堆解法：时间 O(n log k)，空间 O(k)。快速选择：平均时间 O(n)，最坏 $O(n^2)$，空间 O(n)（或原地 O(1)）。

核心知识点

- Top K 问题：用「大小为 k 的堆」把复杂度从 O(n log n) 降到 O(n log k)，这是堆最经典的应用。
- 第 k 大用小顶堆（保留最大的 k 个），第 k 小用大顶堆（Python 中存负数模拟）。
- 快速选择（Quick Select）：只递归一侧的分治，平均 O(n)，是「不排序也能找第 k 位」的利器。
- 随机化基准：避免在有序数组上退化到 $O(n^2)$。
- 三路划分：把数组分成 >pivot、==pivot、<pivot 三部分，能优雅处理重复元素。

347. 前 K 个高频元素 【中等】

题目描述

给你一个整数数组 nums 和一个整数 k，请你返回其中出现频率前 k 高的元素。你可以按任意顺序返回答案。进阶：你所设计算法的时间复杂度必须优于 O(n log n)，其中 n 是数组大小。

> **示例：**
>
> 输入：nums = [1,1,1,2,2,3], k = 2
> 输出：[1,2]

解题思路

分两步：

第一步：用哈希表统计每个元素的出现次数（O(n)）。

第二步：从「元素 → 频次」中找出频次最大的 k 个元素。

第二步的三种做法：

1）按频次排序后取前 k 个：O(n log n)，答案正确但不符合「优于 O(n log n)」的要求。

2）小顶堆：维护大小为 k 的小顶堆（按频次比较），遍历所有元素，堆满就与堆顶比较，频次更高则替换堆顶。复杂度 O(n log k)。

3）桶排序：频次的范围是 [1, n]，所以可以用「频次作为桶的下标」，把元素放进 buckets[freq]。最后从高频桶往低频桶取，直到凑够 k 个。复杂度 O(n)，是本题最优解。

推荐掌握桶排序（最优）与堆（通用）两种写法。

参考代码

```
from collections import Counter
import heapq

# 方法一：桶排序 (O(n)，最优)
def topKFrequent(nums, k):
    cnt = Counter(nums)                  # 元素 -> 频次
    n = len(nums)
    buckets = [[] for _ in range(n + 1)] # 下标 = 频次
    for num, freq in cnt.items():
        buckets[freq].append(num)
    res = []
    for freq in range(n, 0, -1):         # 从高频到低频取
        if buckets[freq]:
            res.extend(buckets[freq])
            if len(res) >= k:
                return res[:k]
    return res

# 方法二：小顶堆 (O(n log k)，通用)
def topKFrequent2(nums, k):
    cnt = Counter(nums)
    heap = []
    for num, freq in cnt.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
            heapq.heappop(heap)          # 弹出频次最小的
    return [num for freq, num in heap]
```

复杂度分析

桶排序：时间 O(n)，空间 O(n)。小顶堆：时间 O(n log k)，空间 O(k)。

核心知识点

- 哈希计数 + Top K 的两阶段套路：先用哈希统计，再用合适结构取前 k。
- 桶排序：当「排序依据是一个有界的整数」（如频次 $\in [1, n]$）时，可以用下标直接表示键值，做到 O(n)。
- 小顶堆维护 Top K：与「第 K 大元素」同一套思路，只是比较的关键字换成了频次。
- Counter 计数器：Python 标准库的计数工具，比手写字典循环更简洁。
- 复杂度意识：题目要求「优于 O(n log n)」，直接排序会不达标，这是必须读出来的暗示。

295. 数据流的中位数 【困难】

题目描述

中位数是有序整数列表中间的数。如果列表的大小是偶数，则没有中间的数，中位数是中间两个数的平均值。实现 MedianFinder 类：MedianFinder() 初始化 MedianFinder 对象；void addNum(int num) 将数据流中的整数 num 添加到数据结构中；double findMedian() 返回到目前为止所有元素的中位数。与实际答案相差 10^{-5} 以内的答案将被接受。

> **示例：**
>
> 输入：["MedianFinder","addNum","addNum","findMedian","addNum","findMedian"]
> [[],[1],[2],[],[3],[]]
> 输出：[null,null,null,1.5,null,2.0]

解题思路

核心思路：用「两个堆」把数据分成左右两半。

- 大顶堆 left：存较小的一半数据（堆顶是左半部分的最大值）。

- 小顶堆 right：存较大的一半数据（堆顶是右半部分的最小值）。

只要保证「left 的大小 == right 的大小」或「left 比 right 多 1 个」，那么中位数就是 left 堆顶（奇数个）或 (left 堆顶 + right 堆顶)/2（偶数个）。

addNum 的步骤：

1. 先加入 left（大顶堆），然后把 left 的堆顶（也就是最大值）弹出来给 right。

   这一步保证「left 里的所有数 <= right 里的所有数」。

2. 若此时 right 比 left 多了一个元素，就把 right 的堆顶还给 left，保持平衡。

为什么需要「先左后右、再调整」这个顺序？因为直接判断该进哪个堆容易出错，「先无条件进 left，再把 left 的最大值转移给 right」可以自动保证两堆的大小关系。

Python 的 heapq 是小顶堆，所以 left 要存负数来实现大顶堆。

参考代码

```
import heapq

class MedianFinder:
    def __init__(self):
        self.left = []      # 大顶堆 (存负数): 较小的一半
        self.right = []     # 小顶堆: 较大的一半

    def addNum(self, num):
        # 1. 先放进 left(大顶堆), 再把 left 的最大值转移到 right
        heapq.heappush(self.left, -num)
        heapq.heappush(self.right, -heapq.heappop(self.left))
        # 2. 保持 left 的大小 >= right 的大小
        if len(self.right) > len(self.left):
            heapq.heappush(self.left, -heapq.heappop(self.right))

    def findMedian(self):
        if len(self.left) > len(self.right):
            return -self.left[0]                        # 奇数个: 左半最大值
        return (-self.left[0] + self.right[0]) / 2      # 偶数个: 中间两数平均
```

复杂度分析

addNum：O(log n)（堆操作）。findMedian：O(1)（只看堆顶）。空间复杂度 O(n)。

核心知识点

- 对顶堆（双堆）技巧：用大顶堆 + 小顶堆把数据分成两半，是「动态求中位数」的标准解法。
- Python 大顶堆的模拟：heapq 只有小顶堆，存负数即可反转为大顶堆。
- 「先进 left 再转移」的顺序：自动保证左半部分全部 <= 右半部分，避免复杂的条件判断。
- 大小平衡：时刻维持 len(left) == len(right) 或 len(left) == len(right) + 1。
- 为什么不用有序数组：插入操作是 O(n)，总复杂度 $O(n^2)$；双堆把插入降到 O(log n)。
- 同类应用：滑动窗口中位数（480）、数据流第 k 大元素等。

第 14 章 贪心算法

本章核心思想

贪心算法（Greedy）的核心是「每一步都做当前看起来最优的选择，并且相信这样能得到全局最优」。

贪心和动态规划的区别：
- 动态规划会考虑所有可能的选择并在其中取最优（有「状态转移方程」）。
- 贪心只做一个选择（局部最优），不做回退。

关键：**贪心不是万能的**，只有能证明「局部最优可以推出全局最优」时才能用。面试时如果想到贪心，最好能简单说明「为什么这样做不会错过最优解」。

常见的贪心套路：按某个关键字排序后依次处理、维护当前能到达的最远位置、边界的动态扩展。

121. 买卖股票的最佳时机 【简单】

题目描述

给定一个数组 prices，它的第 i 个元素 prices[i] 表示一支给定股票第 i 天的价格。你只能选择某一天买入这只股票，并选择在未来的某一个不同的日子卖出该股票。设计一个算法来计算你所能获取的最大利润。返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 0。

示例：

```
输入：prices = [7,1,5,3,6,4]
输出：5
解释：在第 2 天（价格 = 1）买入，在第 5 天（价格 = 6）卖出，最大利润 = 6 - 1 = 5。
```

解题思路

贪心策略：遍历价格时，始终维护「到目前为止的历史最低价」。对每一天，用「当天价格 - 历史最低价」作为「如果今天卖出能获得的利润」，取最大值。

为什么贪心是对的？因为「今天卖出」这个决策下，最优的买入价一定是之前所有天数里的最低价，两者是独立的，所以每一步取局部最优（当天卖出所得的最大利润）不会错过全局最优。

具体步骤：

1. min_price = ∞，best = 0。

2. 遍历每天价格 p：best = max(best, p - min_price)，然后 min_price = min(min_price, p)。

3. 返回 best。

注意顺序：先算利润再更新最低价，保证「卖出日在买入日之后」。（其实本题先更新再算也不会出错，因为同一天买卖利润为 0，但保持「先算后更新」更严谨。）

参考代码

```
def maxProfit(prices):
    min_price = float('inf')    # 历史最低价
    best = 0                    # 最大利润
    for p in prices:
        best = max(best, p - min_price)   # 假设今天卖出
        min_price = min(min_price, p)     # 更新历史最低价
    return best
```

复杂度分析

时间复杂度 O(n)：一次遍历。空间复杂度 O(1)：只用了两个变量。

核心知识点

- 贪心的「一次遍历维护历史最优」：不需要嵌套循环就能表达「买在前、卖在后」的约束。
- 状态压缩：本题的 DP 状态只有「历史最低价」一个，所以可以用变量代替数组。
- 顺序保证：先算利润、后更新最低价，确保卖出时间晚于买入时间。
- 系列题对比：122（可多次交易）用「累加所有上涨段」；123/188（限 k 次）需要正经 DP；309（含冷冻期）、714（含手续费）也是 DP。

55. 跳跃游戏 【中等】

题目描述

给你一个非负整数数组 nums，你最初位于数组的第一个下标。数组中的每个元素代表你在该位置可以跳跃的最大长度。判断你是否能够到达最后一个下标，如果可以，返回 true；否则，返回 false。

示例：

```
输入：nums = [2,3,1,1,4]
输出：true
解释：可以先跳 1 步到下标 1，然后再从下标 1 跳 3 步到达最后一个下标。
```

解题思路

贪心策略：维护一个变量 reach，表示「当前能够到达的最远下标」。从左到右遍历，如果当前下标 i <= reach（说明 i 可达），就用 i + nums[i] 更新 reach。如果遍历结束后 reach >= n-1，说明能到达终点。

关键推理：我们不需要知道「具体怎么跳」，只需要知道「最远能到哪」。只要最远能到终点，就一定存在跳跃方案。

具体步骤：

1. reach = 0。

2. 遍历 i 从 0 到 n-1：

   - 若 i > reach，说明下标 i 不可达，后面的位置也没法从更远处出发了……其实此时应该直接返回 False（因为已经断开）。

   - 否则 reach = max(reach, i + nums[i])。

   - 若 reach >= n-1，提前返回 True（优化）。

3. 遍历结束返回 reach >= n-1。

注意：遍历到 n-2 就够了，因为终点不需要再跳。

参考代码

```
def canJump(nums):
    reach = 0                       # 当前能到达的最远下标
    n = len(nums)
    for i in range(n):
        if i > reach:               # 下标 i 不可达，已经断开
            return False
        reach = max(reach, i + nums[i])
        if reach >= n - 1:          # 提前到达终点
            return True
    return True
```

复杂度分析

时间复杂度 O(n)：一次遍历。空间复杂度 O(1)。

核心知识点

- 「最远可达距离」的贪心：把「能否到达」的问题转化为「最远能到哪」，是这类跳跃问题的通用思路。
- 可达性判定：i > reach 说明出现了无法跨越的断点，可以立即返回 False。
- 单调扩展：reach 是单调不减的，所以一次正向遍历就足够了。
- 衍生题：跳跃游戏 II（45）求最少跳跃次数，思路相似但需要记录「当前层的边界」。
- 另一种解法（从后往前贪心）：从终点开始往前找「最靠左的可达点」，也能判断是否可达，但本题正向更简洁。

45. 跳跃游戏 II 【中等】

题目描述

给定一个长度为 n 的整数数组 nums。初始位置为 nums[0]。每个元素 nums[i] 表示从索引 i 向前跳转的最大长度。换句话说，如果你在 nums[i] 处，你可以跳转到任意 nums[i + j] 处，其中 0 <= j <= nums[i]。返回到达 nums[n - 1] 的最小跳跃次数。生成的测试用例可以到达 nums[n - 1]。

示例：

输入：nums = [2,3,1,1,4]

输出：2
解释：从下标 0 跳到下标 1（跳 1 步），然后跳 3 步到最后一个下标，总共跳 2 次。

解题思路

与上一题的「能否到达」不同，本题要求「最少步数」。贪心策略：把跳跃过程按「步数」分层，每一层代表「跳 1 次能覆盖的所有位置」。

具体步骤：

1. 维护 cur_end（当前这一步能到达的最远边界）、next_end（下一步能到达的最远边界）、steps（已跳步数）。

2. 遍历 i 从 0 到 n-2（不需要处理终点）：

- next_end = max(next_end, i + nums[i])（在当前位置可及的范围内，更新下一步能到的最远处）。

- 若 i == cur_end：说明已经走到了当前层的尽头，必须再跳一次，于是 steps += 1、cur_end = next_end。若 cur_end >= n-1 则返回 steps。

3. 返回 steps。

直观理解：在「跳一次能到达的范围内」，把下一步能到的最远处尽量推远，当必须迈出下一步时（走到边界），就跳一次。这就是「最优策略」。

另一种等价的 O(n²) DP 写法：dp[i] 表示到达下标 i 的最少步数。

参考代码

```
def jump(nums):
    n = len(nums)
    if n == 1:
        return 0
    cur_end = 0        # 当前这一跳能到达的最远边界
    next_end = 0       # 下一跳能到达的最远边界
    steps = 0
    for i in range(n - 1):
        next_end = max(next_end, i + nums[i])
        if i == cur_end:           # 走到了当前层尽头，必须跳一次
            steps += 1
            cur_end = next_end
            if cur_end >= n - 1:   # 已达终点
                break
    return steps
```

复杂度分析

时间复杂度 O(n)：一次遍历。空间复杂度 O(1)。

核心知识点

- BFS 分层的贪心实现：把「最少步数」转化为「最少层数」，每层代表一跳的覆盖范围。
- cur_end / next_end 双边界：一个表示「当前跳的边界」，一个表示「下一跳能到的最远」，是本题的核心。
- 在边界处才增加步数：只在「必须跳」的时候跳，保证步数最少。
- 贪心的正确性：在能到达的所有位置中，选「能延伸到最远」的那个作为下一跳起点，一定是最优的。
- 与题 55 的区别：55 只问「能否到达」，45 问「最少几步」，后者需要记录层数信息。

763. 划分字母区间 【中等】

题目描述

给你一个字符串 s。我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。

注意，划分结果需要满足：将所有划分结果按顺序连接，得到的字符串仍然是 s。返回一个表示每个字符串片段的长度的列表。

> **示例：**
>
> 输入：s = "ababcbacadefegdehijhklij"
> 输出：[9,7,8]
> 解释：划分结果为 "ababcbaca"、"defegde"、"hijhklij"。

解题思路

关键观察：一个片段必须包含「它内部所有字母的最后一次出现位置」。所以可以先记录每个字母的最后出现下标，然后在遍历时「不断扩张片段的右边界」。

具体步骤：

1. 先遍历一遍字符串，用字典 last 记录每个字符最后出现的下标。

2. 再遍历一遍字符串，维护 start（当前片段起点）和 end（当前片段至少要到的最远位置）：

- 每遇到一个字符 ch，令 end = max(end, last[ch])（扩张右边界）。

- 若当前下标 i == end，说明从这个起点到 i 的所有字母都不会再出现在更后面了，可以切分：记录长度 i - start + 1，并令 start = i + 1。

3. 返回所有片段长度。

贪心的本质：**在「可以切分」的最早位置就切分**，这样能得到最多的片段数。因为越早切分，后面的空间越大，一定能切出不少于「晚切分」的片段数。

参考代码

```
def partitionLabels(s):
    last = {}
    for i, ch in enumerate(s):
        last[ch] = i                # 每个字符最后出现的下标
    res = []
    start = 0
    end = 0
    for i, ch in enumerate(s):
        end = max(end, last[ch])    # 扩张当前片段的右边界
        if i == end:                # 到达边界，可以切分
            res.append(end - start + 1)
            start = i + 1
    return res
```

复杂度分析

时间复杂度 O(n)：两趟线性扫描。空间复杂度 O(1)：字典最多存 26 个字母（视为常数）。

核心知识点

- 「记录最后一次出现位置」的预处理：这是区间合并类问题的常用手段。
- 右边界的动态扩张：遍历时不断把 end 推到更远，直到「当前位置 == 右边界」才能切分。
- 贪心的切分时机：能切就切，保证片段数最多。
- 与「合并区间」的相似性：本质上都是在合并「有交集的区间」，只是这里用字符的首次/末次出现定义区间。
- 注意要求「尽可能多的片段」，所以不能提前切分（会切断同一字母）。

第 15 章 动态规划

本章核心思想

动态规划（Dynamic Programming, DP）是算法面试中最重要的思想之一。它的核心是「把大问题拆成重叠的子问题，并把子问题的答案记下来避免重复计算」。

做 DP 的四个步骤（强烈建议按顺序思考）：

1. **定义状态**：dp[i] 或 dp[i][j] 到底表示什么？（这是最关键、也最难的一步）

2. **写状态转移方程**：dp[i] 可以由哪些「更小的状态」推出来？

3. **确定初始条件与边界**：dp[0]、dp[1] 是多少？数组下标有没有越界？

4. **确定遍历顺序**：保证「用到的小状态」都已经算好了。

常见题型分类：

- 线性 DP：爬楼梯、打家劫舍、最长递增子序列

- 背包 DP：0-1 背包（分割等和子集）、完全背包（零钱兑换、完全平方数）

- 字符串 DP：单词拆分、最长有效括号

- 双状态 DP：乘积最大子数组（同时维护最大值和最小值）

70. 爬楼梯 【简单】

题目描述

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

示例：

```
输入：n = 3
输出：3
解释：有三种方法可以爬到楼顶。1+1+1、1+2、2+1。
```

解题思路

思考「最后一步」：要到达第 n 阶，最后一步要么从第 n-1 阶爬 1 阶上来，要么从第 n-2 阶爬 2 阶上来。

所以到达第 n 阶的方法数 = 到达第 n-1 阶的方法数 + 到达第 n-2 阶的方法数。

状态定义：dp[i] 表示「爬到第 i 阶的方法数」。

转移方程：dp[i] = dp[i-1] + dp[i-2]。

初始条件：dp[0] = 1（站在地面算 1 种）、dp[1] = 1。

这正是斐波那契数列！

由于 dp[i] 只依赖前两个状态，可以用两个变量滚动更新，空间降到 O(1)。

拓展思考：如果每次可以爬 1、2 或 3 阶，转移方程就变成 dp[i] = dp[i-1] + dp[i-2] + dp[i-3]，这就是「零钱兑换」类问题的思路雏形。

参考代码

```
# 标准 DP 写法
def climbStairs(n):
    if n <= 2:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    dp[2] = 2
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

# 空间优化: 滚动变量 (O(1) 空间)
def climbStairs2(n):
    if n <= 2:
        return n
    a, b = 1, 2          # a = dp[i-2], b = dp[i-1]
    for _ in range(3, n + 1):
        a, b = b, a + b
    return b
```

复杂度分析

时间复杂度 O(n)。空间复杂度：数组版本 O(n)，滚动变量版本 O(1)。

核心知识点

- 从「最后一步」倒推：DP 题最常用的思考起点——到达当前状态的最后一步有哪几种可能？
- 状态定义的重要性：dp[i] 的含义必须先说清楚，转移方程才写得出来。
- 滚动数组优化：只依赖前几个状态时，用变量代替数组。
- 斐波那契的变形：很多计数类 DP 最终都归结为斐波那契式的递推。
- DP vs 递归：朴素递归是 $O(2^n)$（大量重复计算），DP 通过「记忆化」把它降到 O(n)。

118. 杨辉三角 【简单】

题目描述

给定一个非负整数 numRows，生成「杨辉三角」的前 numRows 行。在「杨辉三角」中，每个数是它左上方和右上方的数的和。

> **示例：**
>
> 输入：numRows = 5
> 输出：[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]

解题思路

杨辉三角的规律：每行的首尾都是 1，中间的元素等于「上一行同位置元素 + 上一行前一个位置元素」。

具体步骤：

1. 结果列表 res 先放入第一行 [1]。

2. 对每一行 i（从 1 到 numRows-1）：

   - 取出上一行 prev = res[-1]。

   - 新建 cur 列表，长度为 i+1，首尾置 1，中间 cur[j] = prev[j-1] + prev[j]。

   - 把 cur 加入 res。

3. 返回 res。

另一种更简洁的生成技巧：从上一行相邻两项求和得到下一行，比如 [1,3,3,1] 相邻求和得 [1,4,6,4]，再在两端补 1 就得到 [1,4,6,4,1]。

这道题本身简单，但它是理解「二维 DP 递推」的入门，也是「组合数」的直观展示（第 n 行第 k 个数就是组合数 C(n,k)）。

参考代码

```
# 写法一：显式按位置计算
def generate(numRows):
    res = [[1]]
    for i in range(1, numRows):
        prev = res[-1]
        cur = [1] * (i + 1)          # 首尾都是 1
        for j in range(1, i):
            cur[j] = prev[j - 1] + prev[j]
        res.append(cur)
    return res[:numRows]

# 写法二：相邻求和 + 两端补 1（更简洁）
def generate2(numRows):
    res = []
    row = [1]
    for _ in range(numRows):
        res.append(row)
        row = [1] + [row[i] + row[i + 1] for i in range(len(row) - 1)] + [1]
    return res
```

复杂度分析

时间复杂度 $O(n^2)$：共生成 $n(n+1)/2$ 个数字。空间复杂度 $O(n^2)$（或 $O(n)$，若只保留上一行）。

核心知识点

- 二维递推：每行的状态由上一行推出，是最基础的自底向上 DP。
- 边界处理：首尾固定为 1，中间才需要计算，注意循环范围是 1 到 i-1。
- 组合数性质：杨辉三角第 n 行第 k 个数等于组合数 C(n, k)，与「不同路径」「组合总和」等题相关。
- 列表推导式：Python 中构造新列表的简洁写法，配合两端补 1 很优雅。
- numRows = 0 的边界：应返回空列表，res[:numRows] 可以自动处理。

198. 打家劫舍 【中等】

题目描述

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。给定一个代表每个房屋存放金额的非负整数数组，计算你不触动警报装置的情况下，一夜之内能够偷窃到的最高金额。

> **示例：**
>
> 输入：nums = [1,2,3,1]
> 输出：4
> 解释：偷窃 1 号房屋（金额 = 1），然后偷窃 3 号房屋（金额 = 3），总金额 = 4。

解题思路

经典的选择型 DP：对每间房，要么偷、要么不偷，且不能偷相邻的。

状态定义：dp[i] 表示「考虑前 i 间房时能偷到的最大金额」。

转移方程：dp[i] = max(dp[i-1], dp[i-2] + nums[i-1])。

含义：

- 不偷第 i 间：最大金额就是 dp[i-1]。

- 偷第 i 间：那第 i-1 间不能偷，最大金额是 dp[i-2] + 当前金额。

两者取最大。

初始条件：dp[0] = 0（没有房子），dp[1] = nums[0]。

同样可以滚动变量优化到 O(1) 空间：用两个变量分别表示 dp[i-2] 和 dp[i-1]。

理解重点：**「不能相邻」这个约束，正是通过 dp[i-2] 体现出来的**。看到「相邻不能同时选」的题目，都可以往这个模板上套。

参考代码

```
# 标准 DP
def rob(nums):
```

```
    n = len(nums)
    if n == 0:
        return 0
    if n == 1:
        return nums[0]
    dp = [0] * (n + 1)
    dp[1] = nums[0]
    for i in range(2, n + 1):
        dp[i] = max(dp[i - 1], dp[i - 2] + nums[i - 1])
    return dp[n]

# 空间优化：滚动变量
def rob2(nums):
    prev2, prev1 = 0, 0      # dp[i-2], dp[i-1]
    for x in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + x)
    return prev1
```

复杂度分析

时间复杂度 O(n)：一次遍历。空间复杂度：数组版本 O(n)，滚动变量版本 O(1)。

核心知识点

- 「选或不选」型 DP：核心是列出「选当前」和「不选当前」两种情况的转移。
- 相邻约束的建模：通过跳过一个状态（dp[i-2]）表达「不能相邻」的限制。
- 滚动变量优化：只依赖前两个状态，可省掉数组。
- 「不相邻子序列最大和」的通用性：同类约束的问题都可套此模板。
- 衍生题：打家劫舍 II（213）房屋成环，需分「偷第一间」和「不偷第一间」两种情况讨论；打家劫舍 III（337）在二叉树上做，需要树形 DP（返回两个值：偷/不偷该结点的最大收益）。

279. 完全平方数 【中等】

题目描述

给你一个整数 n，返回和为 n 的完全平方数的最少数量。完全平方数是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

示例：

输入：n = 12

输出：3

解释：12 = 4 + 4 + 4

输入：n = 13

输出：2

解释：13 = 4 + 9

解题思路

这是「完全背包」问题的变形：物品是各个完全平方数（1, 4, 9, 16, ...），每个物品可以无限次使用，目标是「恰好凑出 n，且物品数量最少」。

状态定义：dp[i] 表示「凑出 i 所需的最少完全平方数个数」。

转移方程：dp[i] = min(dp[i - j*j] + 1)，其中 j*j <= i。

含义：从 i 中减去一个完全平方数 j*j，剩下的 i-j*j 用最优方案凑出，再加上这一个数。

初始条件：dp[0] = 0，其余初始化为一个大数（表示暂时无法凑出）。

具体步骤：

1. dp = [∞] * (n+1)，dp[0] = 0。

2. i 从 1 到 n：对每个 j 从 1 开始、j*j <= i，dp[i] = min(dp[i], dp[i - j*j] + 1)。

3. 返回 dp[n]。

遍历顺序说明：外层遍历「容量 i」、内层遍历「物品 j*j」，这种顺序允许同一物品被重复使用，所以叫「完全背包」。（0-1 背包则反过来，外层物品、内层容量倒序。）

参考代码

```
def numSquares(n):
    dp = [float('inf')] * (n + 1)
    dp[0] = 0                          # 凑出 0 需要 0 个数
    for i in range(1, n + 1):          # 遍历容量
        j = 1
        while j * j <= i:              # 遍历物品(完全平方数)
            dp[i] = min(dp[i], dp[i - j * j] + 1)
            j += 1
    return dp[n]

# 进阶: 数学解法 (拉格朗日四平方和定理), O(sqrt(n))
def numSquares_math(n):
    # 若 n 本身是完全平方数, 答案 1
    if int(n ** 0.5) ** 2 == n:
```

```
        return 1
    # 若 n = 4^a * (8b + 7), 答案 4
    m = n
    while m % 4 == 0:
        m //= 4
    if m % 8 == 7:
        return 4
    # 检查能否表示为两个平方数之和
    j = 1
    while j * j <= n:
        rest = n - j * j
        if int(rest ** 0.5) ** 2 == rest:
            return 2
        j += 1
    return 3
```

复杂度分析

DP 解法：时间复杂度 O(n√n)，空间复杂度 O(n)。数学解法：时间 O(√n)，空间 O(1)。

核心知识点

- 完全背包模型：物品可无限次使用，「恰好凑出目标值」是这类问题的标志性描述。
- 状态定义要包含「目标量」：dp[i] 表示凑出 i 的最优解，i 就是「容量」。
- min 类型的 DP：初始化要用「无穷大」表示「尚不可达」。
- 循环顺序的含义：外层容量、内层物品 → 完全背包（可重复）；外层物品、内层容量倒序 → 0-1 背包（不可重复）。
- 数学定理（拉格朗日四平方和定理）：任何正整数都可表示为不超过 4 个完全平方数之和，所以答案只可能是 1、2、3、4。

322. 零钱兑换 【中等】

题目描述

给你一个整数数组 coins，表示不同面额的硬币；以及一个整数 amount，表示总金额。计算并返回可以凑成总金额所需的最少的硬币个数。如果没有任何一种硬币组合能组成总金额，返回 -1。你可以认为每种硬币的数量是无限的。

示例：

```
输入：coins = [1,2,5], amount = 11
```

输出：3

解释：11 = 5 + 5 + 1

解题思路

这是完全背包问题的标准形式：每种硬币可以用无限次，求「凑出 amount 的最少硬币数」。

状态定义：dp[i] 表示「凑出金额 i 所需的最少硬币数」。

转移方程：dp[i] = min(dp[i - coin] + 1)，对所有满足 coin <= i 的硬币面额取最小。

含义：最后一枚硬币的面额是 coin，那么前面的 i - coin 用最优方案凑出。

初始条件：dp[0] = 0，其余为 ∞。

具体步骤：

1. dp = [∞] * (amount+1)，dp[0] = 0。

2. i 从 1 到 amount：遍历每种 coin，若 coin <= i，则 dp[i] = min(dp[i], dp[i-coin] + 1)。

3. 返回 dp[amount]（若是 ∞ 则返回 -1）。

与「完全平方数」的对比：两者的框架完全一致，只是「物品」不同——一个是完全平方数，一个是给定面额的硬币。这就是 DP 的「模型迁移」。

参考代码

```
def coinChange(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for i in range(1, amount + 1):
        for coin in coins:
            if coin <= i:
                dp[i] = min(dp[i], dp[i - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

复杂度分析

时间复杂度 O(amount × n)，n 为硬币种类数。空间复杂度 O(amount)。

核心知识点

- 完全背包的「最少硬币数」变体：与「完全平方数」同源，是必掌握的 DP 模型。

- 无穷大初始化的意义：区分「这里还没找到合法方案」与「凑出需要 0 个硬币」。
- 最后要判断是否可达：dp[amount] 仍为 ∞ 说明无解，返回 -1。
- 为什么不能用贪心：贪心（每次选最大面额）在一般面额体系下会出错，例如 coins = [1,3,4]、amount = 6，贪心得 4+1+1 = 3 枚，而最优是 3+3 = 2 枚。
- 复杂度权衡：内层也可以写成「遍历 coin / 遍历 i」的顺序，效果相同。

139. 单词拆分 【中等】

题目描述

给你一个字符串 s 和一个字符串列表 wordDict 作为字典。如果可以利用字典中出现的一个或多个单词拼接出 s 则返回 true。注意：不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

示例：

```
输入：s = "leetcode", wordDict = ["leet","code"]
输出：true
解释：返回 true 因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。
```

解题思路

把问题看成「字符串切分」：dp[i] 表示「s 的前 i 个字符能否被字典中的单词拼出」。

转移方程：dp[i] = True，当且仅当存在某个 j < i，使得 dp[j] 为 True 且 s[j:i] 在字典中。

含义：如果前 j 个字符可以被拼出，且剩下的 s[j:i] 恰好是一个字典单词，那么前 i 个字符也可以被拼出。

初始条件：dp[0] = True（空字符串可以被拼出）。

具体步骤：

1. 把 wordDict 转成 set（查找 O(1)）。

2. dp = [False] * (n+1)，dp[0] = True。

3. i 从 1 到 n：j 从 0 到 i-1，若 dp[j] 为 True 且 s[j:i] 在字典中，则 dp[i] = True 并 break。

4. 返回 dp[n]。

优化：内层循环 j 可以从后往前遍历，或者限制 j >= i - maxLen（maxLen 是字典单词的最长长度），避免检查不可能的长度，能提升效率。

参考代码

```
def wordBreak(s, wordDict):
    words = set(wordDict)              # 转集合，查找 O(1)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True                       # 空串可以被拼出
    max_len = max((len(w) for w in wordDict), default=0)
    for i in range(1, n + 1):
        # 优化: j 只需回看 max_len 个字符
        for j in range(max(0, i - max_len), i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[n]
```

复杂度分析

时间复杂度 $O(n^2 \times L)$（不做优化时），L 为子串比较长度；优化后为 $O(n \times maxLen \times L)$。空间复杂度 O(n)。

核心知识点

- 「前缀可达性」型 DP：dp[i] 表示「前缀 i 是否可行」，是字符串切分类问题的标准状态定义。
- 切分点枚举：用「枚举上一个切分位置 j」来表达「最后一次拼接的单词」。
- 用 set 加速查找：把 list 转 set，查找从 O(k) 降到 O(1)。
- dp[0] = True 的必要性：代表「空字符串天然可行」，是所有切分点枚举的起点。
- 优化技巧：用最长单词长度限制 j 的回看范围，避免无效检查。
- 同类题：单词拆分 II（140）要求输出所有方案，需要用回溯 + 记忆化。

300. 最长递增子序列 【中等】

题目描述

给你一个整数数组 nums，找到其中最长严格递增子序列的长度。子序列是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。例如，[3,6,2,7] 是数组 [0,3,1,6,2,2,7] 的子序列。

> **示例：**
>
> 输入：nums = [10,9,2,5,3,7,101,18]
> 输出：4
> 解释：最长递增子序列是 [2,3,7,101]，因此长度为 4。

解题思路

方法一（经典 O(n²) DP，最易理解）：

状态定义：dp[i] 表示「以 nums[i] 结尾的最长严格递增子序列的长度」。

转移方程：dp[i] = max(dp[j] + 1)，对所有 j < i 且 nums[j] < nums[i]。

初始条件：dp[i] = 1（每个元素自身构成一个长度为 1 的子序列）。

答案是 max(dp)。

方法二（贪心 + 二分，O(n log n)，进阶）：

维护一个数组 tails，tails[k] 表示「长度为 k+1 的递增子序列的末尾元素的最小值」。这个数组本身是严格递增的。遍历每个数字 x 时，用二分查找找到 tails 中第一个 >= x 的位置：

- 若找到，就用 x 替换它（让该长度的末尾元素更小，更有利于后续延长）。

- 若没找到（x 比所有都大），就追加到末尾（子序列长度 +1）。

最终 tails 的长度就是答案。

「贪心」体现在「尽量让每个长度的末尾元素更小」，「二分」用于快速定位替换位置。

参考代码

```
# 方法一：经典 DP (O(n^2)，必掌握)
def lengthOfLIS(nums):
    n = len(nums)
    if n == 0:
        return 0
    dp = [1] * n                        # 每个元素自身长度为 1
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

# 方法二：贪心 + 二分 (O(n log n)，进阶)
import bisect

def lengthOfLIS2(nums):
    tails = []                          # tails[k] = 长度 k+1 的子序列的最小末尾
    for x in nums:
        pos = bisect.bisect_left(tails, x)   # 第一个 >= x 的位置
        if pos == len(tails):
            tails.append(x)             # x 比所有都大，延长
        else:
            tails[pos] = x              # 替换，让末尾更小
    return len(tails)
```

复杂度分析

DP 解法：时间 $O(n^2)$，空间 $O(n)$。贪心 + 二分：时间 $O(n \log n)$，空间 $O(n)$。

核心知识点

- 「以 i 结尾」的状态定义：子序列类问题几乎都要这样定义，否则无法写出转移方程。
- $O(n^2) \rightarrow O(n \log n)$ 的优化：用「维护单调数组 + 二分」替代「遍历所有前驱」。
- 贪心的正确性：更小的末尾元素一定不劣于更大的，所以可以放心替换。
- 注意题目要求「严格递增」，所以 b(search) 用 bisect_left；若允许非严格递增，则用 bisect_right。
- tail 数组本身不是答案的子序列，它的长度才是答案 —— 这是初学者常见的误解。
- 衍生题：俄罗斯套娃信封问题（354）、最长数对链（646）都是 LIS 的应用。

152. 乘积最大子数组 【中等】

题目描述

给你一个整数数组 nums，请你找出数组中乘积最大的非空连续子数组（该子数组中至少包含一个数字），并返回该子数组所对应的乘积。测试用例的答案是一个 32-位整数。子数组是数组的连续子序列。

> **示例：**
>
> 输入：nums = [2,3,-2,4]
> 输出：6
> 解释：子数组 [2,3] 有最大乘积 6。

解题思路

本题与「最大子数组和」很像，但有个关键区别：**负数的存在**。

一个很大的负数乘以一个负数会变成很大的正数，所以「最小值」也可能变成「最大值」。

因此需要**同时维护两个状态**：

- max_dp[i]：以 i 结尾的子数组的最大乘积。

- min_dp[i]：以 i 结尾的子数组的最小乘积（可能是很负的数）。

转移方程：设 a = max_dp[i-1] * nums[i]，b = min_dp[i-1] * nums[i]，c = nums[i]。

- max_dp[i] = max(a, b, c)

- min_dp[i] = min(a, b, c)

（三个候选：接在前面的最大乘积后面、接在最小乘积后面、或者自己另起一段。）

最终答案是 max(max_dp)。

由于只用到了前一组状态，可以滚动成变量，空间 O(1)。

参考代码

```
def maxProduct(nums):
    # 同时维护以当前位置结尾的最大乘积和最小乘积
```

```
    max_dp = min_dp = nums[0]
    ans = nums[0]
    for x in nums[1:]:
        a, b = max_dp * x, min_dp * x
        max_dp = max(a, b, x)      # 三个候选取最大
        min_dp = min(a, b, x)      # 三个候选取最小
        ans = max(ans, max_dp)
    return ans
```

复杂度分析

时间复杂度 O(n)：一次遍历。空间复杂度 O(1)：只用了常数个变量。

核心知识点

- 双状态 DP：因为「负负得正」，必须同时记录最大和最小，这是本题与「最大子数组和」的根本差别。
- 三个候选的含义：接最大、接最小、另起炉灶，缺一不可。
- 就地滚动：用 max_dp / min_dp 两个变量滚代替数组。
- 小心变量更新顺序：「另起炉灶」的候选 x 已经包含在原数组里，所以不需要额外的 dp 数组。
- 同类对比：最大子数组和（53）只需维护一个状态；本题需要两个状态。遇到「乘法/绝对值」类问题要警惕符号翻转。

416. 分割等和子集 【中等】

题目描述

给你一个只包含正整数的非空数组 nums。请你判断是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

示例：

输入：nums = [1,5,11,5]
输出：true
解释：数组可以分割成 [1,5,5] 和 [11]。

解题思路

转化思路：如果能把数组分成两个和相等的子集，那么每个子集的和一定是「总和的一半」。

于是问题变成：**能否从数组中选出若干数，使它们的和恰好等于 sum/2**？这是一个标准的「0-1 背包」问题。

前置判断：若总和 sum 为奇数，直接返回 False（不可能平分）。

状态定义：dp[j] 表示「能否用数组中的数凑出和 j」（布尔 DP）。

转移方程（0-1 背包的逆序遍历）：对每个数字 num，j 从 target 递减到 num：

```
dp[j] = dp[j] or dp[j - num]
```

含义：凑出 j 有两种方式——不用 num（dp[j] 原本为 True），或用 num（则要求 dp[j-num] 为 True）。

为什么 j 要「逆序」遍历？这是 0-1 背包的关键细节：如果正序遍历，dp[j-num] 可能在本次循环中已经被更新过（即 num 被用了两次），那就变成了「完全背包」。

逆序遍历保证每个数字只用一次。

可以把 dp 数组想象成一维的「可达和集合」，最终看 dp[target] 是否为 True。

参考代码

```
def canPartition(nums):
    total = sum(nums)
    if total % 2 == 1:              # 总和为奇数，不可能平分
        return False
    target = total // 2
    dp = [False] * (target + 1)
    dp[0] = True                    # 和为 0 总是可达的
    for num in nums:                # 遍历物品
        # 逆序遍历容量，保证每个数只用一次
        for j in range(target, num - 1, -1):
            dp[j] = dp[j] or dp[j - num]
    return dp[target]
```

复杂度分析

时间复杂度 O(n × target)，target = sum/2。空间复杂度 O(target)。

核心知识点

- 问题转化：把「平分数组」转化为「选出和为 sum/2 的子集」，是 0-1 背包的经典识别方式。

- 布尔 DP：dp[j] 表示「和 j 是否可达」，比「最大价值」型的背包更容易理解。
- 0-1 背包的逆序遍历：防止同一个数字被重复使用，是最关键、最容易写错的细节。
- 空间优化：二维 dp[i][j] 压缩成一维 dp[j]，因为 dp[i] 只依赖 dp[i-1]。
- 前置剪枝：总和为奇数直接返回 False；若 max(nums) > target 也可提前返回 False。
- 同类题：最后一块石头的重量 II（1049）、目标和（494）都是 0-1 背包的变体。

32. 最长有效括号 【困难】

题目描述

给你一个只包含 '(' 和 ')' 的字符串，找出最长有效（格式正确且连续）括号子串的长度。

示例：

```
输入：s = "(()"
输出：2
输入：s = ")()())"
输出：4
解释：最长有效括号子串是 "()()"。
```

解题思路

三种解法，从易到难：

方法一（栈，O(n) 时间 / O(n) 空间，推荐）：

栈里存下标，初始压入 -1 作为「最后一个未被匹配的位置」的哨兵。

遍历字符串：

- 遇到 '(' ：把下标压栈。

- 遇到 ')' ：先弹栈；若栈为空，说明这个右括号没有匹配，把当前下标压栈作为新的「基准」；

若栈非空，则「当前下标 - 栈顶下标」就是以当前位置结尾的最长有效长度。

方法二（DP，O(n) 时间 / O(n) 空间）：

dp[i] 表示「以 s[i] 结尾的最长有效括号长度」。

- 若 s[i] == ')' 且 s[i-1] == '('：dp[i] = dp[i-2] + 2。

- 若 s[i] == ')' 且 s[i-1] == ')'：若 s[i - dp[i-1] - 1] == '('，则 dp[i] = dp[i-1] + 2 + dp[i - dp[i-1] - 2]。

（含义：跳过前一个有效块，检查它前面的字符是否是 '('。）

方法三（双向计数，O(n) 时间 / O(1) 空间，最巧妙）：

正反各扫一遍，用 left / right 分别计数左右括号。正向扫描时，若 right > left 就说明非法，重置计数；若 right == left 就更新答案。反向再扫一遍（此时把含义反转），可以处理「左括号过多」的情况。

参考代码

```
# 方法一：栈（推荐，直观）
def longestValidParentheses(s):
    stack = [-1]          # 哨兵：最后一个未匹配的位置
    ans = 0
    for i, ch in enumerate(s):
        if ch == '(':
            stack.append(i)
        else:
            stack.pop()
            if not stack:
                stack.append(i)          # 无匹配，当前下标成为新基准
            else:
                ans = max(ans, i - stack[-1])
    return ans

# 方法二：动态规划
def longestValidParentheses2(s):
    n = len(s)
    dp = [0] * n
    ans = 0
    for i in range(1, n):
        if s[i] != ')':
            continue
        if s[i - 1] == '(':
            dp[i] = (dp[i - 2] if i >= 2 else 0) + 2
        elif i - dp[i - 1] > 0 and s[i - dp[i - 1] - 1] == '(':
            prev = dp[i - dp[i - 1] - 2] if i - dp[i - 1] >= 2 else 0
            dp[i] = dp[i - 1] + 2 + prev
        ans = max(ans, dp[i])
    return ans

# 方法三：双向计数（空间 O(1)，最巧妙）
def longestValidParentheses3(s):
    def scan(seq):
        left = right = best = 0
        for ch in seq:
            if ch == '(':
```

```
                left += 1
            else:
                right += 1
            if left == right:
                best = max(best, 2 * right)
            elif right > left:           # 右括号过多，非法，重置
                left = right = 0
        return best
    # 正向 + 反向(把括号对调后正向)
    return max(scan(s), scan(s[::-1].replace('(', '#').replace(')', '(').replace('#', ')')))
```

复杂度分析

栈解法：时间 O(n)，空间 O(n)。DP 解法：时间 O(n)，空间 O(n)。双向计数：时间 O(n)，空间 O(1)。

核心知识点

- 栈 + 哨兵 -1：用「最后一个未匹配位置」作为计算长度的基准，是栈解法的精髓。
- 「栈为空则压入当前下标」的技巧：相当于把基准推进到新的位置。
- DP 的两种转移情况：相邻配对、以及「跳过已有有效块后再配对」。
- 双向计数法：正向扫描处理「右括号过多」，反向扫描处理「左括号过多」，两者取最大。
- 「最长有效括号」是括号类问题的最难题型，但核心工具仍是栈和 DP，与前两题（20、22）一脉相承。

第 16 章 多维动态规划

本章核心思想

多维 DP 指的是状态由两个（或多个）维度共同决定，典型形式是 dp[i][j]。常见于：

1）网格路径问题：dp[i][j] 表示「到达格子 (i,j) 的最优值」，由「上方」和「左方」推出。

2）双序列问题（两个字符串 / 两个数组）：dp[i][j] 表示「第一个序列前 i 个、第二个序列前 j 个」的最优值。这是最难也最重要的一类，最长公共子序列、编辑距离都属于它。

解题要点：先想清楚 dp[i][j] 的两个维度各代表什么，再思考「最后一步」如何从 (i-1,j)、(i,j-1)、(i-1,j-1) 转移过来。

另外，很多二维 DP 都可以「按行滚动」压缩成一维，从而优化空间。

62. 不同路径 【中等】

题目描述

一个机器人位于一个 m x n 网格的左上角（起始点在下图中标记为 "Start"）。机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角。问总共有多少条不同的路径？

示例：

```
输入：m = 3, n = 7
输出：28
```

解题思路

状态定义：dp[i][j] 表示「从起点走到格子 (i,j) 的路径数」。

转移方程：dp[i][j] = dp[i-1][j] + dp[i][j-1]。

含义：到达 (i,j) 只能从上方 (i-1,j) 或左方 (i,j-1) 过来，所以路径数是两者之和。

初始条件：第一行和第一列都只有 1 条路径（一直向右 / 一直向下）。

具体步骤：

1. 建 m×n 的 dp 数组，全部初始化为 1。

2. i 从 1 到 m-1，j 从 1 到 n-1：dp[i][j] = dp[i-1][j] + dp[i][j-1]。

3. 返回 dp[m-1][n-1]。

空间优化（滚动数组）：因为 dp[i][j] 只依赖「上一行同列」和「本行前一列」，所以可以用一维数组 dp[j]，在遍历时不断更新：dp[j] = dp[j]（上一行的值）+ dp[j-1]（本行已更新的值）。

这样空间从 O(m×n) 降到 O(n)。

数学解法（组合数）：一共要走 (m-1) + (n-1) 步，其中选 (m-1) 步向下，所以答案是 C(m+n-2, m-1)。

参考代码

```
# 二维 DP
def uniquePaths(m, n):
    dp = [[1] * n for _ in range(m)]      # 第一行/第一列都是 1
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[m - 1][n - 1]

# 空间优化：一维滚动数组 (O(n) 空间)
def uniquePaths2(m, n):
    dp = [1] * n                          # 第一行的初值
    for i in range(1, m):
        for j in range(1, n):
            dp[j] = dp[j] + dp[j - 1]     # dp[j]=上一行，dp[j-1]=本行已更新
    return dp[n - 1]
```

复杂度分析

时间复杂度 O(m×n)。空间复杂度：二维版本 O(m×n)，滚动数组版本 O(n)。

核心知识点

- 二维 DP 的状态设计：dp[i][j] 表示「到达该格子的方案数」，是网格路径类问题的标准定义。
- 「只能从上方或左方来」→ 转移方程是两者相加，这是计数型 DP 的典型形式。
- 边界初始化：第一行/第一列只有唯一路径，要单独赋值为 1。
- 滚动数组优化：注意到「只依赖上一行」就可以降维，注意遍历方向要用「本行已更新的值」。

- 组合数视角：路径问题等价于「在总步数中选出若干步用于某个方向」，与杨辉三角同理。
- 衍生题：不同路径 II（63）加入障碍物；最小路径和（64）把「计数」换成「求最小和」。

64. 最小路径和 【中等】

题目描述

给定一个包含非负整数的 m x n 网格 grid，请找出一条从左上角到右下角的路径，使得路径上的数字总和为最小。说明：每次只能向下或者向右移动一步。

> **示例：**
>
> 输入：grid = [[1,3,1],[1,5,1],[4,2,1]]
>
> 输出：7
>
> 解释：路径 1→3→1→1→1 的总和最小。

解题思路

与「不同路径」框架完全一致，只是把「求和」换成了「求最小」。

状态定义：dp[i][j] 表示「从起点走到 (i,j) 的最小路径和」。

转移方程：dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1])。

初始条件：

- dp[0][0] = grid[0][0]。

- 第一行：只能从左边来，dp[0][j] = dp[0][j-1] + grid[0][j]。

- 第一列：只能从上边来，dp[i][0] = dp[i-1][0] + grid[i][0]。

也可以在原数组上直接修改（原地 DP），不需要额外空间。

面试小技巧：先写二维版本，再提到「可以用一维数组滚动优化」，这是展现理解深度的好方式。

参考代码

```
# 二维 DP
def minPathSum(grid):
    m, n = len(grid), len(grid[0])
```

```
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = grid[0][0]
    for j in range(1, n):                  # 第一行
        dp[0][j] = dp[0][j - 1] + grid[0][j]
    for i in range(1, m):                  # 第一列
        dp[i][0] = dp[i - 1][0] + grid[i][0]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = grid[i][j] + min(dp[i - 1][j], dp[i][j - 1])
    return dp[m - 1][n - 1]

# 空间优化：一维滚动（也可直接原地修改 grid）
def minPathSum2(grid):
    m, n = len(grid), len(grid[0])
    dp = [0] * n
    dp[0] = grid[0][0]
    for j in range(1, n):
        dp[j] = dp[j - 1] + grid[0][j]
    for i in range(1, m):
        dp[0] += grid[i][0]                # 第一列只能从上边来
        for j in range(1, n):
            dp[j] = grid[i][j] + min(dp[j], dp[j - 1])
    return dp[n - 1]
```

复杂度分析

时间复杂度 O(m×n)。空间复杂度：二维 O(m×n)，滚动数组 O(n)，原地修改 O(1)。

核心知识点

- 「最优子结构」：到达某格子的最优路径，一定由「到达其上方或左方的最优路径」延伸而来。
- 边界初始化：第一行只能向右、第一列只能向下，要单独处理。
- min 型 DP 的模板：dp[i][j] = 当前代价 + min(两个来源)。
- 空间优化：一维滚动时，dp[j] 在更新前代表「上一行同列」，更新后代表「本行同列」。
- 与不同路径（62）的对比：同一模型，一个求方案数（相加），一个求最小代价（取 min）。

5. 最长回文子串 【中等】

题目描述

给你一个字符串 s，找到 s 中最长的回文子串。

示例：

```
输入：s = "babad"
```

输出："bab"

解释："aba" 同样是符合题意的答案。

解题思路

方法一（中心扩展，O(n²) 时间 / O(1) 空间，最推荐）：

回文串的特点是「关于中心对称」。所以可以枚举所有可能的中心，向两边扩展，直到不再是回文为止，记录最长的那个。

中心有两类：

- 奇数长度回文，中心是 1 个字符（如 "aba" 的中心是 'b'）。

- 偶数长度回文，中心是 2 个字符之间（如 "abba" 的中心在两个 'b' 之间）。

所以要对「以 i 为中心」和「以 i 和 i+1 之间为中心」两种情况进行扩展。

方法二（DP，O(n²) 时间 / O(n²) 空间）：

dp[i][j] 表示「s[i..j] 是否为回文」。转移：

$$dp[i][j] = (s[i] == s[j]) \text{ and } (j - i <= 2 \text{ or } dp[i+1][j-1])$$

遍历时 i 从大到小、j 从小到大，保证 dp[i+1][j-1] 已经被计算。

方法三（Manacher 算法，O(n)）：竞赛级解法，理解难度较大，面试中一般不需要。

参考代码

```
# 方法一: 中心扩展 (推荐, O(1) 空间)
def longestPalindrome(s):
    if not s:
        return ""
    start, max_len = 0, 1

    def expand(l, r):            # 从 l,r 向两侧扩展, 返回回文长度
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1
            r += 1
        return r - l - 1          # 退出时 l,r 已越界/不匹配, 长度要减 1

    for i in range(len(s)):
        len1 = expand(i, i)       # 奇数长度中心
        len2 = expand(i, i + 1)   # 偶数长度中心
        cur = max(len1, len2)
        if cur > max_len:
```

```
            max_len = cur
            start = i - (cur - 1) // 2   # 反推起点
    return s[start:start + max_len]

# 方法二：区间 DP
def longestPalindrome2(s):
    n = len(s)
    if n < 2:
        return s
    dp = [[False] * n for _ in range(n)]
    start, max_len = 0, 1
    for i in range(n):
        dp[i][i] = True
    for i in range(n - 1, -1, -1):
        for j in range(i + 1, n):
            if s[i] == s[j] and (j - i <= 2 or dp[i + 1][j - 1]):
                dp[i][j] = True
                if j - i + 1 > max_len:
                    max_len = j - i + 1
                    start = i
    return s[start:start + max_len]
```

复杂度分析

中心扩展：时间 $O(n^2)$，空间 $O(1)$。区间 DP：时间 $O(n^2)$，空间 $O(n^2)$。

核心知识点

- 中心扩展法：利用回文的对称性，枚举中心后向两侧扩展，是最直观高效的方法。
- 两类中心：奇数长度（1 个字符）与偶数长度（两个字符之间），缺一不可。
- 起点反推公式：start = i - (len - 1) // 2，是中心扩展中容易写错的地方，建议画图验证。
- 区间 DP 的遍历方向：i 倒序、j 正序，保证 dp[i+1][j-1] 已被计算。
- 回文判定递推式：s[i]==s[j] and (j-i<=2 or dp[i+1][j-1])，是所有回文问题的通用判别公式。

1143. 最长公共子序列 【中等】

题目描述

给定两个字符串 text1 和 text2，返回这两个字符串的最长公共子序列的长度。如果不存在公共子序列，返回 0。一个字符串的子序列是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。例如，"ace" 是

"abcde" 的子序列，但 "aec" 不是。两个字符串的公共子序列是这两个字符串所共同拥有的子序列。

示例：

```
输入：text1 = "abcde", text2 = "ace"
输出：3
解释：最长公共子序列是 "ace"，它的长度为 3。
```

解题思路

这是「双序列 DP」的标杆题目，模板可迁移到大量字符串问题上。

状态定义：dp[i][j] 表示「text1 的前 i 个字符」与「text2 的前 j 个字符」的最长公共子序列长度。

转移方程：

- 若 text1[i-1] == text2[j-1]：dp[i][j] = dp[i-1][j-1] + 1（这个字符可以同时用上）。

- 否则：dp[i][j] = max(dp[i-1][j], dp[i][j-1])（去掉其中一个末尾字符，取较优）。

初始条件：dp[0][*] = dp[*][0] = 0（空串与任何串的 LCS 都是 0）。

具体步骤：

1. 建 (m+1)×(n+1) 的 dp 数组，全部初始化为 0。

2. i 从 1 到 m、j 从 1 到 n，按上面的转移方程填充。

3. 返回 dp[m][n]。

为什么 dp 数组要多一行一列？因为要表示「空串」的情况，避免下标出现 -1。

这是双序列 DP 的标准做法。

参考代码

```
def longestCommonSubsequence(text1, text2):
    m, n = len(text1), len(text2)
    # dp[i][j] = text1 前 i 个 与 text2 前 j 个 的 LCS 长度
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
```

```
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

复杂度分析

时间复杂度 O(m×n)。空间复杂度 O(m×n)，可用滚动数组优化到 O(min(m,n))。

核心知识点

- 双序列 DP 的通用模板：dp[i][j] 表示「两个序列各取前 i / 前 j 个」的最优解。
- 多一行一列的意义：表示空串状态，让下标从 1 开始，避免边界判断。
- 字符相等则「同时消耗」两个字符（i-1, j-1 转移）；不等则「只消耗一个」（取 max）。
- 对比「最长公共子串」：子串要求连续，转移方程不同（不等时要清零）。
- 衍生应用：编辑距离（72）、最长回文子序列（516）、两个字符串的删除操作（583）都基于此模板。

72. 编辑距离 【中等】

题目描述

给你两个单词 word1 和 word2，请返回将 word1 转换成 word2 所使用的最少操作数。你可以对一个单词进行如下三种操作：插入一个字符、删除一个字符、替换一个字符。

示例：

```
输入：word1 = "horse", word2 = "ros"
输出：3
解释：horse -> rorse (替换 'h' 为 'r') -> rose (删除 'r') -> ros (删除 'e')
```

解题思路

这是双序列 DP 中最经典的题目，也是 NLP 中计算字符串相似度的基础算法。

状态定义：dp[i][j] 表示「把 word1 的前 i 个字符变成 word2 的前 j 个字符」所需的最少操作数。

转移方程：

- 若 word1[i-1] == word2[j-1]：dp[i][j] = dp[i-1][j-1]（字符相同，无需操作）。

- 否则：dp[i][j] = 1 + min(

  dp[i-1][j], # 删除 word1 的一个字符

  dp[i][j-1], # 在 word1 中插入一个字符

  dp[i-1][j-1] # 替换一个字符

)

初始条件：

- dp[i][0] = i（把 word1 前 i 个字符变成空串，需要删 i 次）。

- dp[0][j] = j（把空串变成 word2 前 j 个字符，需要插入 j 次）。

理解三个操作对应的状态转移：

- 删除：word1 少一个字符，所以从 dp[i-1][j] 来。

- 插入：word2 少一个字符（等价于在 word1 里补一个），所以从 dp[i][j-1] 来。

- 替换：两个都少一个字符，所以从 dp[i-1][j-1] 来。

参考代码

```
def minDistance(word1, word2):
    m, n = len(word1), len(word2)
    # dp[i][j] = word1 前 i 个 -> word2 前 j 个 的最少操作数
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    # 初始化: 变成空串 / 从空串变来
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]      # 字符相同，无操作
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],                # 删除
                    dp[i][j - 1],                # 插入
                    dp[i - 1][j - 1],            # 替换
                )
    return dp[m][n]
```

复杂度分析

时间复杂度 O(m×n)。空间复杂度 O(m×n)，可用滚动数组优化到 O(n)。

核心知识点

- 编辑距离（Levenshtein Distance）：字符串相似度度量的经典算法，应用广泛（拼写纠错、DNA 比对、模糊匹配）。
- 双序列 DP 的三分支转移：删除 / 插入 / 替换，分别对应 dp[i-1][j]、dp[i][j-1]、dp[i-1][j-1]。
- 初始化的含义：dp[i][0] 和 dp[0][j] 代表「与空串之间的转换」，必须显式赋值。
- 理解要点：把「对 word1 的操作」重新表述为「两个序列的下标如何一起前移」。
- 空间优化：dp[i][j] 只依赖上一行和本行左侧，所以可以压缩成一维数组，但要注意保存 dp[i-1][j-1]。

第 17 章 技巧

本章核心思想

这一组题目考察的是「巧妙的观察」——它们往往有不止一种解法，但存在利用题目特殊条件的巧妙解法，能把时间或空间复杂度降到极低。

这类题目是最能拉开水平差距的：能写出通用解法是及格，能想到巧妙解法是优秀。核心能力是「观察题目给出的特殊约束」，比如「只有一个数出现一次」「一定存在多数元素」「元素范围是 [1,n]」。

136. 只出现一次的数字 【简单】

题目描述

给你一个非空整数数组 nums，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。你必须设计并实现线性时间复杂度的算法来解决此问题，且该算法只使用常量额外空间。

示例：

```
输入：nums = [4,1,2,1,2]
输出：4
```

解题思路

最巧妙也最简洁的解法：**异或运算（XOR）**。

异或的三个关键性质：

1. a ^ a = 0（相同的数异或得 0）。

2. a ^ 0 = a（任何数与 0 异或等于自身）。

3. 异或满足交换律和结合律（顺序不重要）。

于是把数组里所有数全部异或起来：成对出现的数会互相抵消变成 0，最后剩下的就是那个只出现一次的数。

具体步骤：

1. ans = 0。

2. 遍历每个数 x：ans ^= x。

3. 返回 ans。

其他解法（供对比）：哈希表统计（O(n) 空间，不满足空间要求）；排序后相邻比较（O(n log n) 时间，不满足时间要求）。只有异或同时满足两个要求。

参考代码

```
def singleNumber(nums):
    ans = 0
    for x in nums:
        ans ^= x          # 成对的数会互相抵消为 0
    return ans

# 一行写法
from functools import reduce
def singleNumber2(nums):
    return reduce(lambda a, b: a ^ b, nums, 0)
```

复杂度分析

时间复杂度 O(n)：一次遍历。空间复杂度 O(1)：只用一个变量。

核心知识点

- 异或（XOR）运算：位运算中最常用的「消重」工具，性质 a^a=0、a^0=a 是核心。
- 交换律与结合律：所以异或的顺序无关，可以一次遍历全部异或。
- 位运算的价值：在特定约束下可以实现 O(1) 空间的技巧性解法。
- 读题关键：题目「其余每个元素均出现两次」这个条件，正是为异或解法量身定做的。
- 衍生题：只出现一次的数字 II（137，其余出现 3 次，需按位统计）/ III（260，两个数只出现一次，需按位分组）。

169. 多数元素 【简单】

题目描述

给定一个大小为 n 的数组 nums，返回其中的多数元素。多数元素是指在数组中出现次数大于 ⌊n/2⌋ 的元素。你可以假设数组是非空的，并且给定的数组总是存在多数元素。

示例：

```
输入：nums = [3,2,3]
输出：3
```

解题思路

方法一（哈希表计数，O(n) 空间，最直观）：统计每个数的出现次数，找到超过 n/2 的那个。

方法二（排序，O(n log n)）：排序后下标为 n//2 的元素一定是多数元素（因为它出现超过一半，必定覆盖数组正中间的位置）。

方法三（Boyer-Moore 投票算法，O(n) 时间 / O(1) 空间，最优解）：

核心思想：「多数元素出现次数超过一半，所以它能抵消掉所有其他元素」。

具体步骤：

1. candidate = None，count = 0。

2. 遍历每个数 x：

   - 若 count == 0：说明之前的候选被消耗完了，把 x 作为新候选（candidate = x，count = 1）。

   - 否则若 x == candidate：count += 1（支持它）。

   - 否则：count -= 1（反对它，抵消一个）。

3. 遍历结束后，candidate 就是多数元素。

直观比喻：把每个非多数元素都看作「反对票」，多数元素每张赞成票只能抵消一张反对票，但由于它数量过半，最后一定还剩着。

参考代码

```
# 方法一：哈希计数
from collections import Counter

def majorityElement(nums):
    counts = Counter(nums)
    return counts.most_common(1)[0][0]

# 方法二：排序取中位
def majorityElement2(nums):
    nums.sort()
    return nums[len(nums) // 2]

# 方法三：Boyer-Moore 投票算法（最优）
def majorityElement3(nums):
    candidate = None
    count = 0
    for x in nums:
        if count == 0:
            candidate = x          # 换新候选
            count = 1
        elif x == candidate:
            count += 1             # 支持
        else:
            count -= 1             # 反对，消耗一张票
    return candidate
```

复杂度分析

哈希法：时间 O(n)，空间 O(n)。排序法：时间 O(n log n)，空间 O(1)（原地排序）。投票算法：时间 O(n)，空间 O(1)。

核心知识点

- Boyer-Moore 投票算法：用「互相抵消」的思路在 O(1) 空间找出出现次数超过一半的元素，非常巧妙。
- 多数元素的性质：出现次数 > n/2，所以无法被完全抵消，这是算法的正确性基础。
- 排序取中位：因为多数元素数量过半，必然占据排序后的中间位置。
- 注意条件：只有在「保证存在多数元素」时，投票算法的结果才一定正确；否则需要再遍历一次验证。
- Counter.most_common：Python 标准库的计数器工具，业务代码中很实用。

75. 颜色分类 【中等】

题目描述

给定一个包含红色、白色和蓝色、共 n 个元素的数组 nums，原地对它们进行排序，使得相同颜色的元素相邻，并按照红色、白色、蓝色顺序排列。我们使用整数 0、1 和 2 分别表示红色、白色和蓝色。必须在不使用库内置的 sort 函数的情况下解决这个问题。

> **示例：**
>
> 输入：nums = [2,0,2,1,1,0]
> 输出：[0,0,1,1,2,2]

解题思路

本题是「荷兰国旗问题」（Dutch National Flag Problem），用三指针一次遍历完成。

三指针的含义：把数组分成三个区域：

- [0, p0)：全是 0。

- [p0, i)：全是 1。

- (p2, n-1]：全是 2。

- [i, p2]：尚未处理的区域。

具体步骤：

1. p0 = 0，i = 0，p2 = n - 1。

2. 当 i <= p2 时循环，检查 nums[i]：

   - 若为 0：与 nums[p0] 交换，p0 += 1，i += 1。

   - 若为 1：i += 1（已经在正确位置）。

   - 若为 2：与 nums[p2] 交换，p2 -= 1（**注意这里 i 不动**，因为换过来的元素还没检查）。

最容易出错的地方就是「遇到 2 时 i 不动」：因为从右侧换过来的元素尚未被检查过，需要下一轮循环再判断。而遇到 0 时交换过来的元素一定是 1（或自身），所以可以直接 i++。

参考代码

```
def sortColors(nums):
    p0 = 0                # 0 区域的右边界(不含)
    p2 = len(nums) - 1    # 2 区域的左边界(不含)
    i = 0                 # 当前检查位置
    while i <= p2:
        if nums[i] == 0:
            nums[p0], nums[i] = nums[i], nums[p0]
            p0 += 1
            i += 1
        elif nums[i] == 1:
            i += 1
        else:                           # nums[i] == 2
            nums[i], nums[p2] = nums[p2], nums[i]
            p2 -= 1                     # i 不移动！换过来的元素还没检查
```

复杂度分析

时间复杂度 O(n)：每个元素最多被交换一次、检查一次。空间复杂度 O(1)：原地交换。

核心知识点

- 荷兰国旗问题（三路划分）：用三个指针把数组分成「小于 / 等于 / 大于」三段的经典算法。
- i 不动的原因：从右侧换过来的元素从未被检查，必须重新判断；这是本题最大的坑。
- 前缀/后缀边界的定义：[0,p0) = 0，(p2, n-1] = 2，中间是 1。
- 计数排序法（对照）：先数出 0、1、2 的个数再重写数组，需要遍历两趟。
- 应用延伸：三路划分也是「快速排序处理大量重复元素」时的优化手段。

31. 下一个排列 【中等】

题目描述

整数数组的一个排列就是将其所有成员以序列或线性顺序排列。例如，arr = [1,2,3]，以下这些都可以视作 arr 的排列：[1,2,3]、[1,3,2]、[3,1,2]、[2,3,1]。整数数组的下一个排列是指其整数的下一个字典序更大的排列。更正式地，如果数组的所有排列根据其字典顺序从小到大排列在一个容器中，那么数组的下一个排列就是在这个有序容器中排在它后面的那个排列。如果不存在下一个更大的排列，那么这个数组必须重排为字典序最小的排列（即，其元素按升序排列）。必须原地修改，只允许使用额外常数空间。

示例：

```
输入：nums = [1,2,3]
输出：[1,3,2]
输入：nums = [3,2,1]
输出：[1,2,3]
```

解题思路

核心思路：为了让字典序「变大得尽可能少」，应该从右往左找到第一个「可以变大的位置」，然后把一个「刚好比它大的数」换上来，最后把后面部分调整为最小（升序）。

具体步骤（四步）：

1. **从右向左找第一个下降点 i**：即找满足 nums[i] < nums[i+1] 的最大下标 i。如果不存在（整个数组是降序，例如 [3,2,1]），说明已经是最大排列，直接反转整个数组并返回。

2. **从右向左找第一个大于 nums[i] 的数 j**：由于 i 右侧是降序的，从右往左找第一个大于 nums[i] 的元素 j，它就是「恰好比 nums[i] 大的最小元素」。

3. **交换 nums[i] 和 nums[j]**。

4. **反转 i+1 到末尾的部分**：交换后，i 右侧仍然是降序的，把它反转为升序，就得到了「最小的后缀」。

举例：nums = [1,5,8,4,7,6,5,3,1]

1. 从右往左找第一个下降点：4 < 7，所以 i 指向 4（下标 3）。

2. 从右往左找第一个 > 4 的数：5（下标 6）。

3. 交换后：[1,5,8,5,7,6,4,3,1]。

4. 反转 i 之后的部分（原为 [7,6,4,3,1]，反转为 [1,3,4,6,7]）。

结果：[1,5,8,5,1,3,4,6,7]。这就是下一个排列。

参考代码

```
def nextPermutation(nums):
    n = len(nums)
    # 1. 从右向左找第一个下降点 i
```

```
    i = n - 2
    while i >= 0 and nums[i] >= nums[i + 1]:
        i -= 1
    if i >= 0:
        # 2. 从右向左找第一个大于 nums[i] 的位置 j
        j = n - 1
        while nums[j] <= nums[i]:
            j -= 1
        # 3. 交换
        nums[i], nums[j] = nums[j], nums[i]
    # 4. 反转 i+1 到末尾 (若 i<0 则整个数组反转，得到最小排列)
    l, r = i + 1, n - 1
    while l < r:
        nums[l], nums[r] = nums[r], nums[l]
        l += 1
        r -= 1
    return nums
```

复杂度分析

时间复杂度 O(n)：最多三次线性扫描。空间复杂度 O(1)：原地操作。

核心知识点

- 字典序的定义与生成算法：理解「下一个排列」的手工推导过程，是掌握本题的关键。
- 「找下降点」的含义：只要后缀是降序，就说明后缀已是最大排列，必须改前面的位置。
- 「找恰好更大的数」：因为 i 右侧是降序，从右往左第一个大于 nums[i] 的元素就是「最小的更大值」。
- 最后的反转：把降序后缀变为升序，得到最小的后缀，保证整体是「下一个」而不是「跳过若干个」。
- 「不存在下降点」的处理：整个数组降序时说明已是最后一个排列，反转得到最小排列。
- 这是 C++ STL 中 std::next_permutation 的实现原理，也是回溯法「按字典序枚举排列」的底层工具。

287. 寻找重复数 【中等】

题目描述

给定一个包含 n + 1 个整数的数组 nums，其数字都在 [1, n] 范围内（包括 1 和 n），可知至少存在一个重复的整数。假设 nums 只有一个重复的整数，返回这个重复的数。你设计的解决方案必须不修改数组 nums 且只用常量级 O(1) 的额外空间。

示例：

```
输入：nums = [1,3,4,2,2]
输出：2
```

解题思路

题目限制很严格：不能修改数组、O(1) 空间、比 O(n²) 快。巧妙解法是把它转化为「链表找环入口」问题。

核心转化：把数组视为「指针」——下标 i 指向 nums[i]。由于 nums 中的值都在 [1, n] 范围内，从下标 0 出发：0 → nums[0] → nums[nums[0]] → ...因为存在重复的数，这个「跳跃」过程必然会进入一个环，而环的入口正好就是重复的那个数！

具体步骤（复用「环形链表 II」的解法）：

1. 用快慢指针找相遇点：slow 每次走 1 步（slow = nums[slow]），fast 每次走 2 步（fast = nums[nums[fast]]）。

2. 相遇后，让 slow 回到起点 0，然后 slow 和 fast 每次都走 1 步，直到再次相遇。

3. 相遇的位置就是环的入口，也就是重复的数。

其他解法对比：

- 二分查找（值域二分）：在 [1,n] 中二分，统计 <= mid 的元素个数，大于 mid 说明重复数在左半。O(n log n) / O(1)。

- 哈希表：O(n) 空间，不满足空间要求。

- 排序 / 标记负号：修改了数组，不满足要求。

参考代码

```
def findDuplicate(nums):
    # 快慢指针找环入口 (等价于环形链表 II)
    slow = fast = 0
    while True:
        slow = nums[slow]                # 走 1 步
        fast = nums[nums[fast]]          # 走 2 步
        if slow == fast:
            break
    slow = 0                             # 一个回到起点
```

```
    while slow != fast:
        slow = nums[slow]
        fast = nums[fast]
    return slow                             # 环入口 = 重复的数

# 方法二: 值域二分 (O(n log n) / O(1))
def findDuplicate2(nums):
    lo, hi = 1, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        cnt = sum(1 for x in nums if x <= mid)
        if cnt > mid:          # 左半部分元素过多 -> 重复数在左半
            hi = mid
        else:
            lo = mid + 1
    return lo
```

复杂度分析

快慢指针：时间 O(n)，空间 O(1)。值域二分：时间 O(n log n)，空间 O(1)。

核心知识点

- 「数组当链表」的转化：下标 i 指向 nums[i]，把数组问题转化为图/链表问题，是极具创造性的思路。
- 鸽巢原理：n+1 个数放在 [1,n] 范围内，必有重复，这是问题有解的根本保证。
- Floyd 判圈算法的复用：与「环形链表 II」完全相同的代码，体现了「算法模型迁移」的能力。
- 值域二分：当「数组索引」和「元素值」范围不同时，可以在「值域」上二分，是一个重要的二分变体。
- 约束驱动解法：「不修改数组 + O(1) 空间」这两个限制，直接排除了排序和哈希，逼出快慢指针或值域二分。